

AI 500

Foundations of

Artificial Intelligence

L6: Supervised Learning

By Murtaza Taj



Types of Learning

Artificial Intelligence

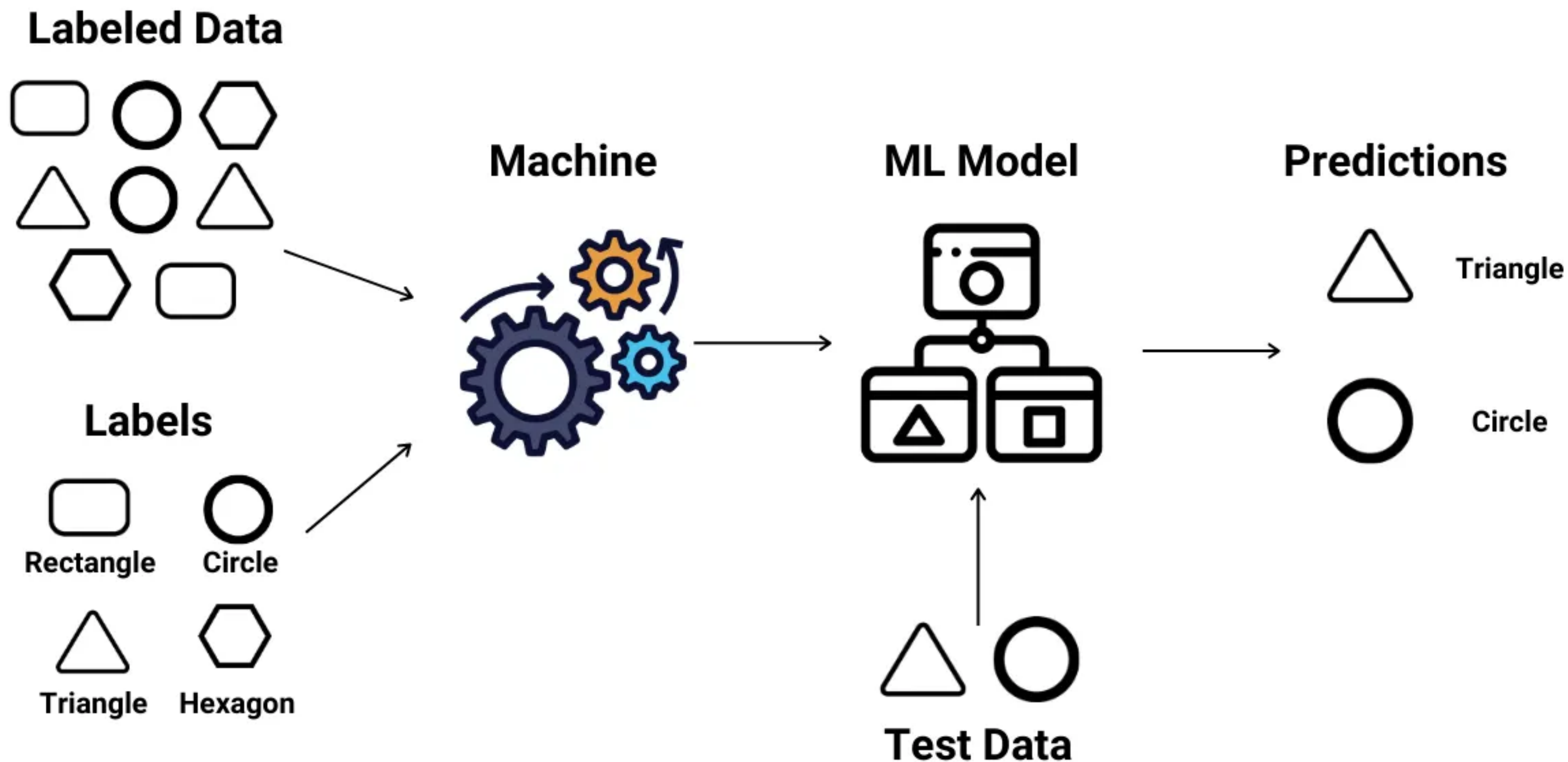
Machine Learning

Supervised
Learning

Unsupervised
Learning

Reinforcement
Learning

Supervised Learning

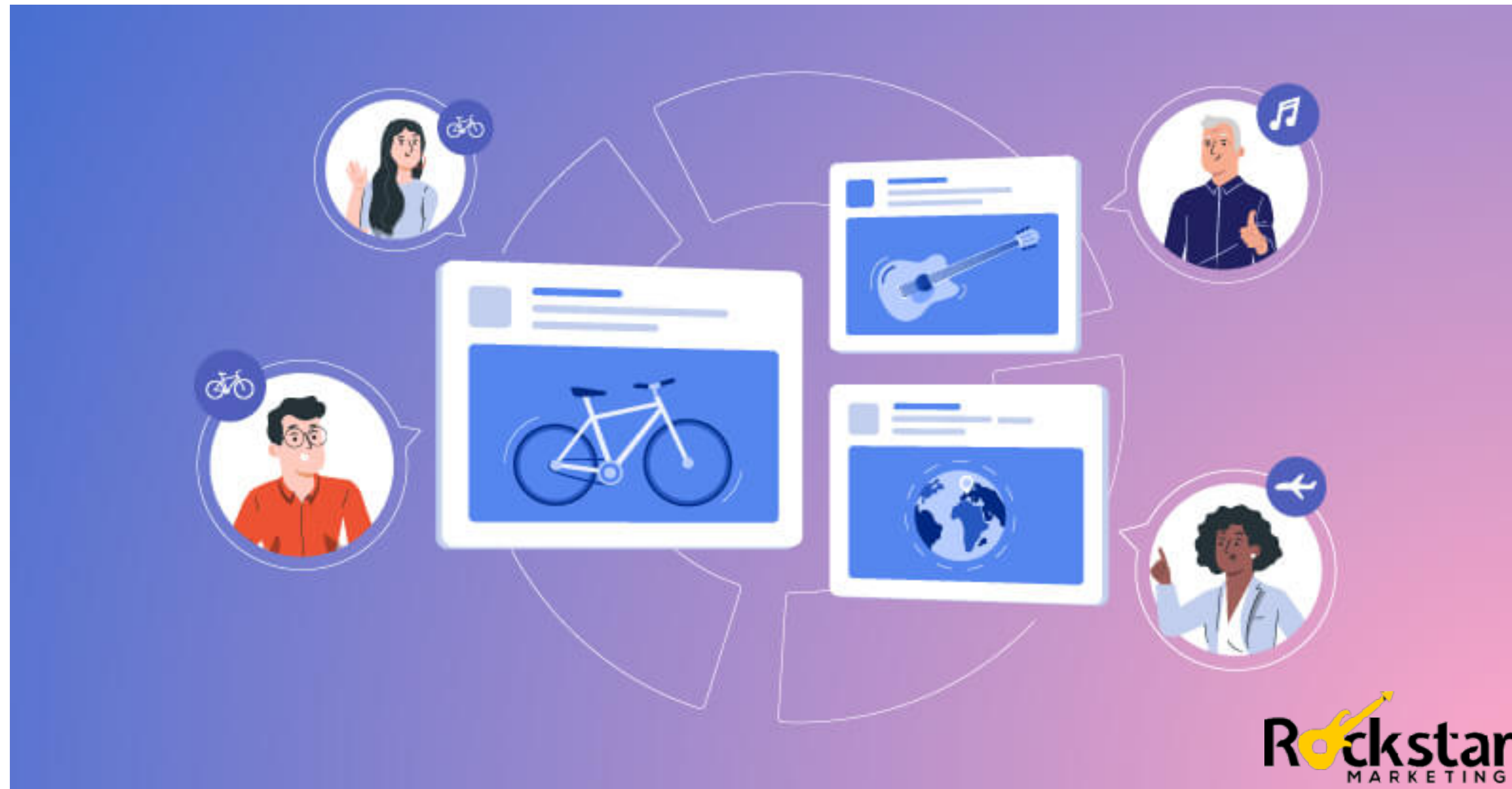


Examples

Supervised Learning Example

Targeted Advertisement

- Based on customer demographics, browsing history, buying patterns you decide whether to show certain Ad to a customer or not.



Supervised Learning Example

Credit Card Fraudulent Transaction Detection



- User transactions history and card holders characteristics - fair/fraud

										Tickets	Pass/G.C.	Donation	Fees	Order Notes	Transactions	Web Log	Letters		
Log # ¹ ^	DateTime	Remote Ip	Message Type	Cart #	First Name	Last Name	Elapsed	Message	Patron #										
53951	2016/12/22 14:43:51.92	127.0.0.1	Online	221	Elizabeth	Holley	581 ms	Get best available.. Looking for 2 seats together, anywhere,...	76										
53952	2016/12/22 14:43:54.40	127.0.0.1	Online	221	Elizabeth	Holley	298 ms	Reserve Tickets	76										
53953	2016/12/22 14:43:55.10	127.0.0.1	Online	221	Elizabeth	Holley	693 ms	Shopping Cart	76										
53954	2016/12/22 14:43:58.24	127.0.0.1	Online	221	Elizabeth	Holley	619 ms	Checkout with Payment	76										
53966	2016/12/22 14:57:59.52	127.0.0.1	Online	221	Elizabeth	Holley	505 ms	Payment Confirmation incomplete. Redirecting back to Chec...	76										
53967	2016/12/22 14:57:59.99	127.0.0.1	Online	221	Elizabeth	Holley	476 ms	Checkout with Payment	76										
53968	2016/12/22 14:58:10.81	127.0.0.1	Online	221	Elizabeth	Holley	398 ms	Payment Confirmation incomplete. Redirecting back to Chec...	76										
53969	2016/12/22 14:58:11.18	127.0.0.1	Online	221	Elizabeth	Holley	374 ms	Checkout with Payment	76										
53970	2016/12/22 14:58:27.95	127.0.0.1	Online	221	Elizabeth	Holley	976 ms	Payment Confirmation ready for Authorization. Card: **** *...	76										
53971	2016/12/22 14:58:30.76	127.0.0.1	Online	221	Elizabeth	Holley	391 ms	Checkout Process Payment	76										
53972	2016/12/22 14:58:33.51	127.0.0.1	Online	221	Elizabeth	Holley	3147 ...	Proxied POST: http://127.0.0.1:5110/TheatreManager/1/on...	76										
53973	2016/12/22 14:58:30.95	127.0.0.1	*ccAuth	221	Elizabeth	Holley	133 ms	1-begin cart checkout and credit card processing - cart is st...	76										
53974	2016/12/22 14:58:30.96	127.0.0.1	*ccAuth	221	Elizabeth	Holley	150 ms	2-locked cart successfully prior to beginning checkout proc...	76										
53975	2016/12/22 14:58:31.02	127.0.0.1	*ccAuth	221	Elizabeth	Holley	200 ms	3-preparing to send credit card info to Orbital Test Account	76										
53976	2016/12/22 14:58:31.33	127.0.0.1	***ccAuth wa...	221	Elizabeth	Holley	517 ms	3c-EMERGENCY MODE - Card#: **** 4242, Expiry: 12/2016 ...	76										
53977	2016/12/22 14:58:32.25	127.0.0.1	*ccAuth	221	Elizabeth	Holley	1433 ...	4-Completed Authorization Process. Auth #	76										
53978	2016/12/22 14:58:32.61	127.0.0.1	*ccAuth	221	Elizabeth	Holley	1800 ...	5-Cart processed successfully	76										
53979	2016/12/22 14:58:32.66	127.0.0.1	*ccAuth	221	Elizabeth	Holley	1833 ...	6-building confirmation email for order #380	76										
53980	2016/12/22 14:58:33.52	127.0.0.1	*Completion	221	Elizabeth	Holley	2700 ...	Request Completed	76										



Supervised Learning Example

Churn Prediction: which customers are likely to cancel a subscription

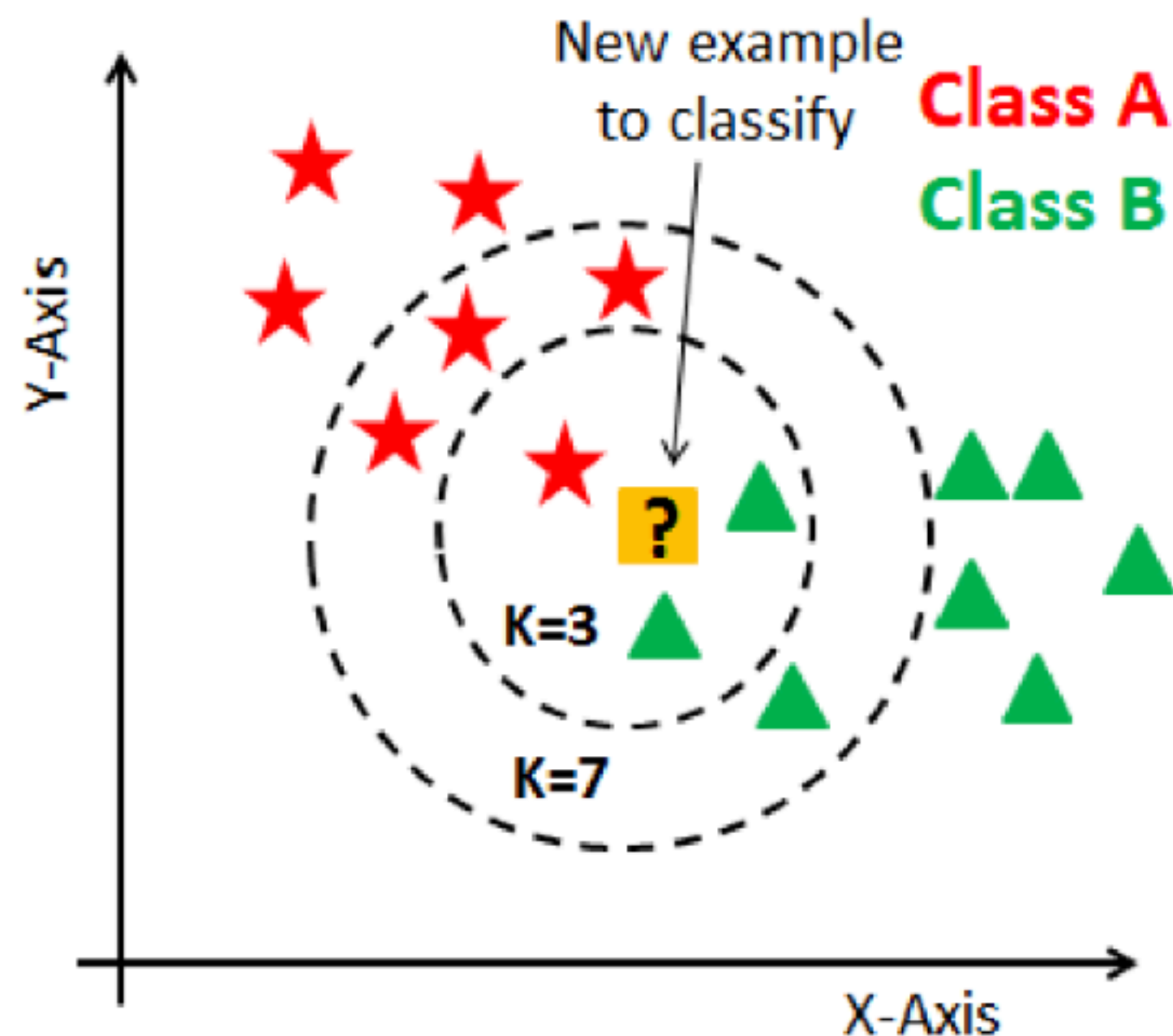
- Targetted Marketing



k-Nearest Neighbor (kNN) Classifier

k-Nearest Neighbor (kNN) Classifier

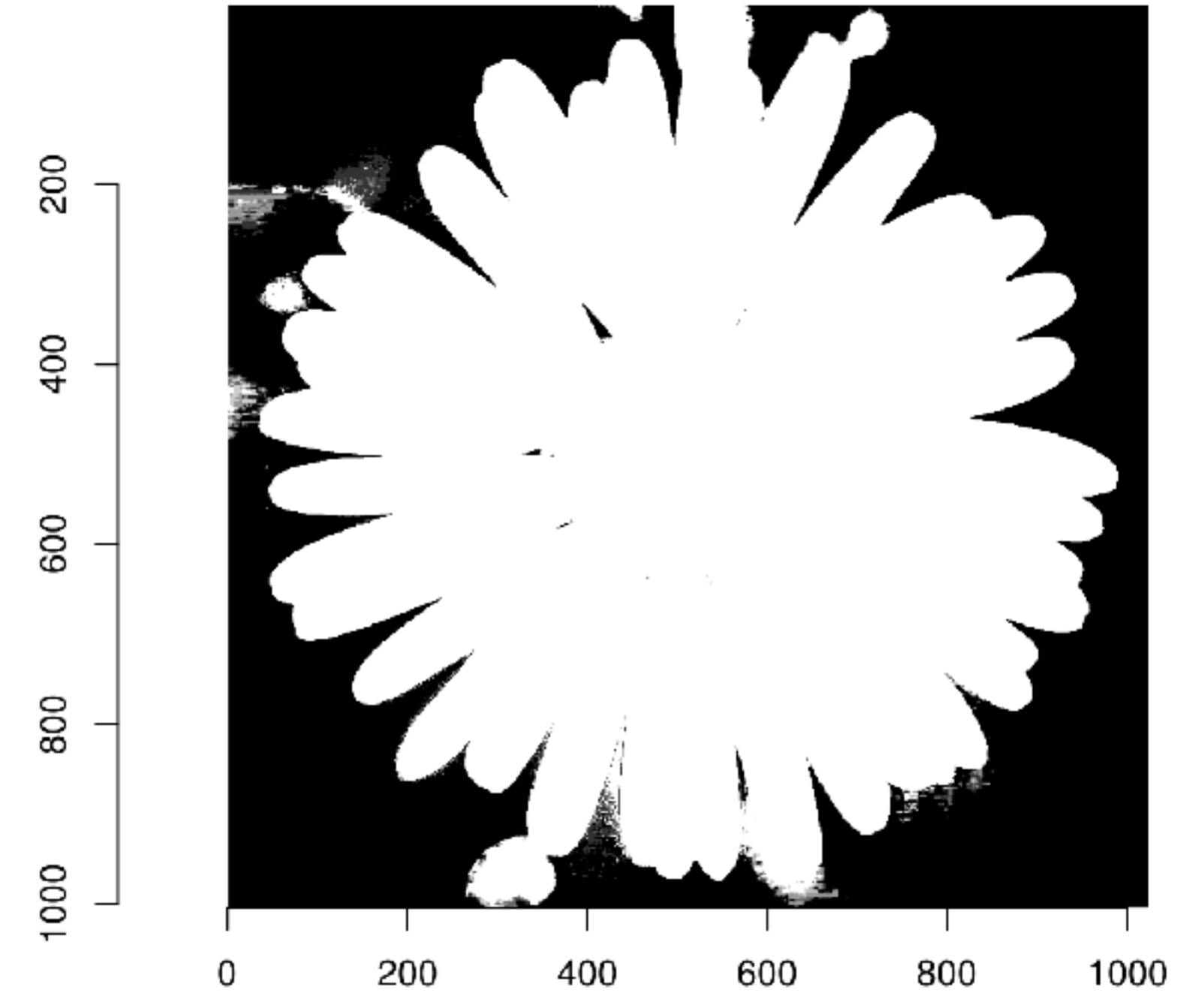
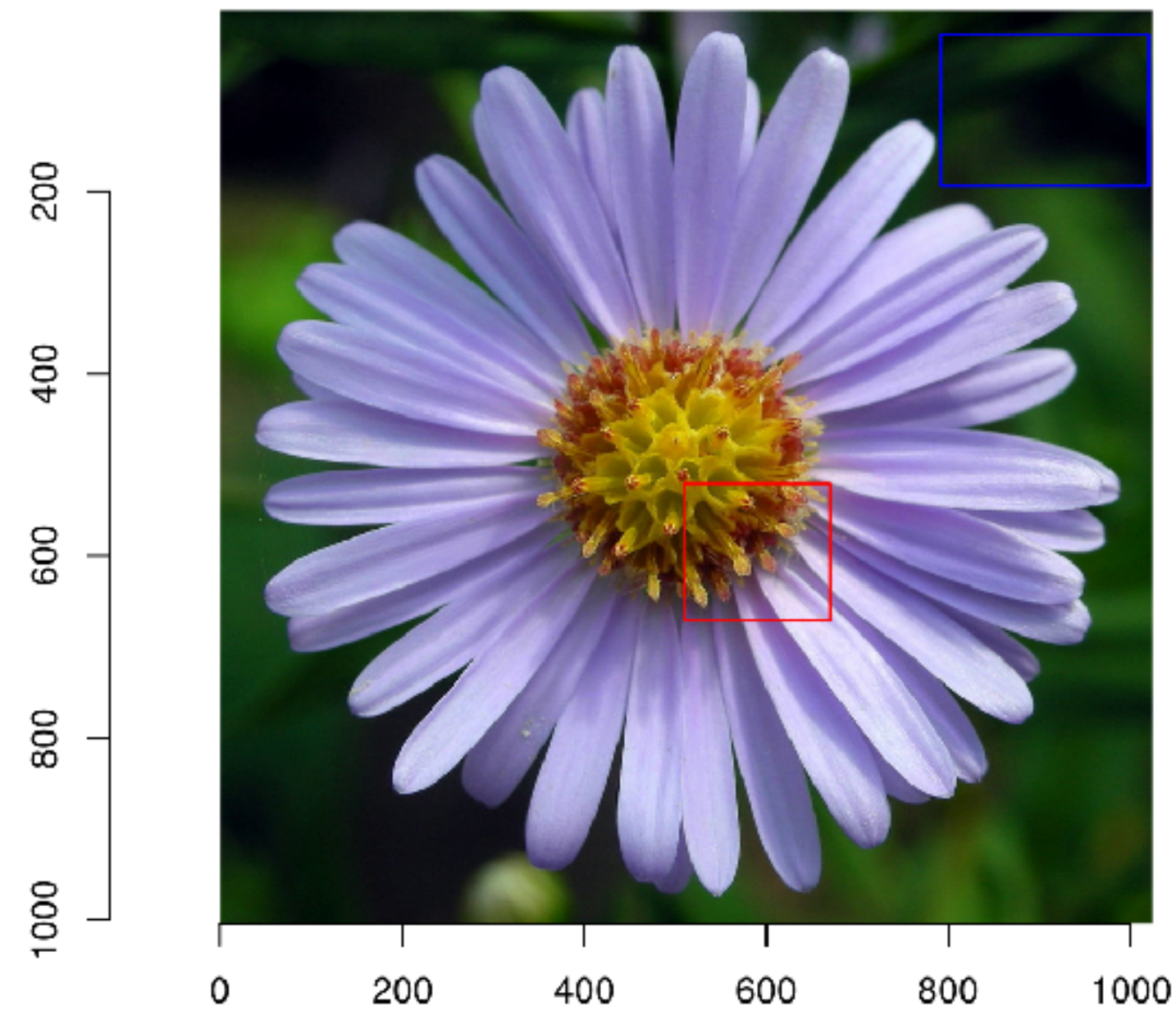
- The class label of a test instance x is predicted to be the most common class among the k nearest neighbors of x in the train set



- Assign the test instance (?) class A (★) or class B (▲)
- $k = 3$ nearest neighbors (l_2 distance) 1★ and 2▲ $\Rightarrow$ assigned label= ▲
- $k = 7$ nearest neighbors (l_2 distance) 4★ and 3▲ $\Rightarrow$ assigned label= ★

k-Nearest Neighbor (kNN) Classifier

Foreground/background segmentation



k-Nearest Neighbor (kNN) Classifier

Python Implementation



```
function euclidean_distance (x1, x2)
    return sqrt(sum((x1-x2)^2))

function init (k)

function fit (X_train, y_train)
# copy data and do nothing
self.X_train = X_train
self.y_train = y_train

function predict (X_test)
# compute distance b/w test and all training samples
for each training sample
    d[i] = euclidean_distance(x_test, x_train[I])

# find closest k=2 samples
ds,idx = sort(d) # [32 11 7 64 9 99] -> [7 9 11 32 64 99]
idx_k = idx[:k]

# majority voting
return most_common(y_train[idx_k])
```

k-Nearest Neighbor (kNN) Classifier

Python Implementation



```
function init (k)
```

```
function fit (X_train, y_train)
```

```
# just copy the data, do nothing
```

```
function predict (X_test)
```

```
    for each input sample
```

```
        for each training sample
```

```
            # compute distance b/w test & train data
```

```
            d_i = euclidean_distance(test_i, train_i)
```

```
        # find k closest training samples
```

```
        ds = sort (d)
```

```
        k_indicies = foo(ds,k)
```

```
        # majority voting
```

```
        labels = y_train [k_indicies]
```

```
function euclidean_distance (x1, x2)
```

```
    d = sqrt(sum((x1-x2)^2))
```

```
    return d
```

Decision Tree

Readings

Section 19.1, 19.2, 19.3 Russel & Norvig 2nd Ed.

Ch3: Introduction to Data Mining (2nd Ed.), Tan, Steinbach, Karpante, Kumar

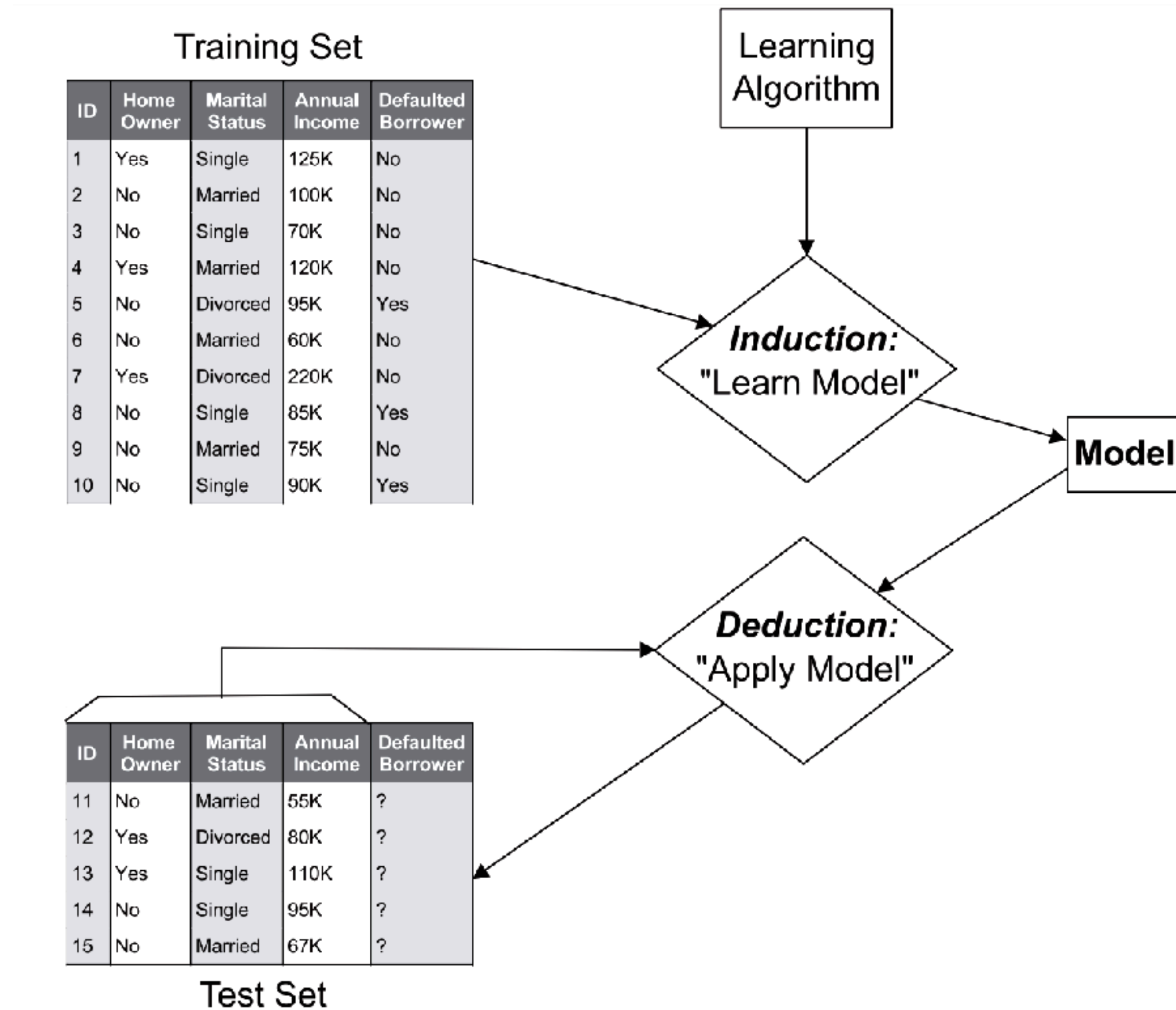
Supervised Learning Example

Loan Borrower Classification Problem

ID	Home Owner	Marital Status	Annual Income	Defaulted?
1	Yes	Single	125000	No
2	No	Married	100000	No
3	No	Single	70000	No
4	Yes	Married	120000	No
5	No	Divorced	95000	Yes
6	No	Single	60000	No
7	Yes	Divorced	220000	No
8	No	Single	85000	Yes
9	No	Married	75000	No
10	No	Single	90000	Yes

Supervised Learning

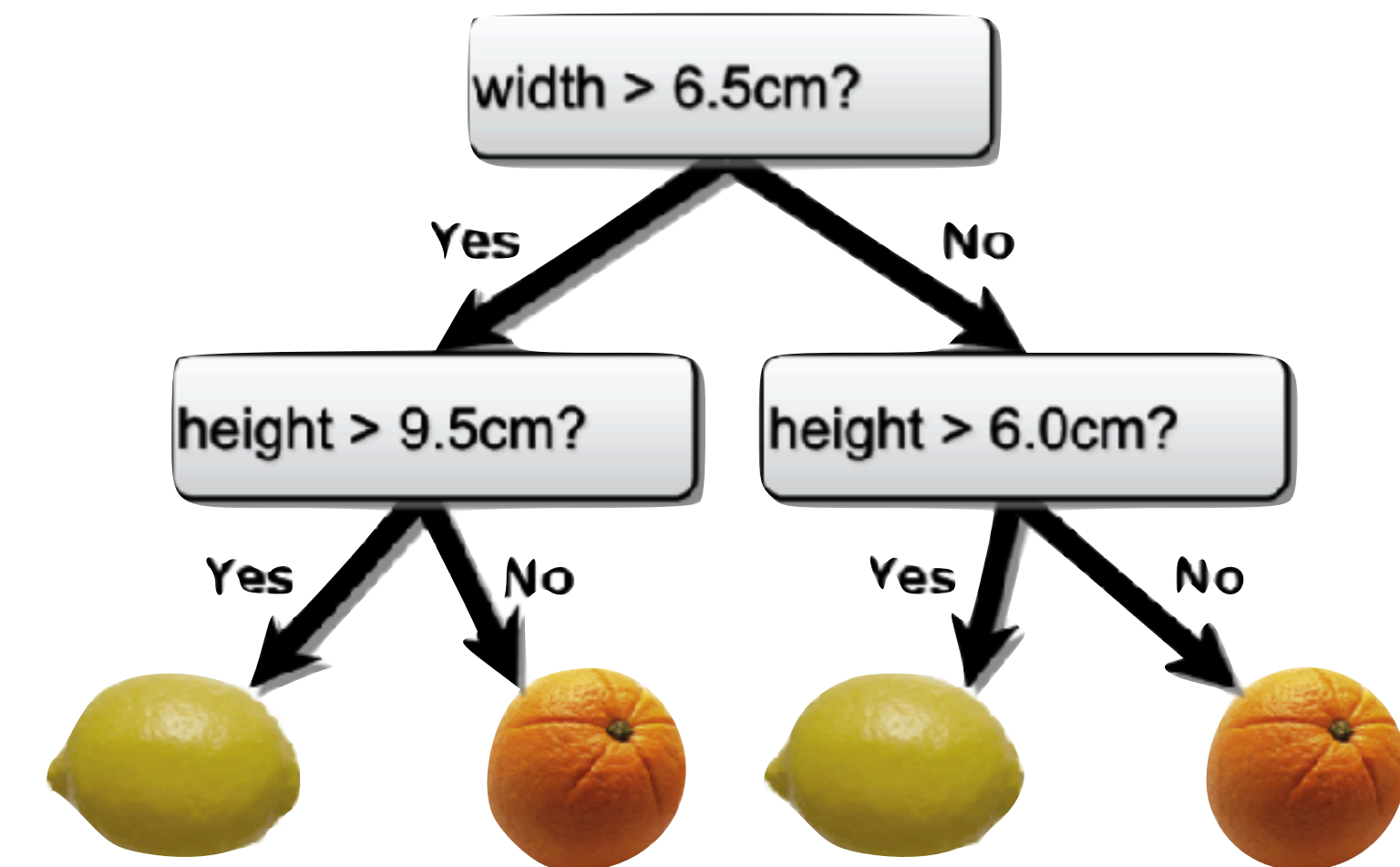
General Framework



Decision Tree

Example 1

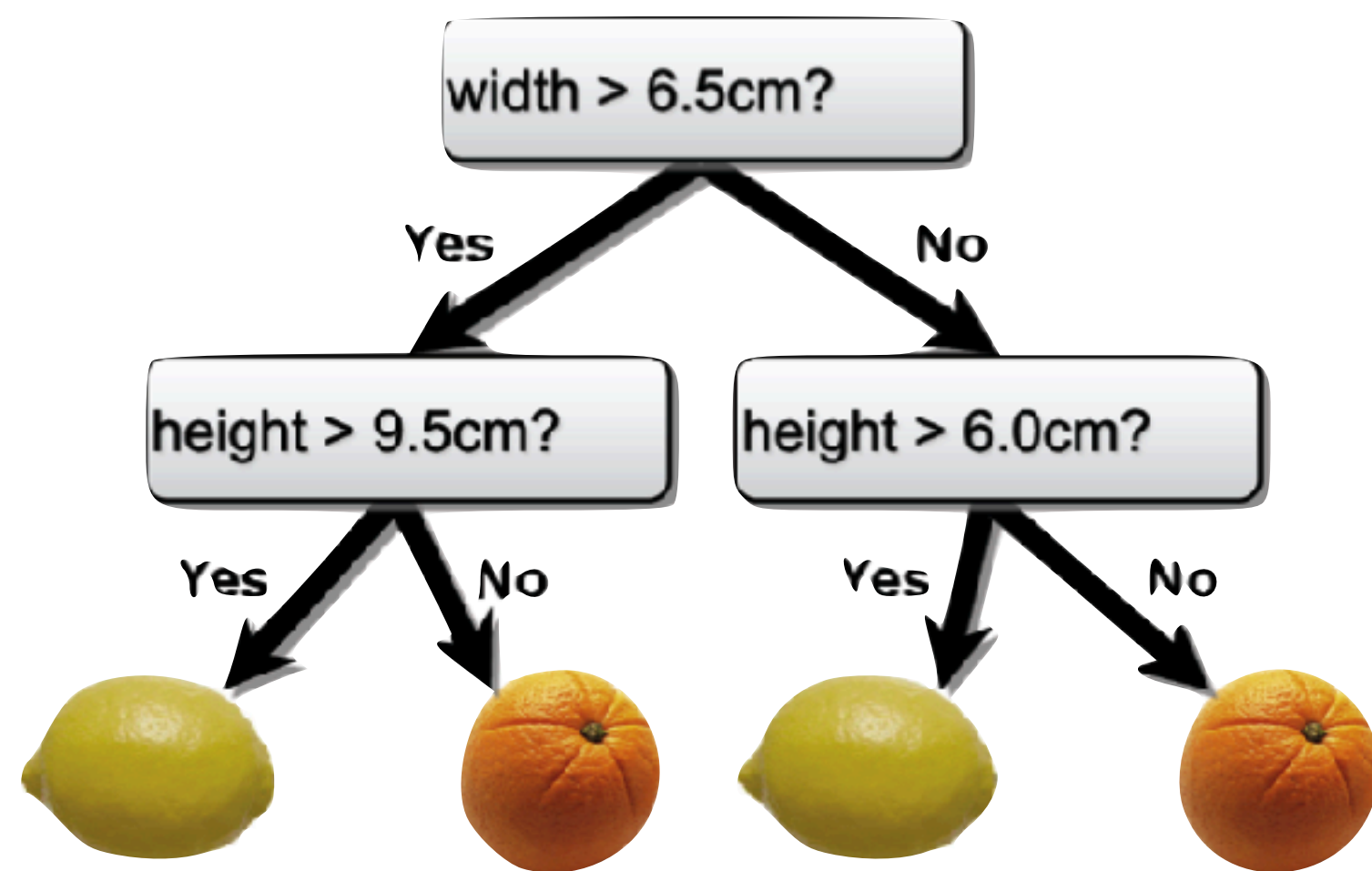
- Fundamentally, an **if-then** rule set for classifying objects
- Builds model in the form of a tree structure
- Each internal node tests an feature x_i
- Branches correspond to possible (subsets of) values of the feature x_i
- Each leaf node assigns a class label y



Zemel, Ustasun, Fidler @ Uni, of Toronto

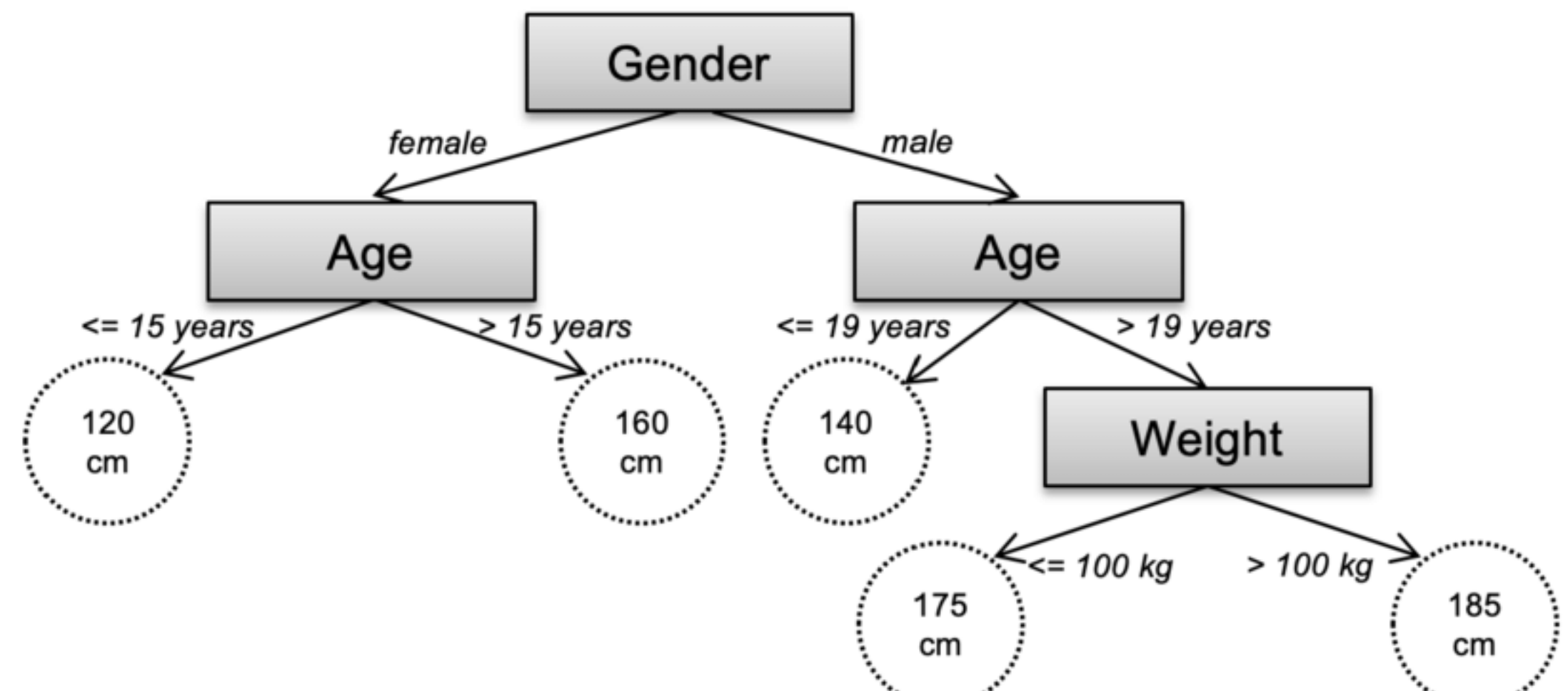
Decision Tree

Classification vs. Regression



Zemel, Ustasun, Fidler @ Uni. of Toronto

Classification Tree



Eero Hiltunen

Regression Tree

Decision Tree

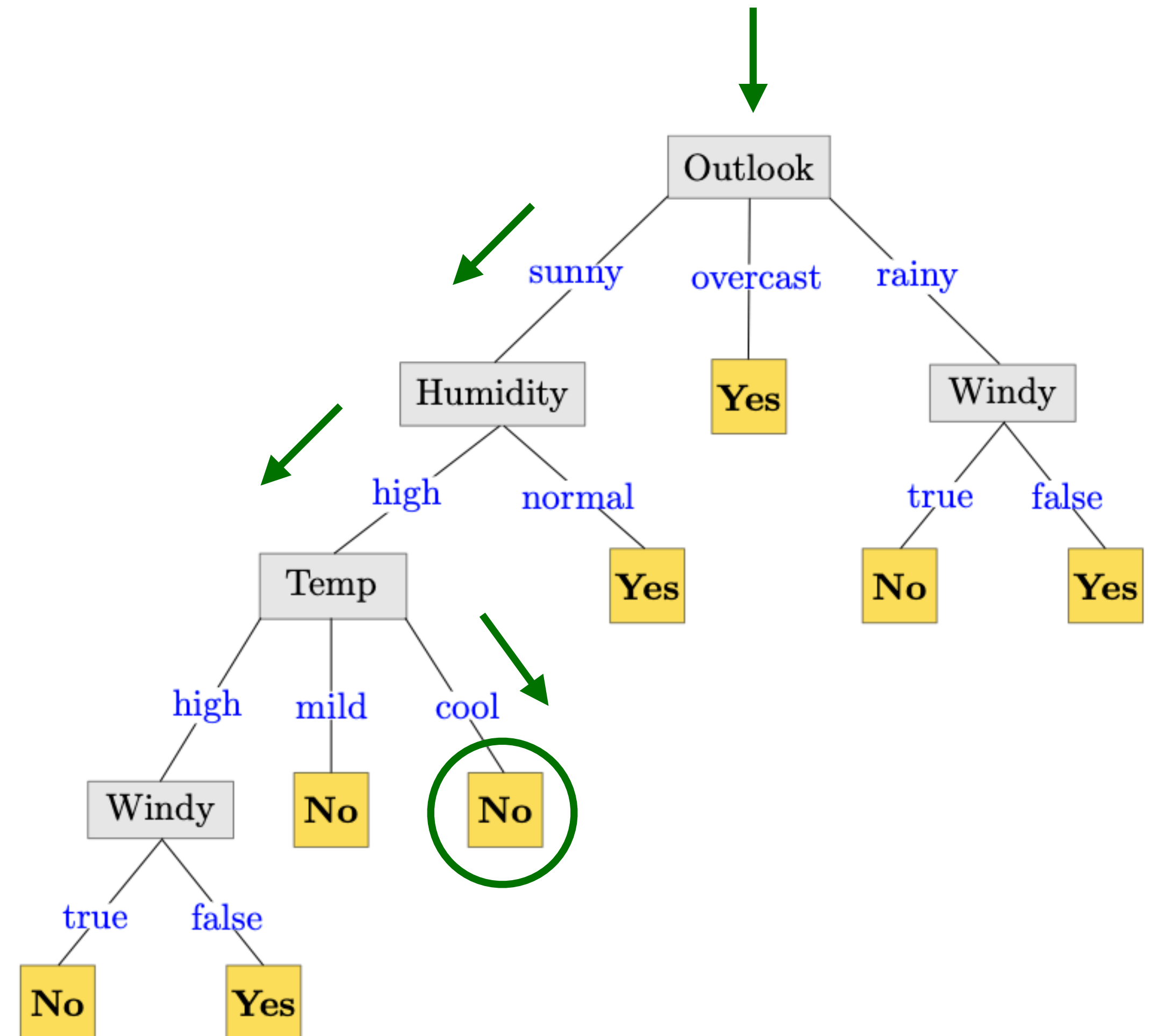
Example 2

Given weather condition

$\mathbf{x} = (\text{sunny, cool, high, true})$

will game be played?

- To classify a test instance $\mathbf{x}$ traverse the tree from root to leaf
- Take branches at internal nodes according to results of their tests
- Predict the class label at the leaf node reached

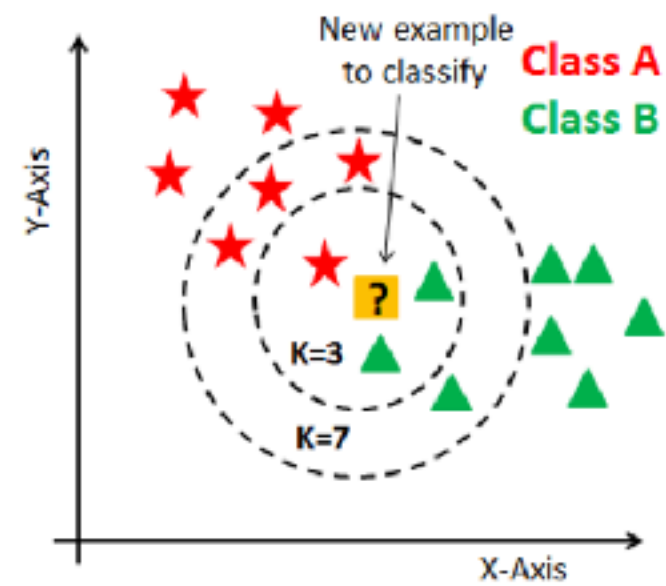


Recap: Supervised Learning

k-Nearest Neighbor (kNN) Classifier



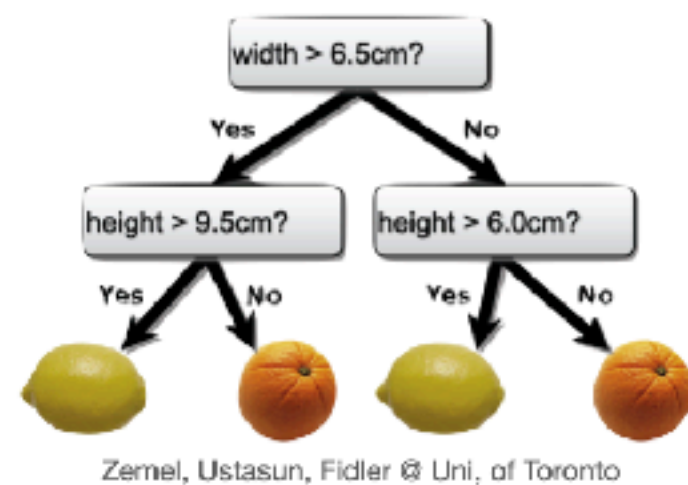
- The class label of a test instance x is predicted to be the most common class among the k nearest neighbors of x in the train set



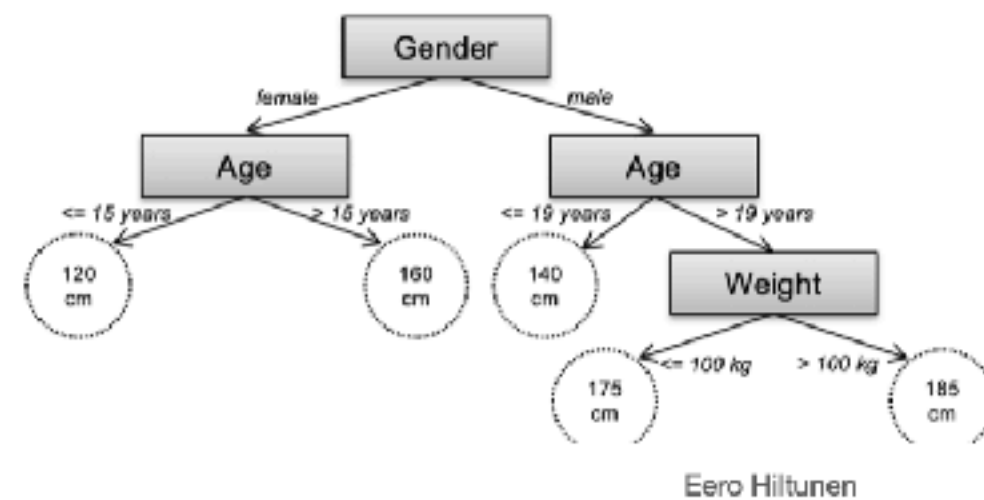
- Assign the test instance (?) class A (★) or class B (▲)
- $k = 3$ nearest neighbors (l_2 distance) 1★ and 2▲ $\Rightarrow$ assigned label= ▲
- $k = 7$ nearest neighbors (l_2 distance) 4★ and 3▲ $\Rightarrow$ assigned label= ★

Decision Tree

Classification vs. Regression



Classification Tree



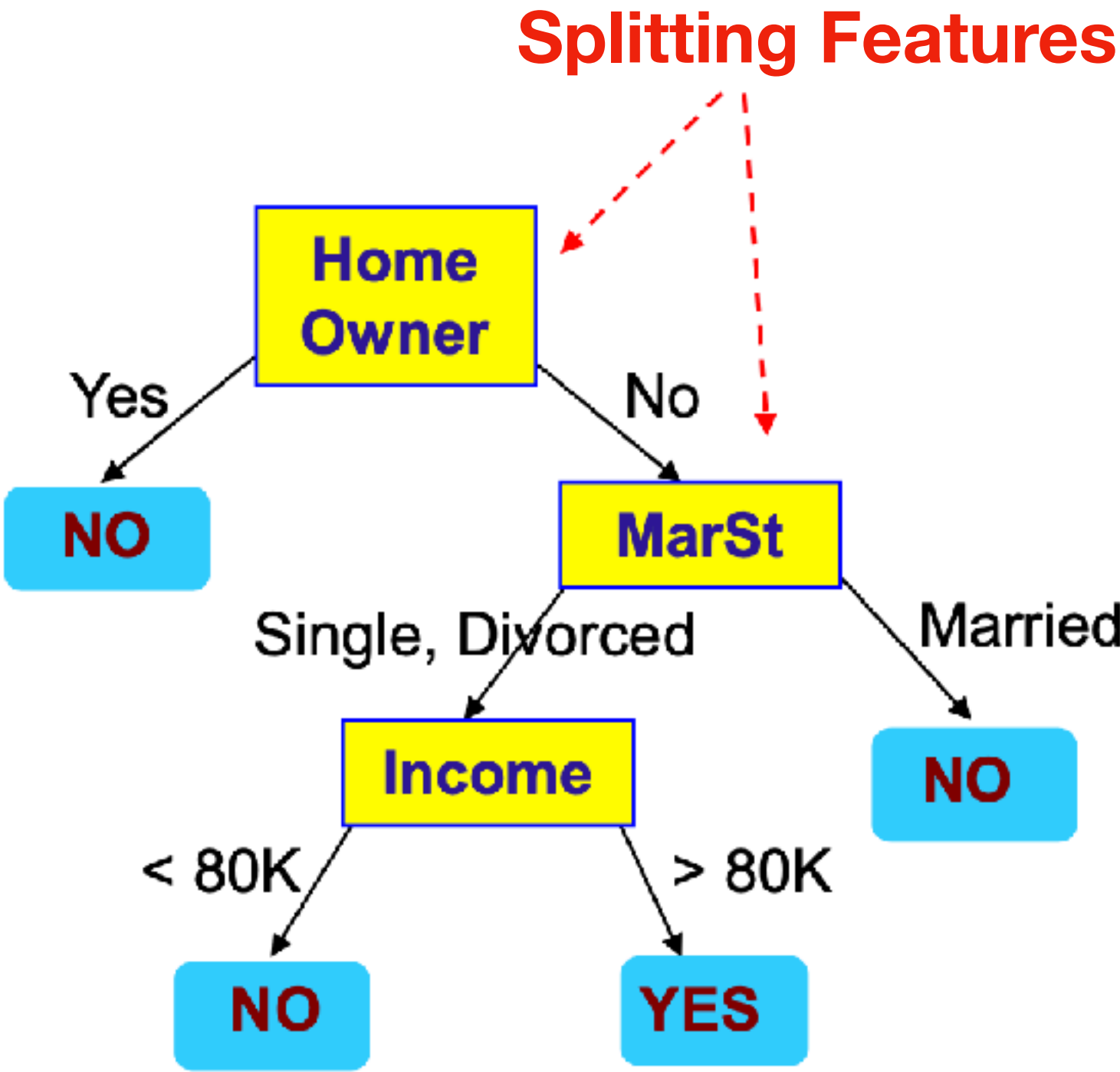
Regression Tree

Decision Tree

Example 3

ID	categorical		categorical		continuous	class
	Home Owner	Marital Status	Annual Income	Defaulted Borrower		
1	Yes	Single	125K	No		
2	No	Married	100K	No		
3	No	Single	70K	No		
4	Yes	Married	120K	No		
5	No	Divorced	95K	Yes		
6	No	Married	60K	No		
7	Yes	Divorced	220K	No		
8	No	Single	85K	Yes		
9	No	Married	75K	No		
10	No	Single	90K	Yes		

Training Data



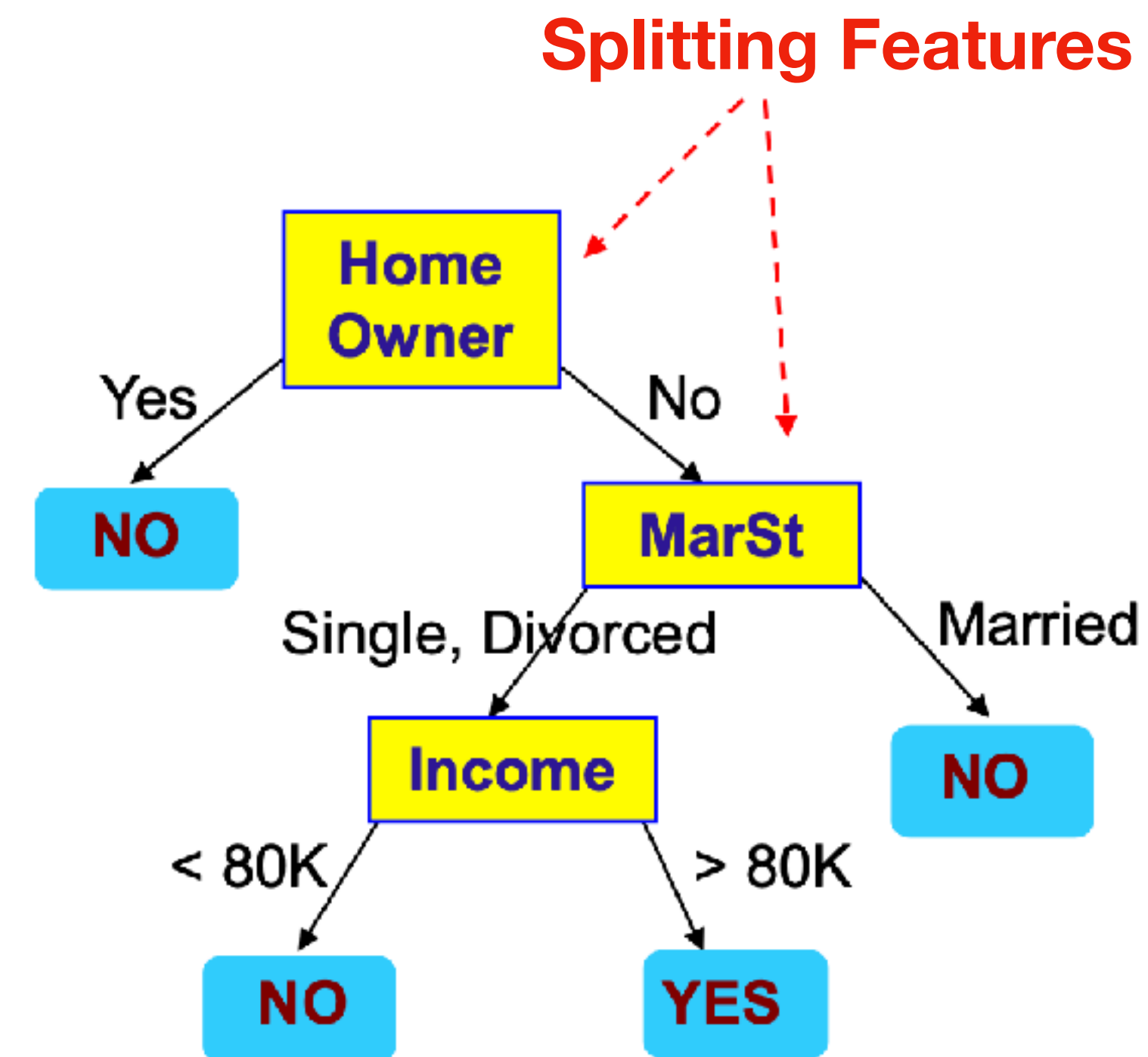
Model: Decision Tree

Decision Tree

Example 3

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

categorical
categorical
continuous
class



Training Data

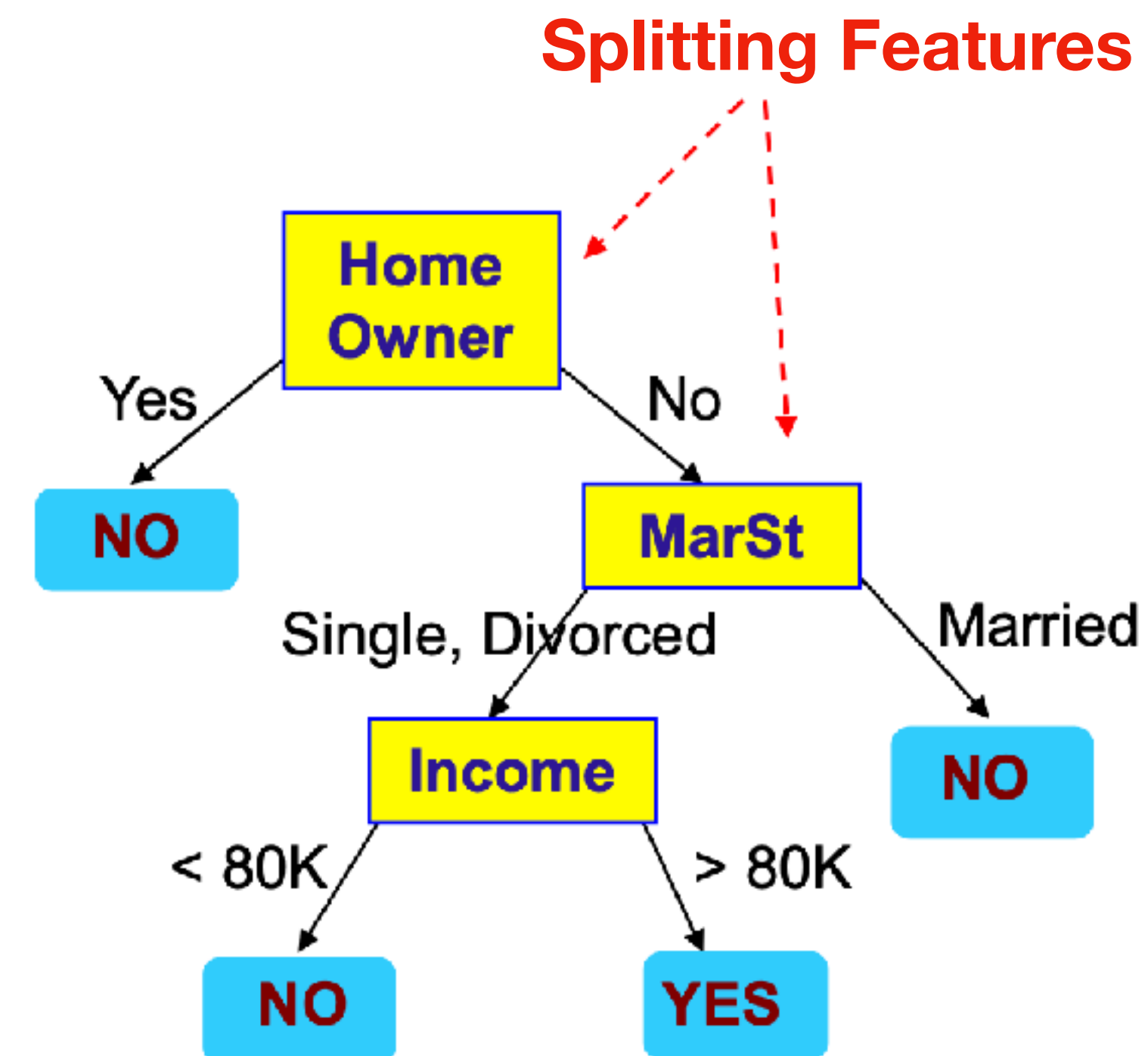
Model: Decision Tree

Decision Tree

Example 3

ID	categorical		categorical		continuous	class
	Home Owner	Marital Status	Annual Income	Defaulted Borrower		
1	Yes	Single	125K	No		
2	No	Married	100K	No		
3	No	Single	70K	No		
4	Yes	Married	120K	No		
5	No	Divorced	95K	Yes		
6	No	Married	60K	No		
7	Yes	Divorced	220K	No		
8	No	Single	85K	Yes		
9	No	Married	75K	No		
10	No	Single	90K	Yes		

Training Data



Model: Decision Tree

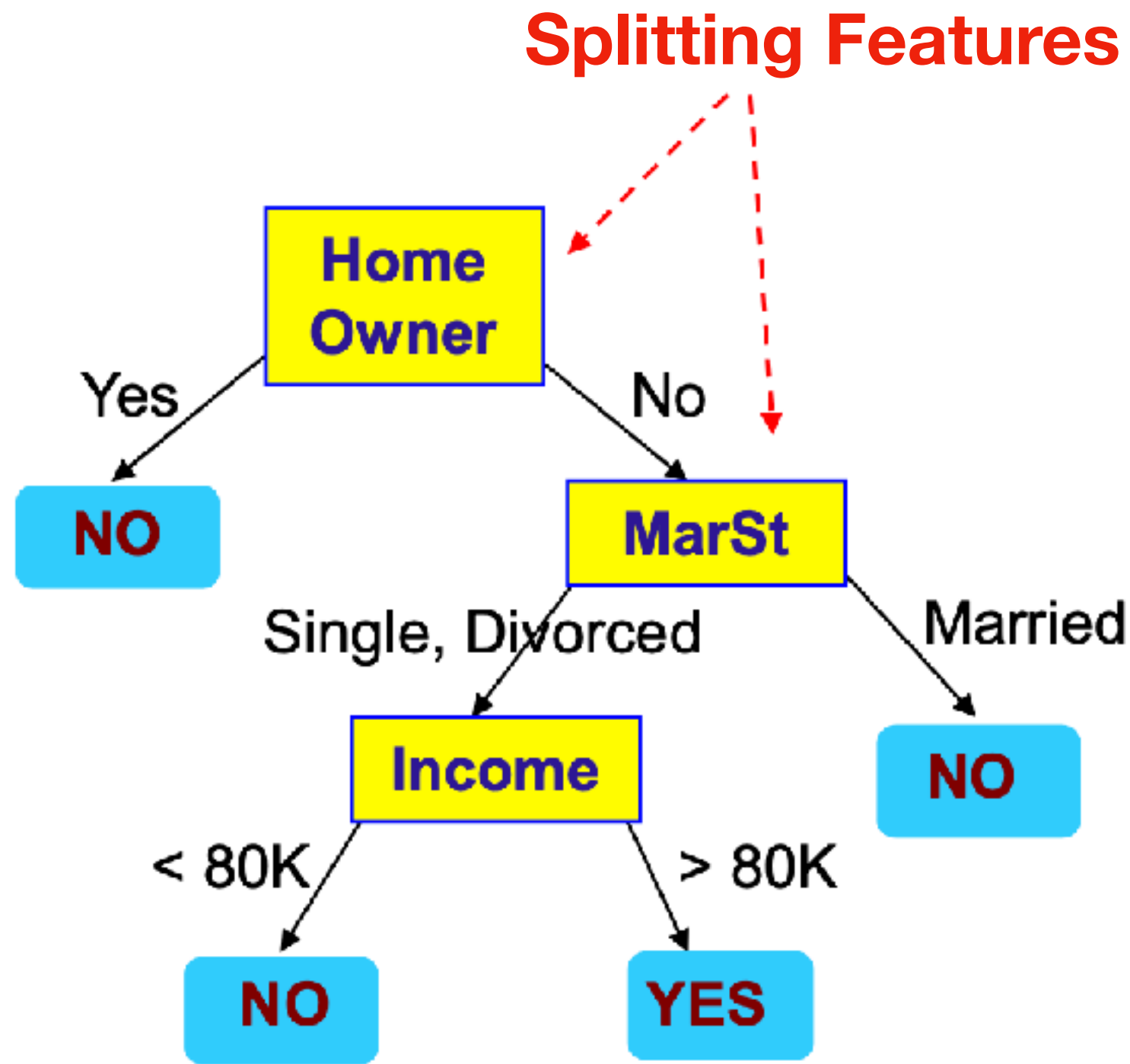
Decision Tree

Example 3



ID	categorical		categorical		continuous	class
	Home Owner	Marital Status	Annual Income	Defaulted Borrower		
1	Yes	Single	125K	No		
2	No	Married	100K	No		
3	No	Single	70K	No		
4	Yes	Married	120K	No		
5	No	Divorced	95K	Yes		
6	No	Married	60K	No		
7	Yes	Divorced	220K	No		
8	No	Single	85K	Yes		
9	No	Married	75K	No		
10	No	Single	90K	Yes		

Training Data



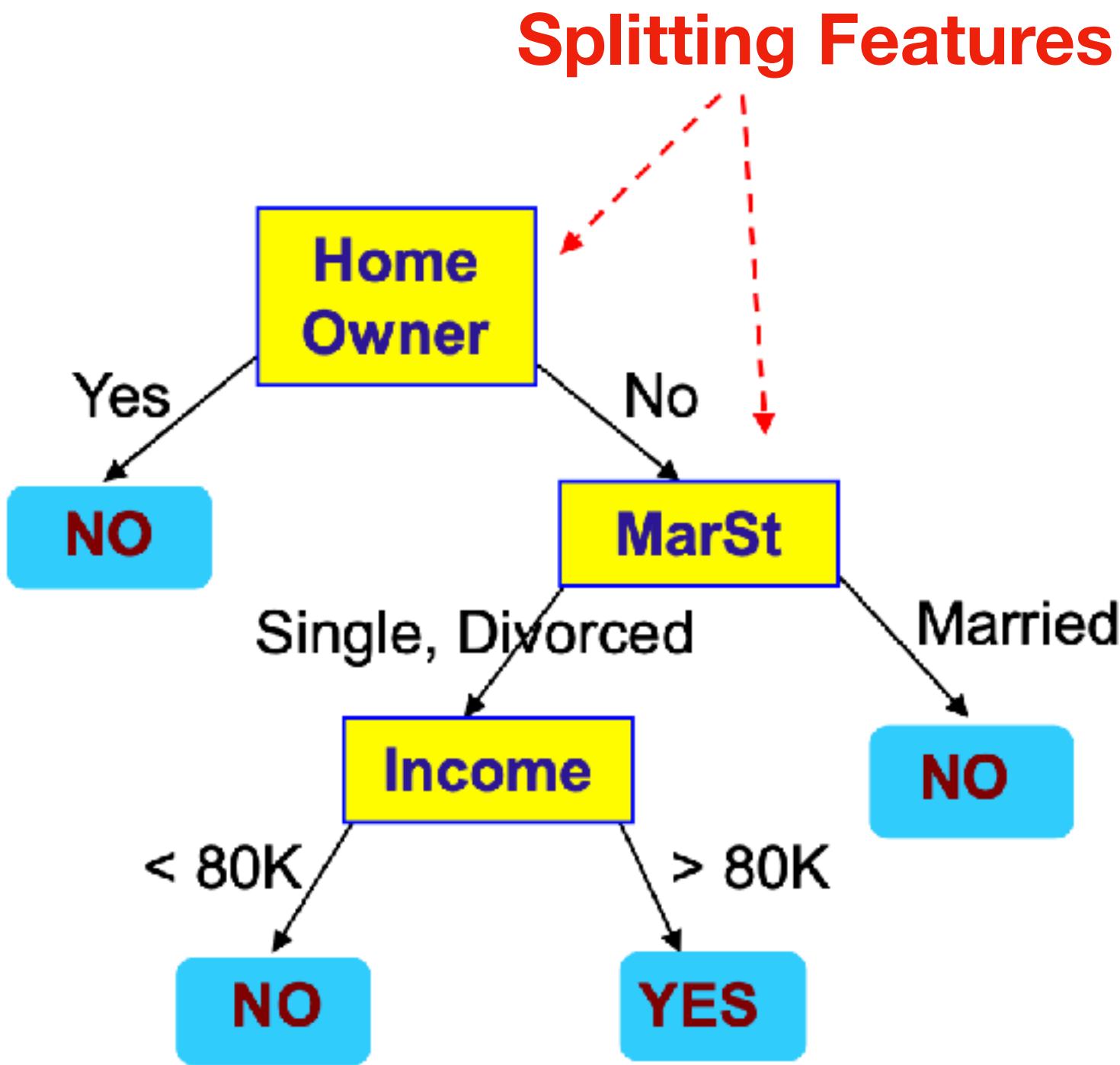
Model: Decision Tree

Decision Tree

Example 3

ID	categorical		categorical		continuous	class
	Home Owner	Marital Status	Annual Income	Defaulted Borrower		
1	Yes	Single	125K	No		
2	No	Married	100K	No		
3	No	Single	70K	No		
4	Yes	Married	120K	No		
5	No	Divorced	95K	Yes		
6	No	Married	60K	No		
7	Yes	Divorced	220K	No		
8	No	Single	85K	Yes		
9	No	Married	75K	No		
10	No	Single	90K	Yes		

Training Data

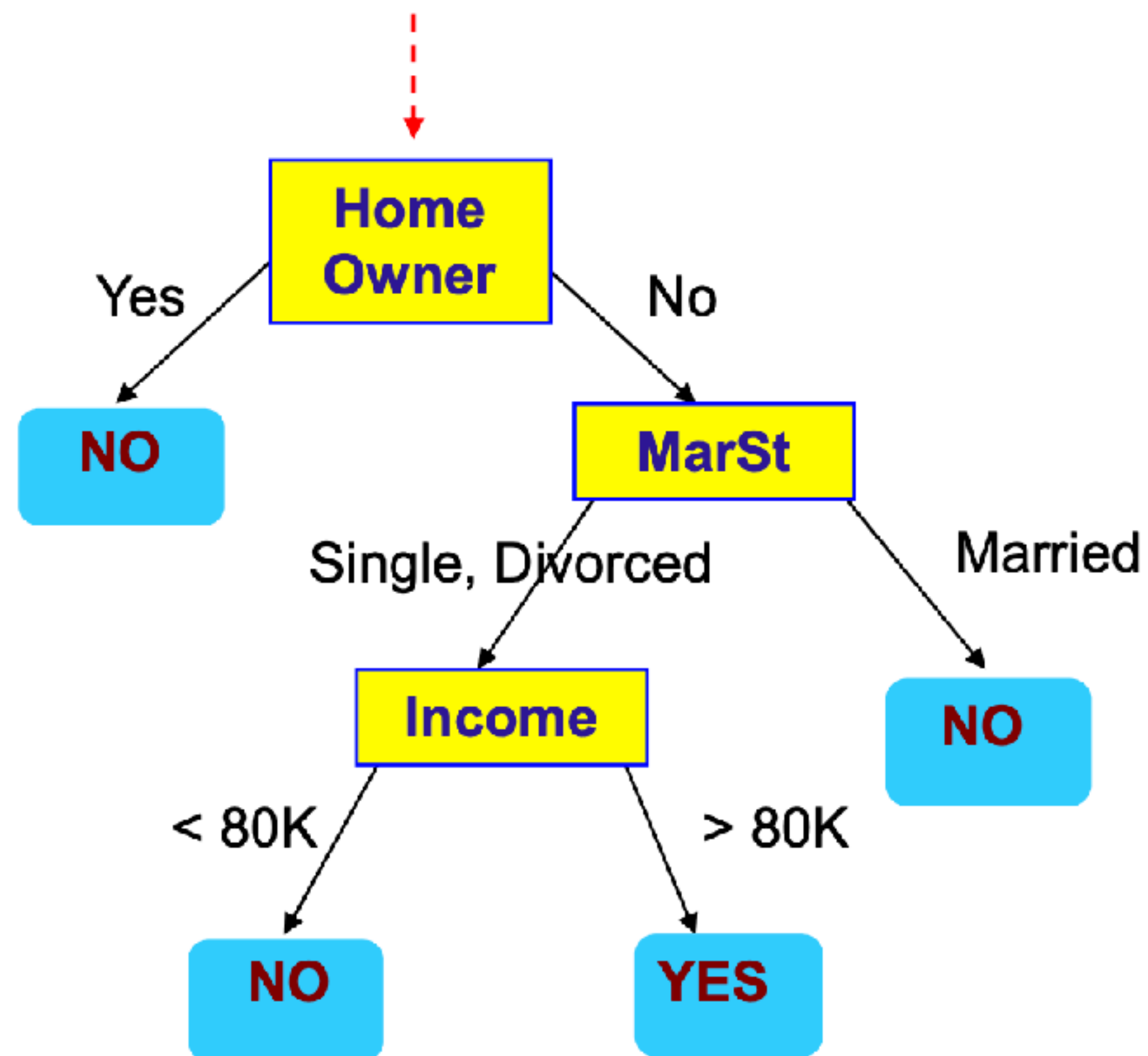


Model: Decision Tree

Decision Tree

Example 3: Testing

Start from the root of tree.



Test Data

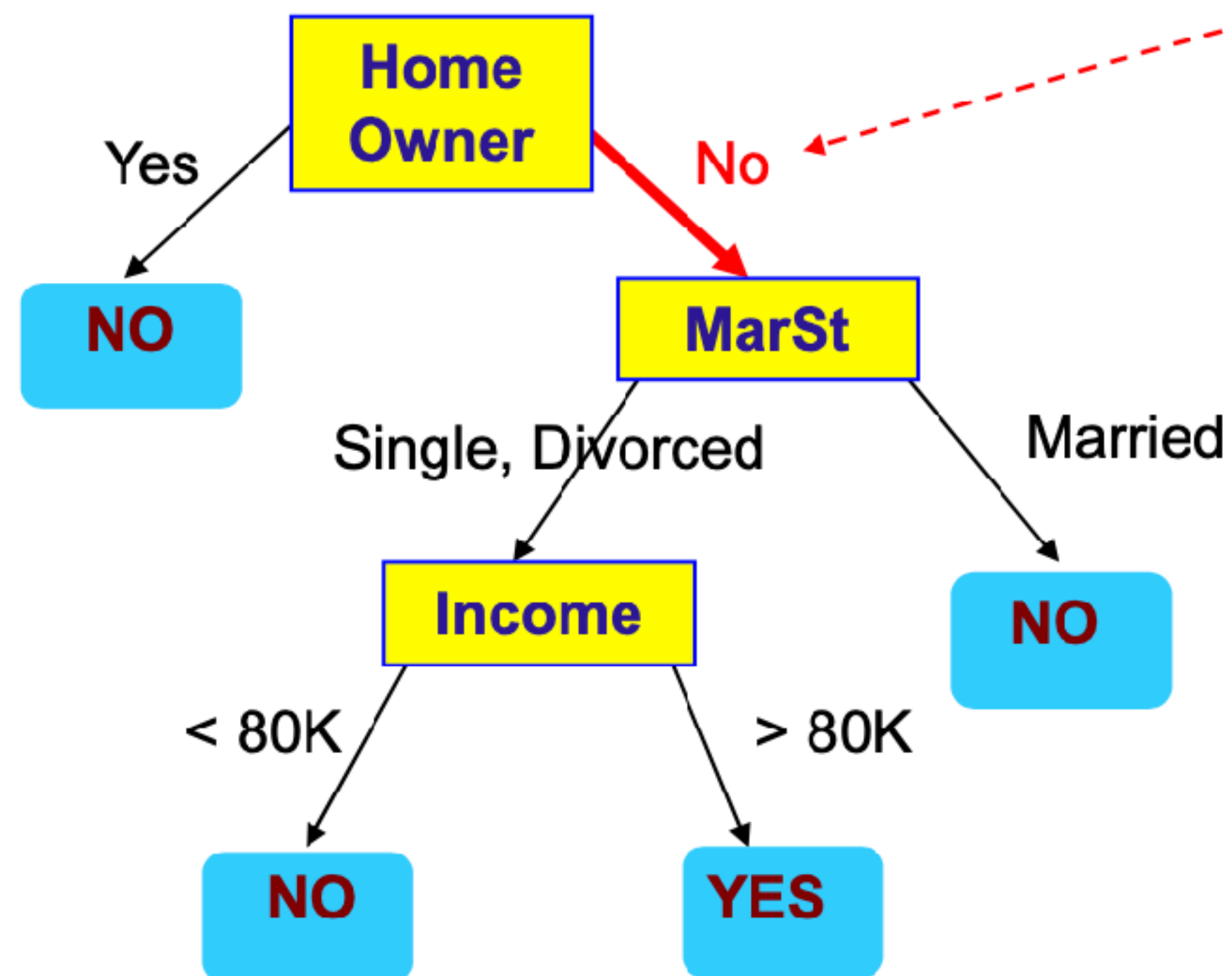
Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?

Decision Tree

Example 3: Testing

Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?

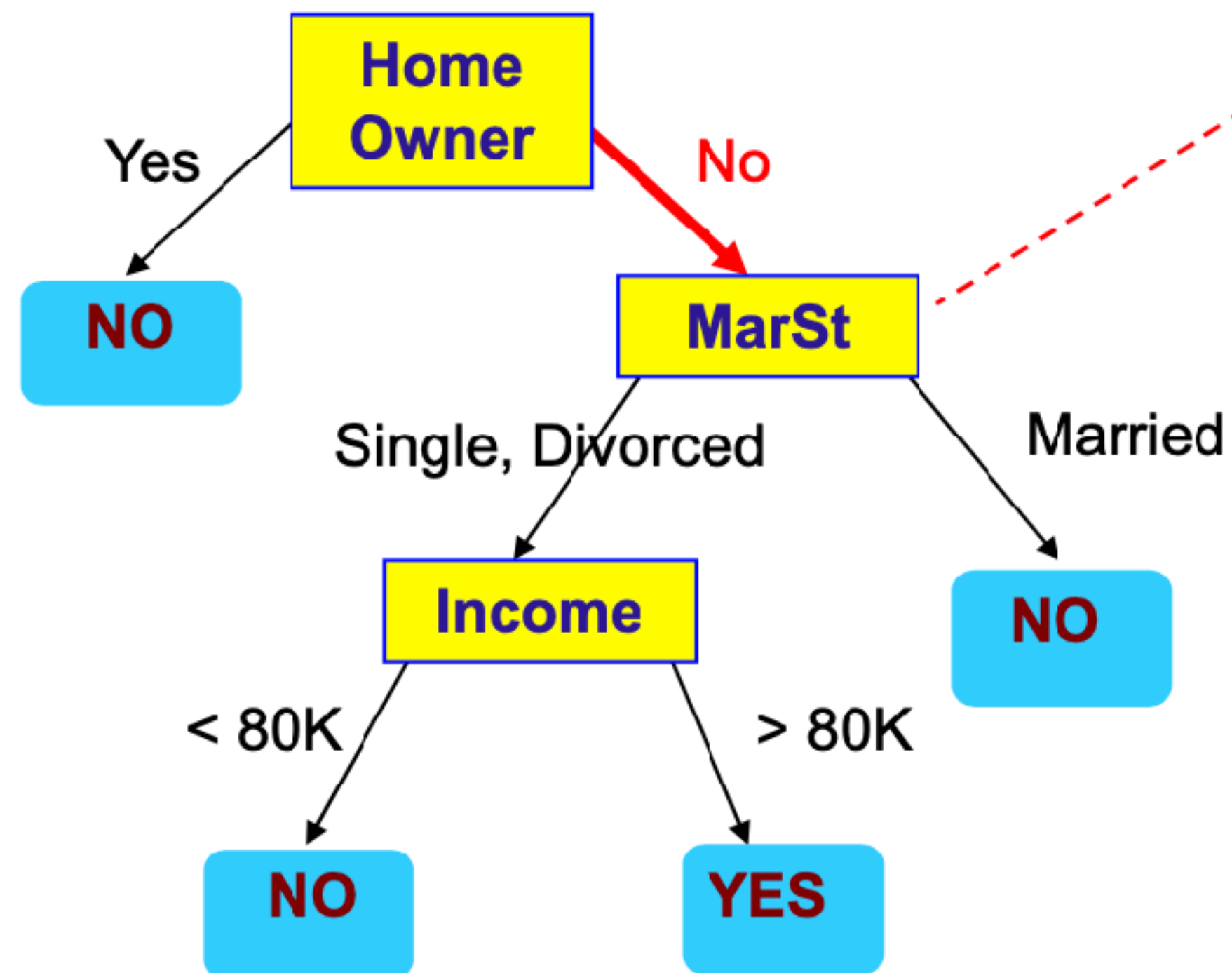


Decision Tree

Example 3: Testing

Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?

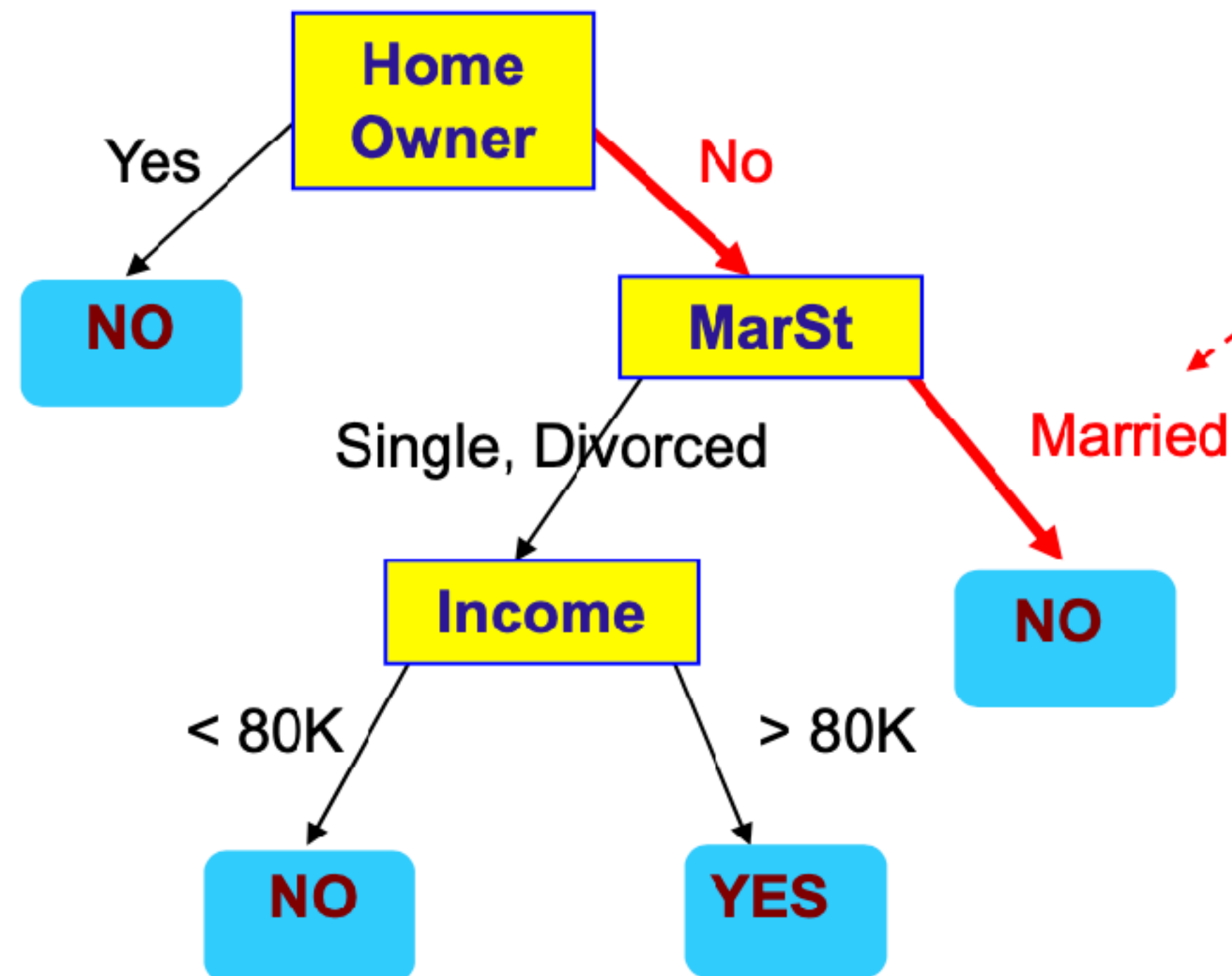


Decision Tree

Example 3: Testing

Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?

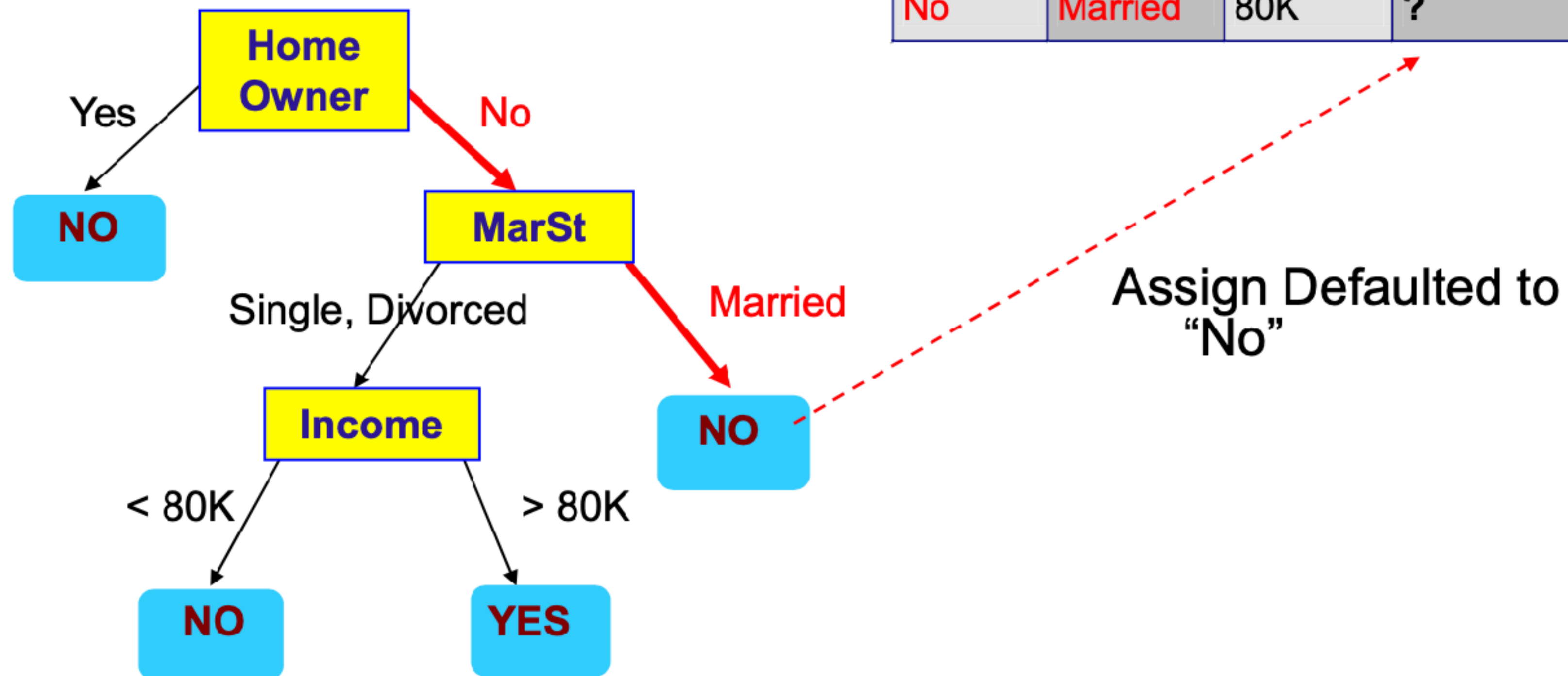


Decision Tree

Example 3: Testing

Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?

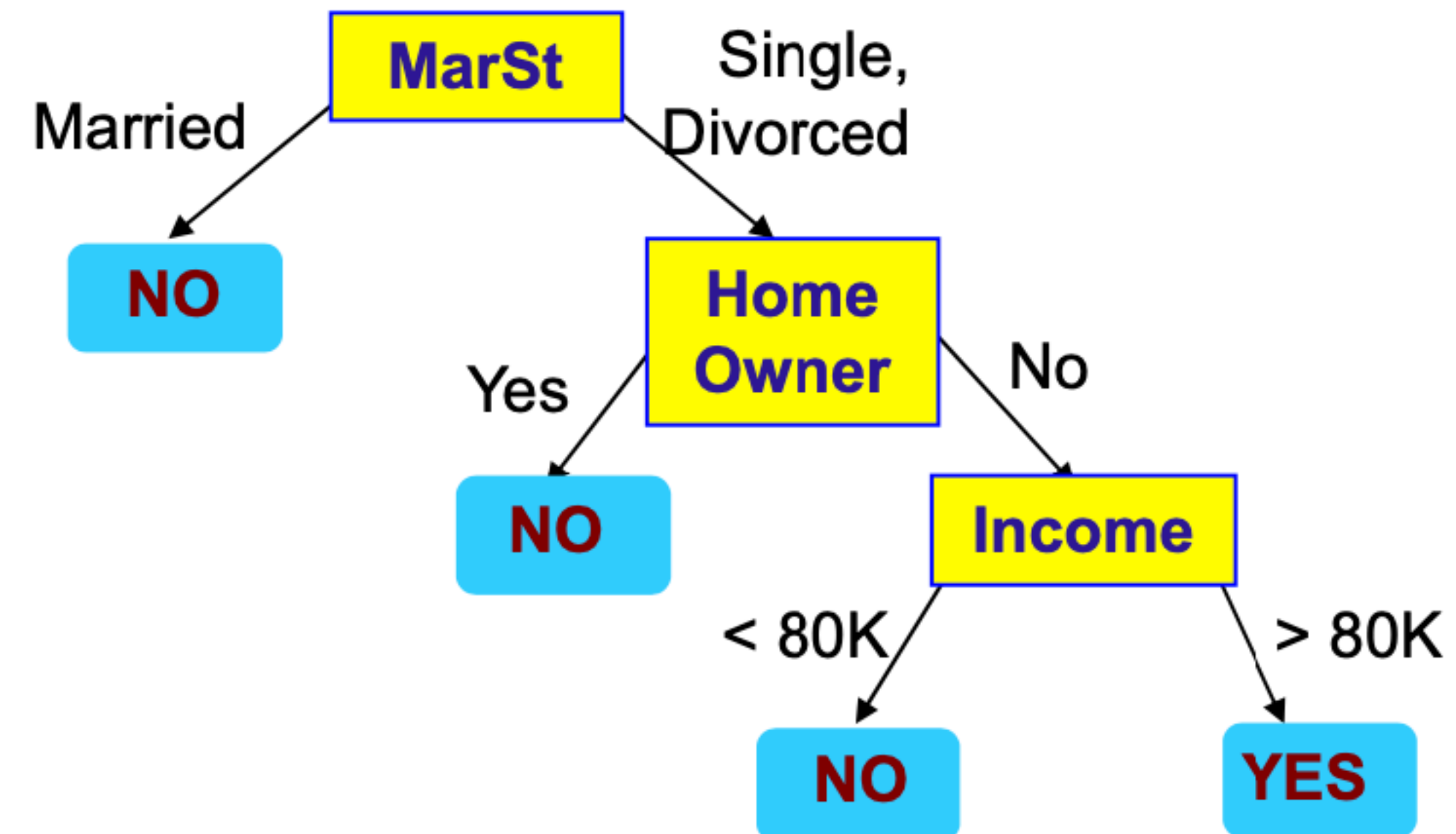


Decision Tree

Example 4

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

categorical
categorical
continuous
class



There could be more than one tree that fits the same data!

Design Issues of Decision Tree

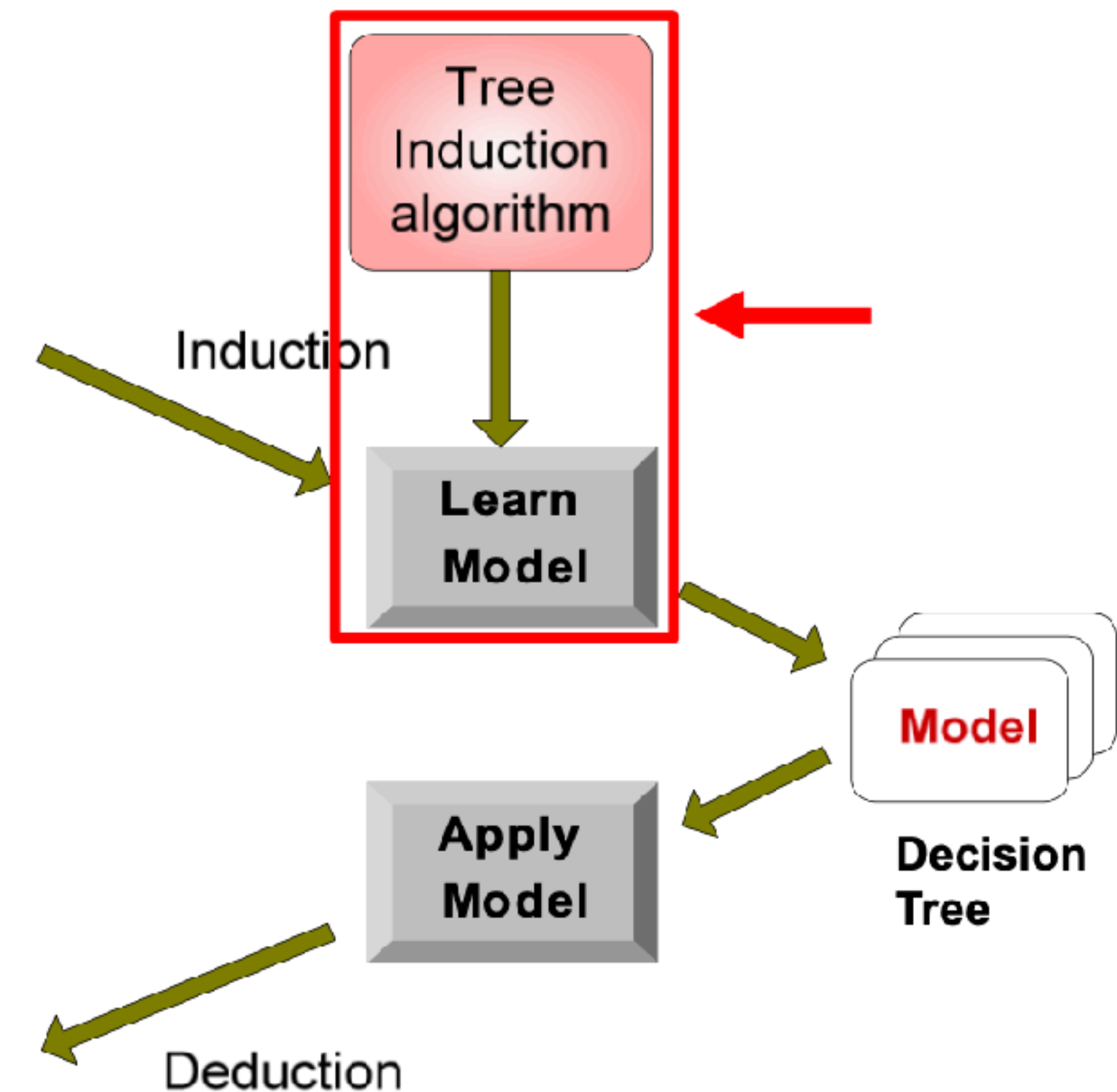
- How should training records be **split**? Which feature to **select**?
- How should the splitting procedure **stop**?

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Training Set

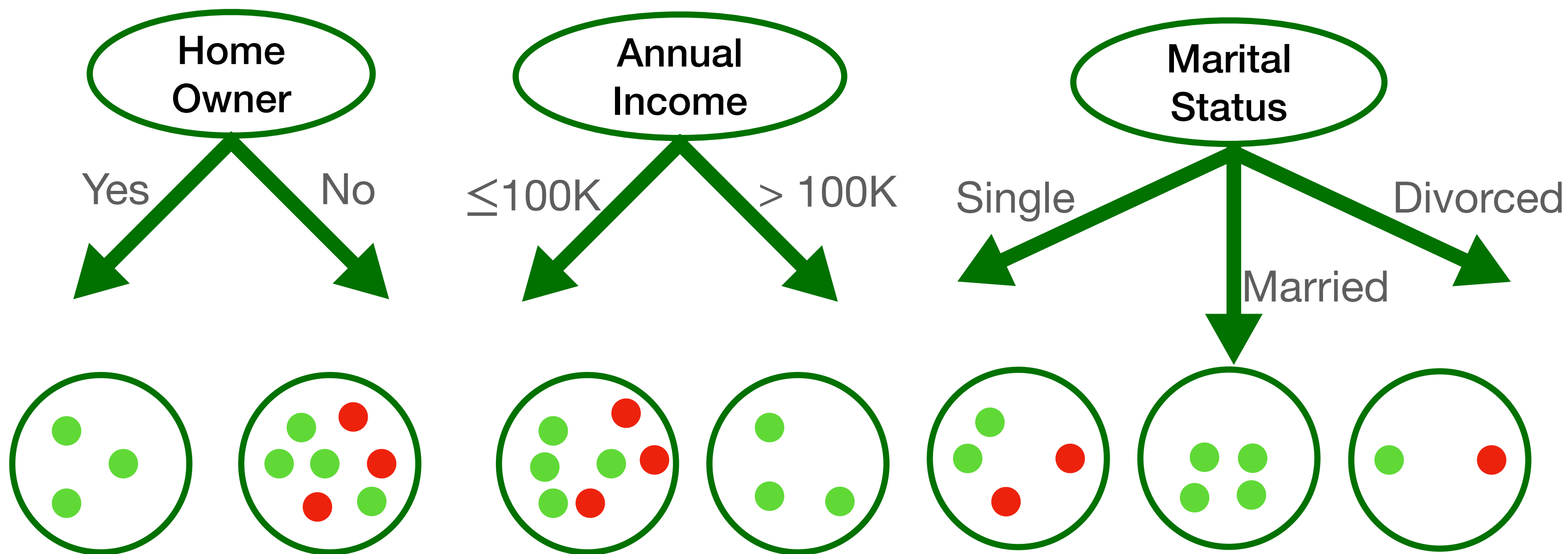
Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

Test Set



Design Issues of Decision Tree

How to determine the best split?



Feature is selected based on **purity**

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower	
1	Yes	Single	125K	No	●
2	No	Married	100K	No	●
3	No	Single	70K	No	●
4	Yes	Married	120K	No	●
5	No	Divorced	95K	Yes	●
6	No	Married	60K	No	●
7	Yes	Divorced	220K	No	●
8	No	Single	85K	Yes	●
9	No	Married	75K	No	●
10	No	Single	90K	Yes	●

Design Issues of Decision Tree

Metrics for attribute selection

- Measures of attribute **purity**

- **Entropy**

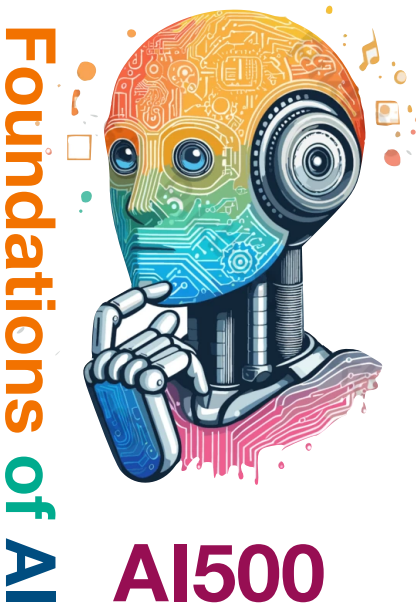
- **Information Gain**

- Gini Index

$$G = 1 - \sum_{i=1}^c p(x)^2$$

Decision Tree

Induction (Sec 3.3 of Introduction to Data Mining (2nd Ed.)

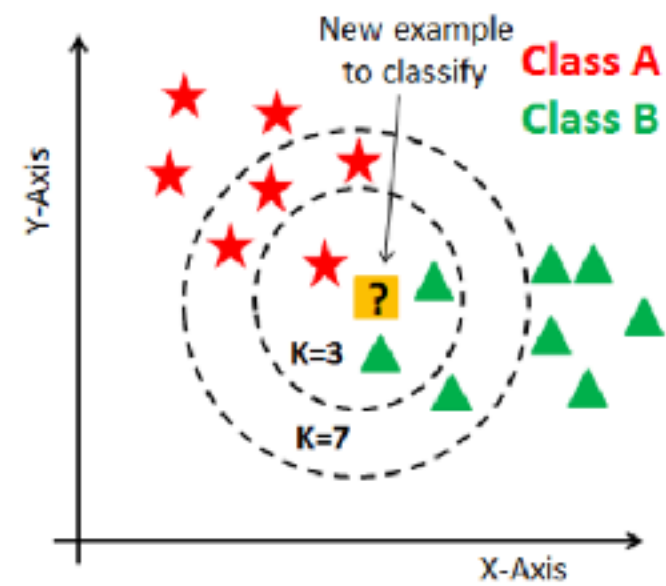


- Many Algorithms:
 - Hunt's Algorithm
 - CART
 - **ID3**
 - C4.5
 - SLIQ
 - SPRINT

Recap: Supervised Learning

k-Nearest Neighbor (kNN) Classifier

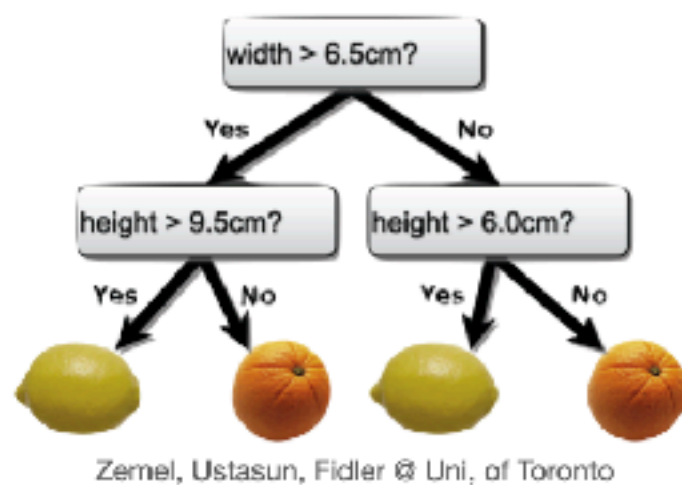
- The class label of a test instance x is predicted to be the most common class among the k nearest neighbors of x in the train set



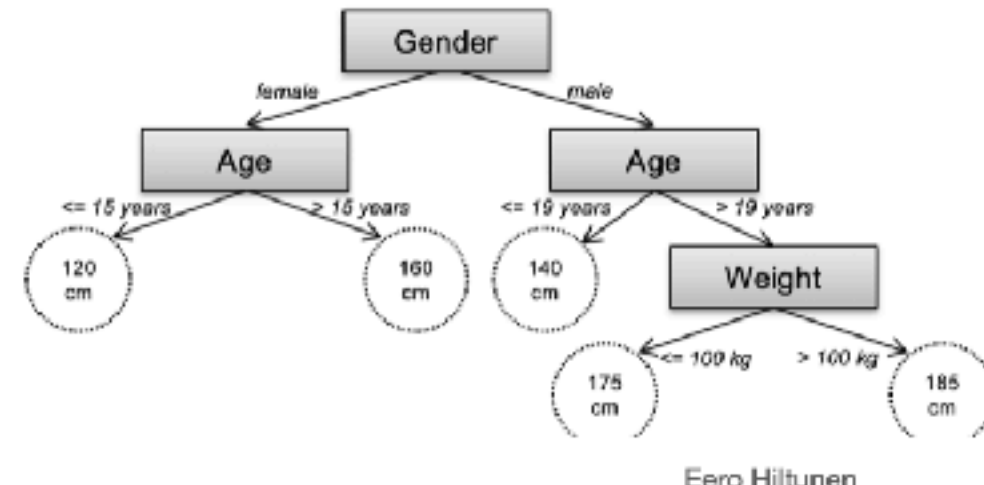
- Assign the test instance (?) class A (★) or class B (▲)
- $k = 3$ nearest neighbors (l_2 distance) 1★ and 2▲ $\Rightarrow$ assigned label= ▲
- $k = 7$ nearest neighbors (l_2 distance) 4★ and 3▲ $\Rightarrow$ assigned label= ★

Decision Tree

Classification vs. Regression



Classification Tree



Regression Tree

Information Theory

$$I = -\log p(x)$$

$$\log(1) = 0$$

100% likely event

$$\log(0.5) = -0.6931$$

50% likely event

$$\log(0) = \text{Inf}$$

unlikely event

Example 1:

- Tomorrow either it will rain or not
- Probability: 1 or 100% $\log(1) = 0$
- Information: ZERO

Example 3:

- In July it will snow in Lahore
- Probability: ~ZERO or 0% (unlikely)
- Information: Very high or 100%

$$\log(0) = \text{Inf}$$

"surprise", or "uncertainty"

Information Theory

Entropy

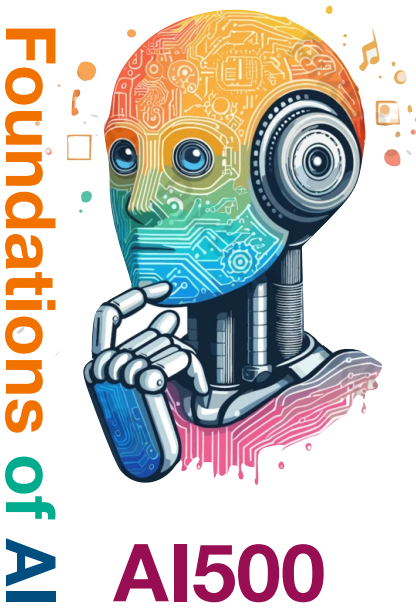
- Average of information

$$H = -\sum_{i=1}^c p(x) \log p(x)$$

Information Gain

$$E(\text{parent}) - \text{weighted average} * E(\text{children})$$

Information Theory



$$I = -\log p(x)$$

$$\log(1) = 0$$

100% likely event

$$\log(0.5) = -0.6931$$

50% likely event

$$\log(0) = \textit{Inf}$$

unlikely event

Information Theory

$$I = -\log p(x)$$

- ▶ Example 1:
 - ▶ Tomorrow either it will rain or not
 - ▶ Probability: 1 or 100% $\log(1) = 0$
 - ▶ Information: ZERO

Information Theory

$$I = -\log p(x)$$

- ▶ Example 1:
 - ▶ Tomorrow either it will rain or not
 - ▶ Probability: 1 or 100% $\log(1) = 0$
 - ▶ Information: ZERO

- ▶ Example 2:
 - ▶ In July it will be very hot in Lahore
 - ▶ Probability: ~ 1 or 100% $\log(1) = 0$
 - ▶ Information: ZERO

Information Theory

$$I = -\log p(x)$$

- ▶ Example 3:
 - ▶ In July it will snow in Lahore
 - ▶ Probability: ~ZERO or 0% (unlikely)
 - ▶ Information: Very high or 100%

$$\log(0) = \textit{Inf}$$

"surprise", or "uncertainty"

Information Theory

- Entropy
 - Average of information

$$H = - \sum_{i=1}^c p(x) \log p(x)$$

- Information Gain

$$E(\text{parent}) - \text{weighted average} * E(\text{children})$$

Entropy

Example

$$E = -p(\text{yes})\log_2(p(\text{yes})) - p(\text{no})\log_2(p(\text{no}))$$

Salary	Age	Purchase
200,000	23	Yes
100,000	47	No
600,000	26	Yes
175,000	32	No
350,000	19	Yes

$$E = -\frac{3}{5}\log_2\left(\frac{3}{5}\right) - \frac{2}{5}\log_2\left(\frac{2}{5}\right)$$

$$E = 0.97$$

Salary	Age	Purchase
75,000	23	No
500,000	47	No
150,000	26	Yes
175,000	32	No
425,000	19	No

$$E = -\frac{1}{5}\log_2\left(\frac{1}{5}\right) - \frac{4}{5}\log_2\left(\frac{4}{5}\right)$$

$$E = 0.72$$

Salary	Age	Purchase
100,000	35	Yes
600,000	21	Yes
400,000	26	Yes
175,000	18	Yes
250,000	37	Yes

$$E = -\frac{5}{5}\log_2\left(\frac{5}{5}\right) - \frac{0}{5}\log_2\left(\frac{0}{5}\right)$$

$$E = 0.0$$

Entropy

Example

$$E = -p(\text{yes})\log_2(p(\text{yes})) - p(\text{no})\log_2(p(\text{no})) - p(\text{may be})\log_2(p(\text{may be}))$$

Salary	Age	Purchase
200,000	23	Yes
100,000	47	No
600,000	67	May be
175,000	32	No
350,000	19	Yes
600,000	45	May be
275,000	32	No
300,000	32	May be
150,000	28	No

$$E = -\frac{2}{9}\log_2\left(\frac{2}{9}\right) - \frac{4}{9}\log_2\left(\frac{4}{9}\right) - \frac{3}{9}\log_2\left(\frac{3}{9}\right)$$

$$E = 1.53$$

Find the best feature

Decision Tree

Example 2

Given weather condition
 $\mathbf{x} = (\text{sunny, cool, high, true})$
will game be played?

- To classify a test instance $\mathbf{x}$ traverse the tree from root to leaf
- Take branches at internal nodes according to results of their tests
- Predict the class label at the leaf node reached

X				y
outlook	temp	humidity	wind	play
sunny	hot	high	weak	no
sunny	hot	high	strong	no
overcast	hot	high	weak	yes
rainy	mild	high	weak	yes
rainy	cool	normal	weak	yes
rainy	cool	normal	strong	no
overcast	cool	normal	strong	yes
sunny	mild	high	weak	no
sunny	cool	normal	weak	yes
rainy	mild	normal	weak	yes
sunny	mild	normal	strong	yes
overcast	mild	high	strong	yes
overcast	hot	normal	weak	yes
rainy	mild	high	strong	no



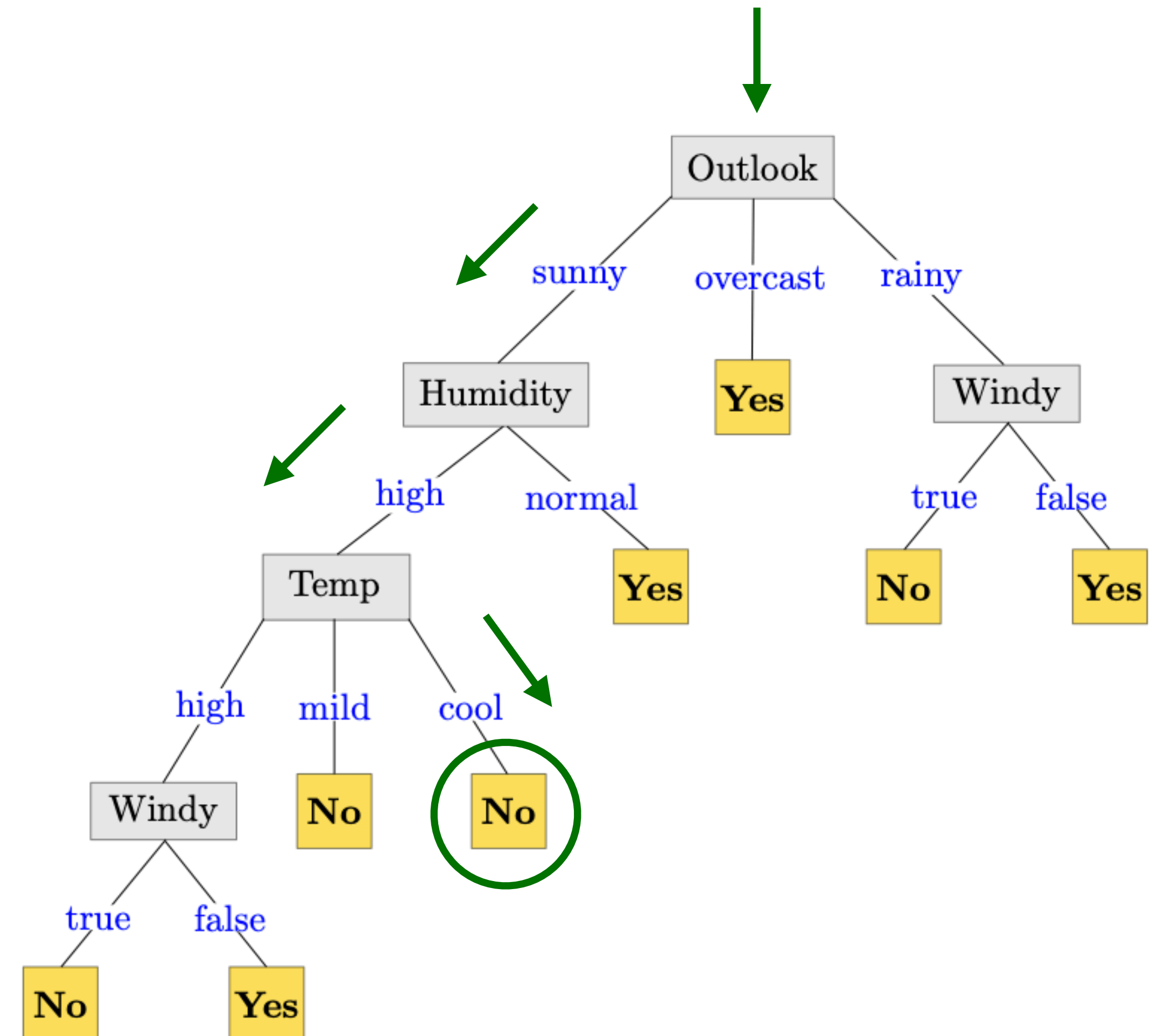
Decision Tree

Example 2

Given weather condition

$X = (\text{sunny, cool, high, true})$

will game be played?



X				y
outlook	temp	humidity	wind	play
sunny	hot	high	weak	no
sunny	hot	high	strong	no
overcast	hot	high	weak	yes
rainy	mild	high	weak	yes
rainy	cool	normal	weak	yes
rainy	cool	normal	strong	no
overcast	cool	normal	strong	yes
sunny	mild	high	weak	no
sunny	cool	normal	weak	yes
rainy	mild	normal	weak	yes
sunny	mild	normal	strong	yes
overcast	mild	high	strong	yes
overcast	hot	normal	weak	yes
rainy	mild	high	strong	no

Feature: outlook
Unique values: sunny, overcast, rainy

play : [9 + ,5−]

$$E(\text{play}) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$$

O_{sunny} : [2 + ,3−]

$$E(O_{\text{sunny}}) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.971$$

O_{overcast} : [4 + ,0−]

$$E(O_{\text{overcast}}) = -\frac{4}{4} \log_2 \frac{4}{4} - \frac{0}{4} \log_2 \frac{0}{4} = 0.0$$

O_{rainy} : [3 + ,2−]

$$E(O_{\text{rainy}}) = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} = 0.971$$

Gain(play, outlook) = $E(\text{play}) - \frac{5}{14}E(O_{\text{sunny}}) - \frac{4}{14}E(O_{\text{overcast}}) - \frac{5}{14}E(O_{\text{rainy}})$

Gain(play, outlook) = $0.94 - \frac{5}{14}0.971 - \frac{4}{14}0.0 - \frac{5}{14}0.971 = 0.2464$

X				y
outlook	temp	humidity	wind	play
sunny	hot	high	weak	no
sunny	hot	high	strong	no
overcast	hot	high	weak	yes
rainy	mild	high	weak	yes
rainy	cold	normal	weak	yes
rainy	cold	normal	strong	no
overcast	cold	normal	strong	yes
sunny	mild	high	weak	no
sunny	cold	normal	weak	yes
rainy	mild	normal	weak	yes
sunny	mild	normal	strong	yes
overcast	mild	high	strong	yes
overcast	hot	normal	weak	yes
rainy	mild	high	strong	no

Feature: temperature
Unique values: hot, mild, cold

play : [9 + ,5−]

$$E(\text{play}) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$$

$T_{hot} : [2 + ,2−]$

$$E(T_{hot}) = -\frac{2}{4} \log_2 \frac{2}{4} - \frac{2}{4} \log_2 \frac{2}{4} = 1.0$$

$T_{mild} : [4 + ,2−]$

$$E(T_{mild}) = -\frac{4}{6} \log_2 \frac{4}{6} - \frac{2}{6} \log_2 \frac{2}{6} = 0.9183$$

$T_{cold} : [3 + ,1−]$

$$E(T_{cold}) = -\frac{3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4} = 0.8113$$

$$\text{Gain}(\text{play}, \text{temp}) = E(\text{play}) - \frac{4}{14} E(T_{hot}) - \frac{6}{14} E(T_{mild}) - \frac{4}{14} E(T_{cold})$$

$$\text{Gain}(\text{play}, \text{temp}) = 0.94 - \frac{4}{14} 1.0 - \frac{6}{14} 0.9183 - \frac{4}{14} 0.8113 = 0.0289$$

X				y
outlook	temp	humidity	wind	play
sunny	hot	high	weak	no
sunny	hot	high	strong	no
overcast	hot	high	weak	yes
rainy	mild	high	weak	yes
rainy	cool	normal	weak	yes
rainy	cool	normal	strong	no
overcast	cool	normal	strong	yes
sunny	mild	high	weak	no
sunny	cool	normal	weak	yes
rainy	mild	normal	weak	yes
sunny	mild	normal	strong	yes
overcast	mild	high	strong	yes
overcast	hot	normal	weak	yes
rainy	mild	high	strong	no

Feature: humidity
Unique values: high, normal

play : [9 + ,5−] $E(\text{play}) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$

H_{high} : [3 + ,4−] $E(H_{high}) = -\frac{3}{7} \log_2 \frac{3}{7} - \frac{4}{7} \log_2 \frac{4}{7} = 0.9852$

H_{normal} : [6 + ,1−] $E(H_{normal}) = -\frac{6}{7} \log_2 \frac{6}{7} - \frac{1}{7} \log_2 \frac{1}{7} = 0.5916$

$\text{Gain}(\text{play}, \text{humidty}) = E(\text{play}) - \frac{7}{14} E(H_{high}) - \frac{7}{14} E(H_{normal})$

$\text{Gain}(\text{play}, \text{humidty}) = 0.94 - \frac{7}{14} 0.9852 - \frac{7}{14} 0.5916 = 0.1516$

X				y
outlook	temp	humidity	wind	play
sunny	hot	high	weak	no
sunny	hot	high	strong	no
overcast	hot	high	weak	yes
rainy	mild	high	weak	yes
rainy	cool	normal	weak	yes
rainy	cool	normal	strong	no
overcast	cool	normal	strong	yes
sunny	mild	high	weak	no
sunny	cool	normal	weak	yes
rainy	mild	normal	weak	yes
sunny	mild	normal	strong	yes
overcast	mild	high	strong	yes
overcast	hot	normal	weak	yes
rainy	mild	high	strong	no

Feature: wind
Unique values: weak, strong

play : [9 + ,5−]

$E(\text{play}) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$

$W_{weak} : [3 + ,3−]$

$E(W_{weak}) = -\frac{3}{6} \log_2 \frac{3}{6} - \frac{3}{6} \log_2 \frac{3}{6} = 1.0$

$W_{strong} : [6 + ,2−]$

$E(W_{strong}) = -\frac{6}{8} \log_2 \frac{6}{8} - \frac{2}{8} \log_2 \frac{2}{8} = 0.8113$

Gain(play, wind) = $E(\text{play}) - \frac{6}{14} E(W_{weak}) - \frac{8}{14} E(W_{strong})$

Gain(play, wind) = $0.94 - \frac{6}{14} 1.0 - \frac{8}{14} 0.8113 = 0.0478$

X				y
outlook	temp	humidity	wind	play
sunny	hot	high	weak	no
sunny	hot	high	strong	no
overcast	hot	high	weak	yes
rainy	mild	high	weak	yes
rainy	cool	normal	weak	yes
rainy	cool	normal	strong	no
overcast	cool	normal	strong	yes
sunny	mild	high	weak	no
sunny	cool	normal	weak	yes
rainy	mild	normal	weak	yes
sunny	mild	normal	strong	yes
overcast	mild	high	strong	yes
overcast	hot	normal	weak	yes
rainy	mild	high	strong	no

Gain(play, outlook) = 0.2464

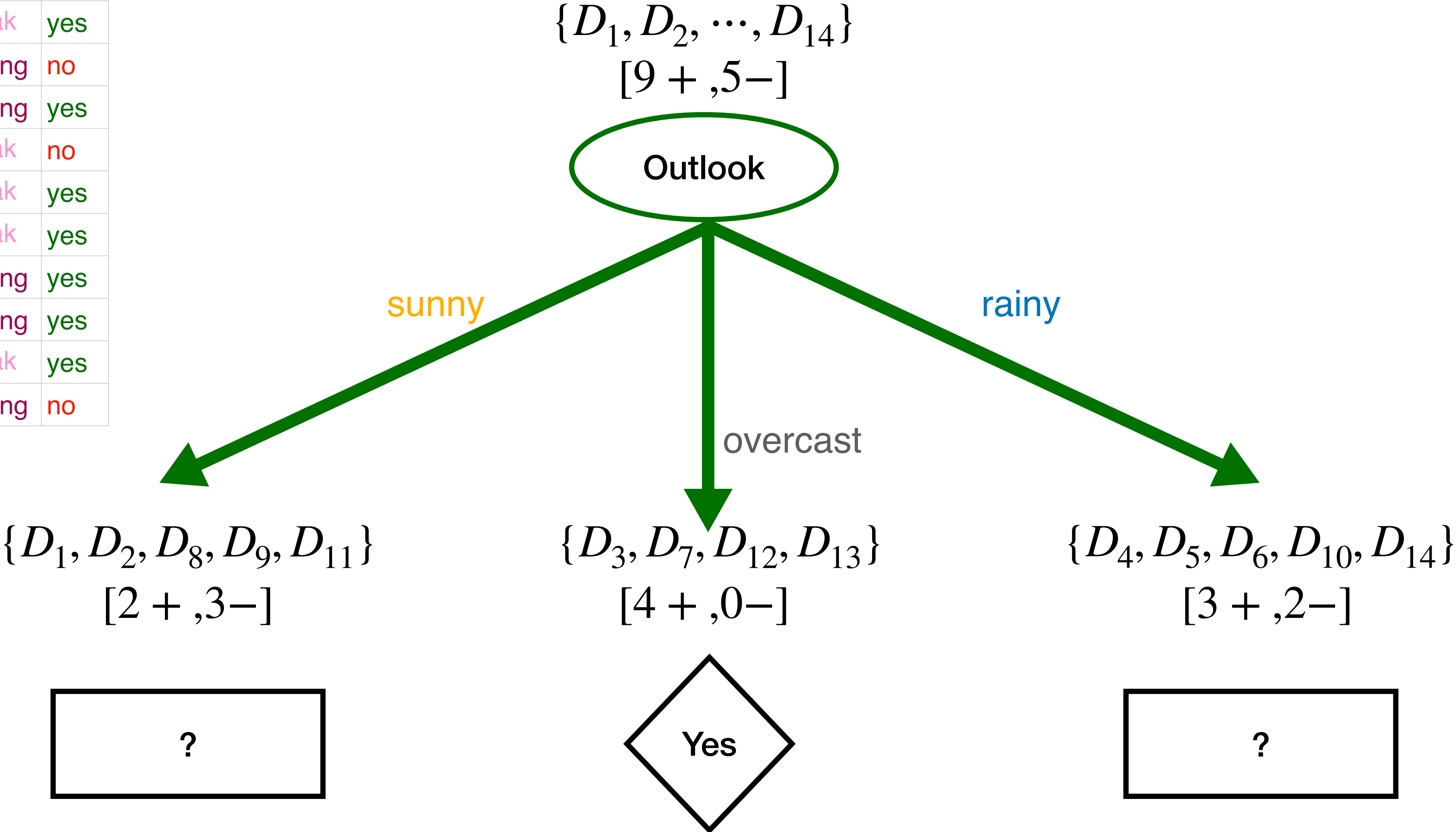
Gain(play, temp) = 0.0289

Gain(play, humidty) = 0.1516

Gain(play, wind) = 0.0478

Split the tree

	X				y
Day	outlook	temp	humidity	wind	play
1	sunny	hot	high	weak	no
2	sunny	hot	high	strong	no
3	overcast	hot	high	weak	yes
4	rainy	mild	high	weak	yes
5	rainy	cool	normal	weak	yes
6	rainy	cool	normal	strong	no
7	overcast	cool	normal	strong	yes
8	sunny	mild	high	weak	no
9	sunny	cool	normal	weak	yes
10	rainy	mild	normal	weak	yes
11	sunny	mild	normal	strong	yes
12	overcast	mild	high	strong	yes
13	overcast	hot	normal	weak	yes
14	rainy	mild	high	strong	no



For each child node
If child node is not a leaf node
Find the next best attribute

	X			y
Day	temp	humidity	wind	play
1	hot	high	weak	no
2	hot	high	strong	no
8	mild	high	weak	no
9	cool	normal	weak	yes
11	mild	normal	strong	yes

Feature: temperature
Unique values: hot, mild, cold

$O_{\text{sunny}} : [2 + , 3 -]$

$E(O_{\text{sunny}}) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.971$

$T_{\text{hot}} : [0 + , 2 -]$

$E(T_{\text{hot}}) = 0.0$

$T_{\text{mild}} : [1 + , 1 -]$

$E(T_{\text{mild}}) = 1.0$

$T_{\text{cold}} : [1 + , 0 -]$

$E(T_{\text{cold}}) = 0.0$

$$\text{Gain}(O_{\text{sunny}}, \text{temp}) = E(O_{\text{sunny}}) - \frac{2}{5}E(T_{\text{hot}}) - \frac{2}{5}E(T_{\text{mild}}) - \frac{1}{5}E(T_{\text{cold}})$$

$$\text{Gain}(O_{\text{sunny}}, \text{temp}) = 0.971 - \frac{2}{5}0.0 - \frac{2}{5}1.0 - \frac{1}{5}0.0 = 0.570$$

	X			y
Day	temp	humidity	wind	play
1	hot	high	weak	no
2	hot	high	strong	no
8	mild	high	weak	no
9	cool	normal	weak	yes
11	mild	normal	strong	yes

Feature: humidity
Unique values: high, normal

$$O_{sunny} : [2 + ,3-] \qquad E(O_{sunny}) = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} = 0.971$$

$$H_{high} : [0 + ,3-] \qquad E(H_{high}) = 0.0$$

$$T_{normal} : [2 + ,0-] \qquad E(H_{normal}) = 0.0$$

$$\text{Gain}(O_{sunny}, \text{humidity}) = E(O_{sunny}) - \frac{3}{5}E(H_{high}) - \frac{2}{5}E(H_{normal})$$

$$\text{Gain}(O_{sunny}, \text{humidity}) = 0.971 - \frac{3}{5}0.0 - \frac{2}{5}0.0 = 0.971$$

	X			y
Day	temp	humidity	wind	play
1	hot	high	weak	no
2	hot	high	strong	no
8	mild	high	weak	no
9	cool	normal	weak	yes
11	mild	normal	strong	yes

Feature: wind
Unique values: weak, strong

$O_{\text{sunny}} : [2 + , 3 -]$

$E(O_{\text{sunny}}) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.971$

$W_{\text{weak}} : [1 + , 2 -]$

$E(W_{\text{weak}}) = -\frac{1}{3} \log_2 \frac{1}{3} - \frac{2}{3} \log_2 \frac{2}{3} = 0.9183$

$W_{\text{strong}} : [1 + , 1 -]$

$E(W_{\text{strong}}) = 1.0$

$$\text{Gain}(O_{\text{sunny}}, \text{wind}) = E(O_{\text{sunny}}) - \frac{2}{5} E(W_{\text{weak}}) - \frac{3}{5} E(H_{\text{strong}})$$

$$\text{Gain}(O_{\text{sunny}}, \text{wind}) = 0.971 - \frac{2}{5} 1.0 - \frac{3}{5} 0.9183 = 0.0192$$

	X			y
Day	temp	humidity	wind	play
1	hot	high	weak	no
2	hot	high	strong	no
8	mild	high	weak	no
9	cool	normal	weak	yes
11	mild	normal	strong	yes

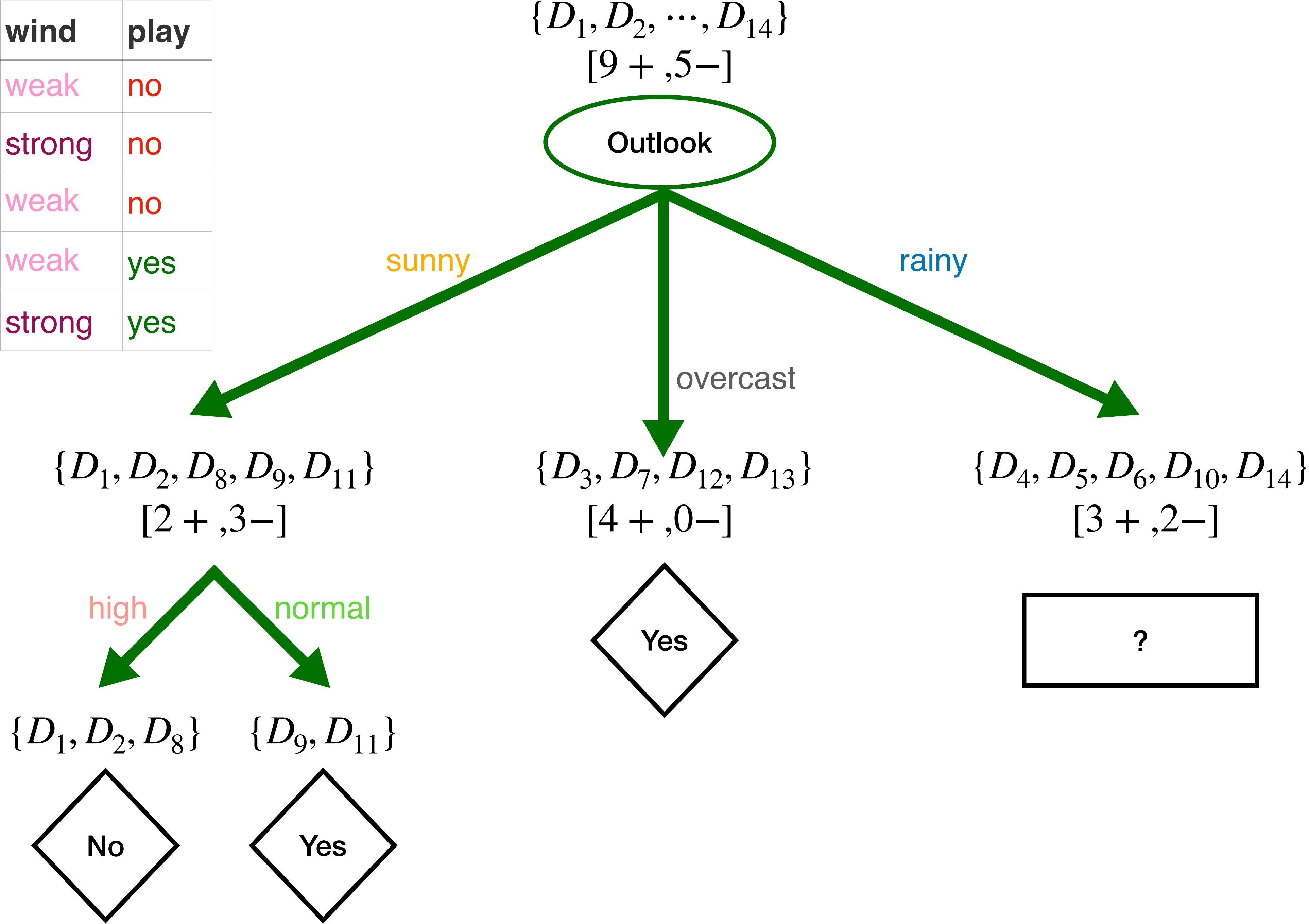
Gain(O_{sunny} , temp) = 0.570

Gain(O_{sunny} , **humidity**) = 0.971

Gain(O_{sunny} , wind) = 0.0192

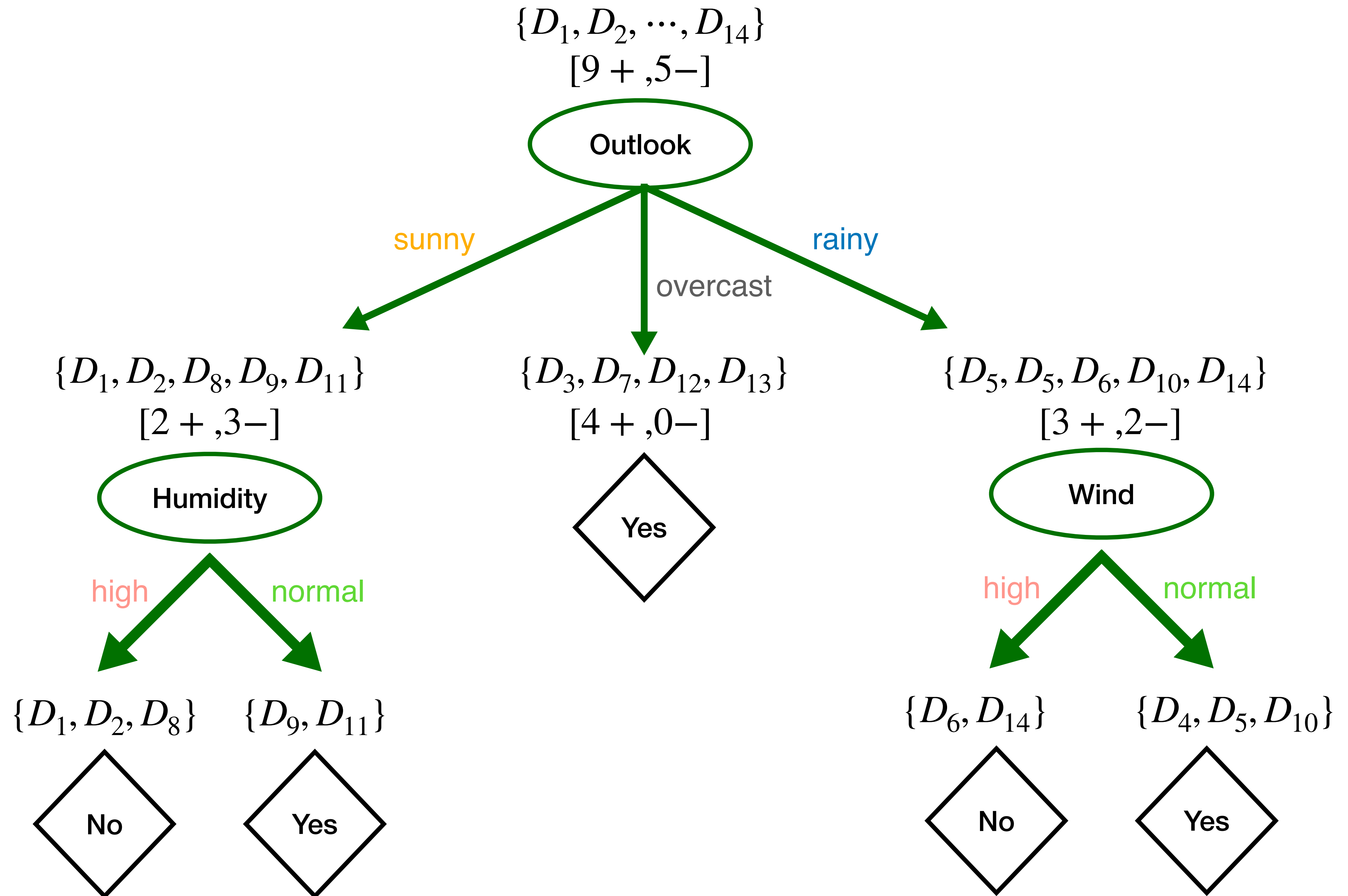
Split the tree

Day	temp	humidity	wind	play
1	hot	high	weak	no
2	hot	high	strong	no
8	mild	high	weak	no
9	cool	normal	weak	yes
11	mild	normal	strong	yes



	<i>X</i>				<i>y</i>
Day	outlook	temp	humidity	wind	play
4	rainy	mild	high	weak	yes
5	rainy	cool	normal	weak	yes
6	rainy	cool	normal	strong	no
10	rainy	mild	normal	weak	yes
14	rainy	mild	high	strong	no

Do it yourself



Decision Tree

Python Implementation



Node

feature

value

left

right

threshold

DecisionTree

n_feature

max_depth

root

min_samples_split

_grow_tree

check the stopping criteria

find the best split

create child nodes

_find_best_split

for each feature

calculate gain

return the feature with highest gain

_split

check the stopping criteria

find the best split

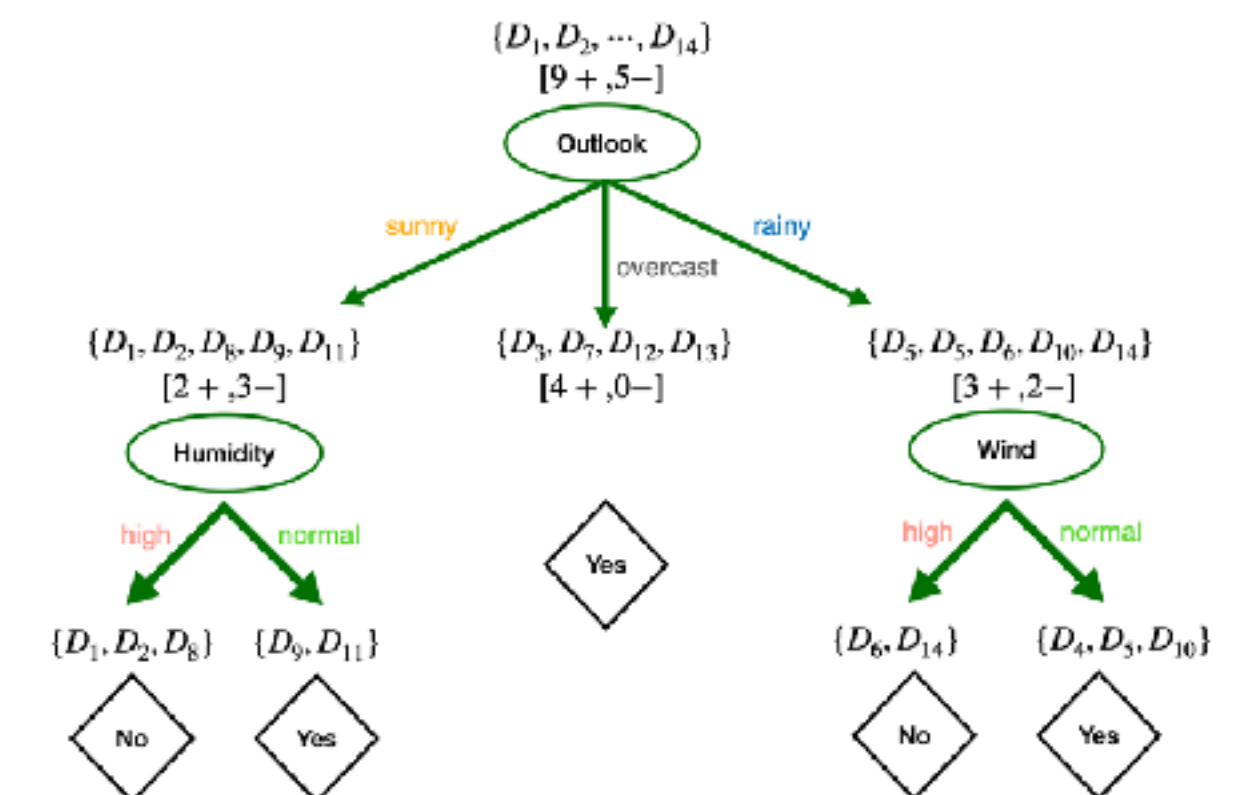
create child nodes

_information_gain

$E(\text{parent}) - \omega * E(\text{children})$

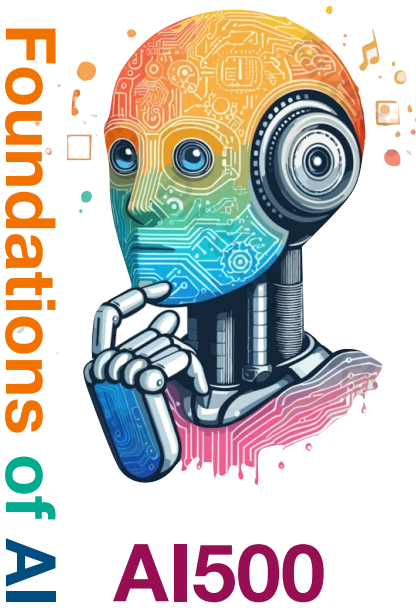
_entropy

$$H = - \sum_{i=1}^c p(x) \log p(x)$$



Decision Tree

Python Implementation



```
def grow_tree (X, y, depth):  
    # check the stopping criteria  
    If depth >= max_depth or  
        n_labels == 1 or  
        n_samples < min_samples_split)  
        return leaf_node_value  
  
    # recursively grow the tree  
    best_feature = find_best_split(X, y, features)  
    left_idx, right_idx = split(X, y, best_feature)  
    left_sub_tree = grow_tree(X(left_idx), y(left_idx))  
    right_sub_tree = grow_tree(X(right_idx), y(right_idx))  
    return Node(best_feature, left_sub_tree, right_sub_tree)
```

```
def find_best_split (X, y, features):  
    best_gain = -1  
  
    for each feature  
        for each unique value of feature  
            gain = info_gain(unique value of feature)  
            if gain > best_gain  
                best_feature = feature  
                best_gain = gain  
  
    return best_feature
```

Class Node
Value
Left
Right
Features

function is_leaf_node()

DecisionTree
Nodes

Function grow_tree
Function Find_best_feature(X,y)
Function split
Function Entropy
Function Info_Gain

Function grow_tree (X,y)
1. [feature_ind] = Find_best_feature
2. Left,right = split
3. is_leaf_node
 1. Return node.value
grow_tree(left_X,left_y)
grow_tree(right_X,right_y)

Function Find_best_feature
best_gain = -1
For each feature
 Gain = info_gain()
 If gain>best_gain
 best_gain = gain

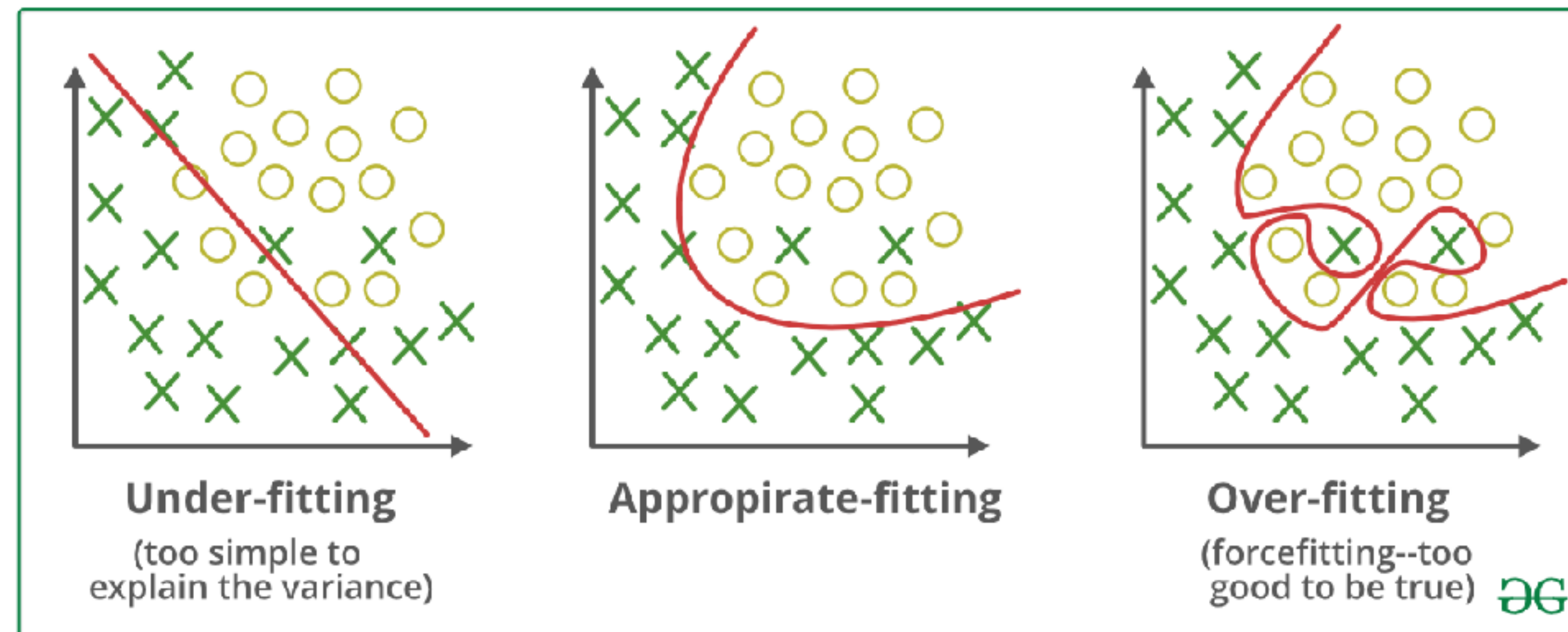
Function info_gain
Entropy of parent
For each feature
 Entropy

$E(\text{parent}) - \text{Entropy of children}$

Decision Trees

Limitations

- In decision tree we **split feature space** at each node, thus large/deep trees are need for complex decision boundary
- Decision trees suffers from **overfitting** as they have **high variance**



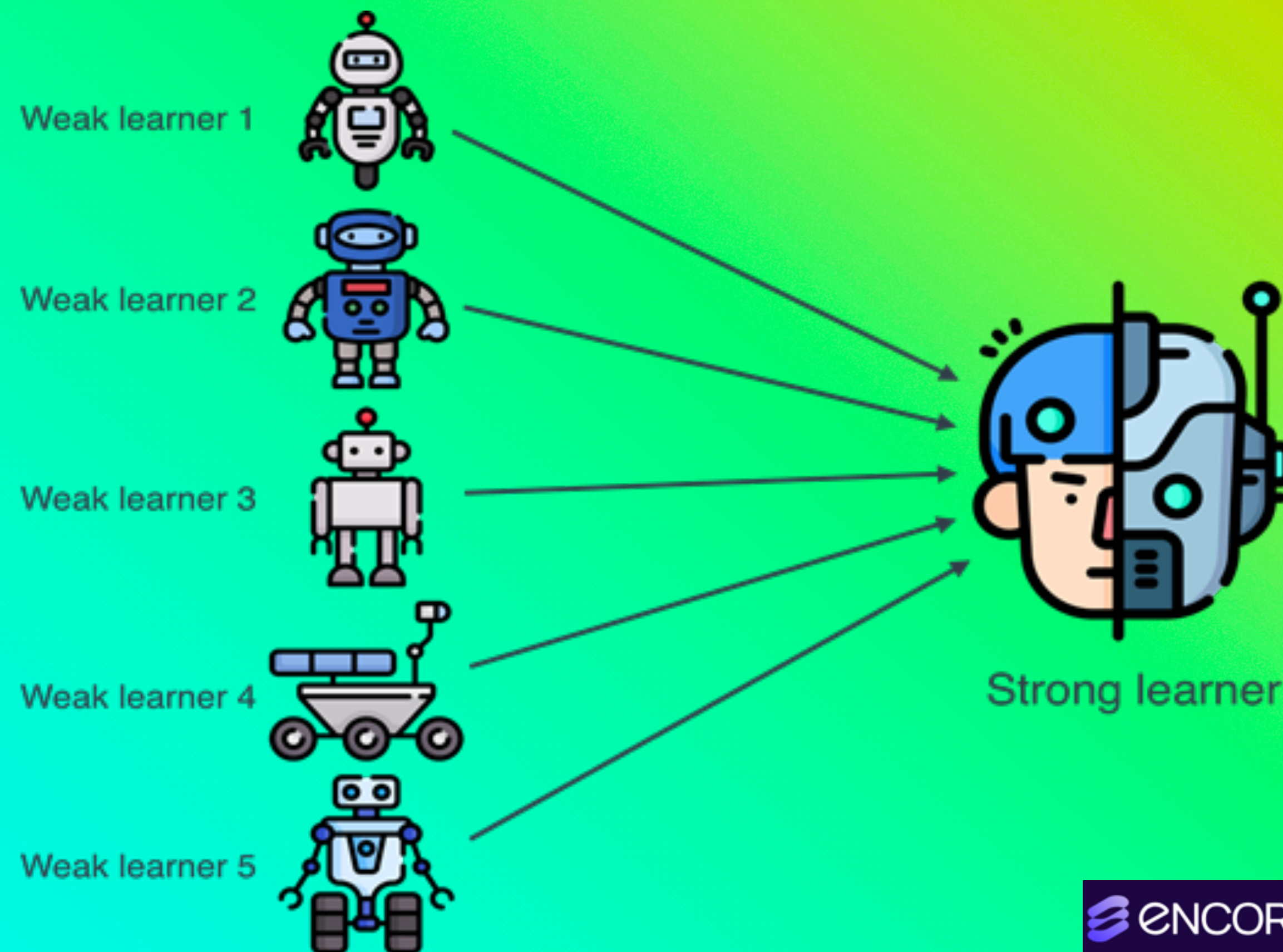
- **Solution:** train multiple models to achieve the same task, combine their output to achieve a single output

Overfitting/Underfitting/Appropriate



- Data split (Training: 60, Validation: 20, Testing: 20)
- Overfitting
 - Training set - 97.6%
 - Validation - unseen data - used for testing during training - 60%
 - Test set - unseen data - 20%
- Appropriate fitting
 - Training set - 91%
 - Validation - unseen data - used for testing during training - 82%
 - Test set - unseen data - 81%
- Under fitting
 - Training set - 71%
 - Validation - unseen data - used for testing during training - 62%
 - Test set - unseen data - 58%

Ensemble learning

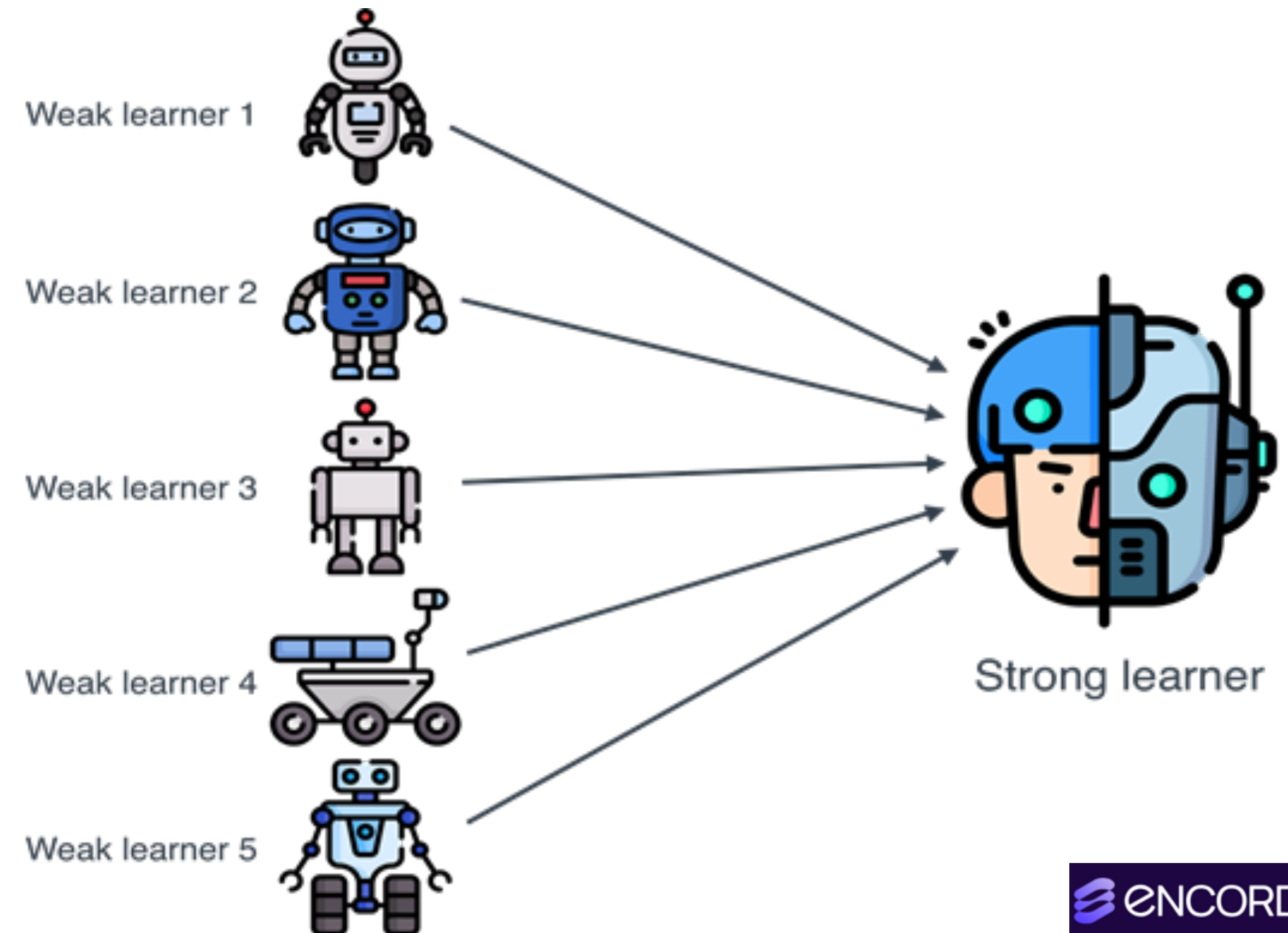


Readings

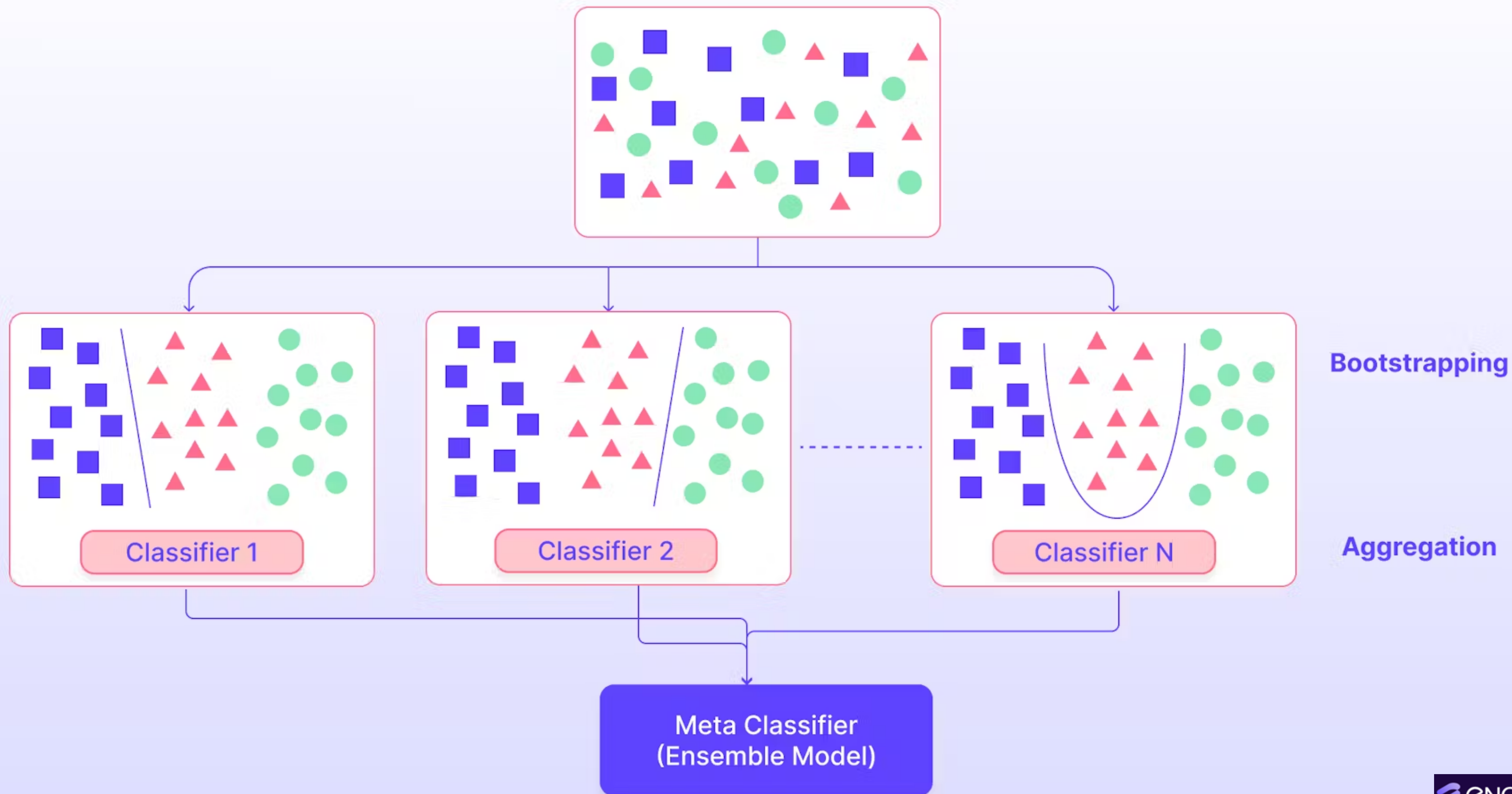
Section 19.8 Russel & Norvig 2nd Ed.

Ensemble learning

- A supervised machine-learning technique that **combines multiple models** to build a more powerful and robust model
- Less likely to **overfit** the data
- Can be used for both **classification** and **regression** tasks

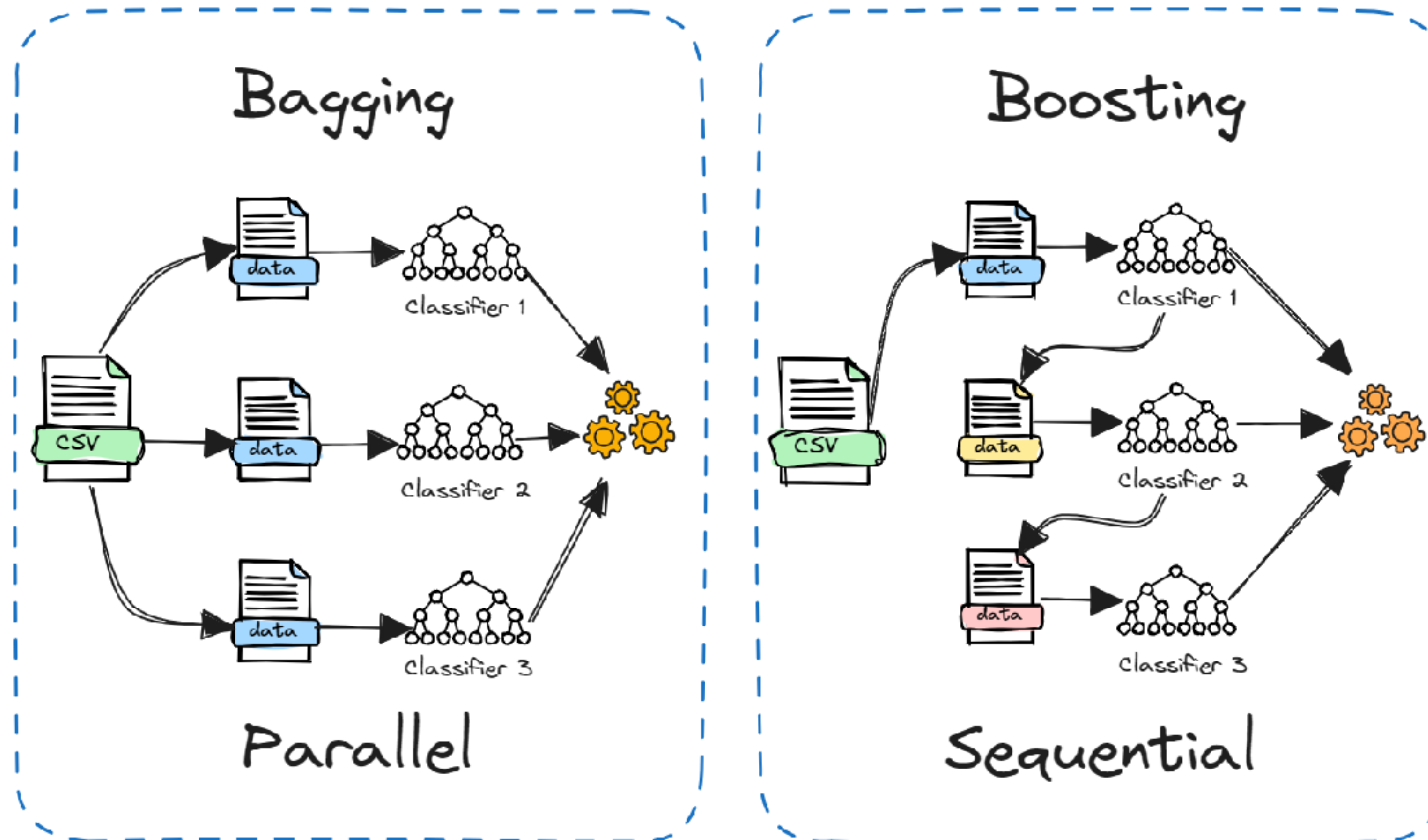


Training Dataset



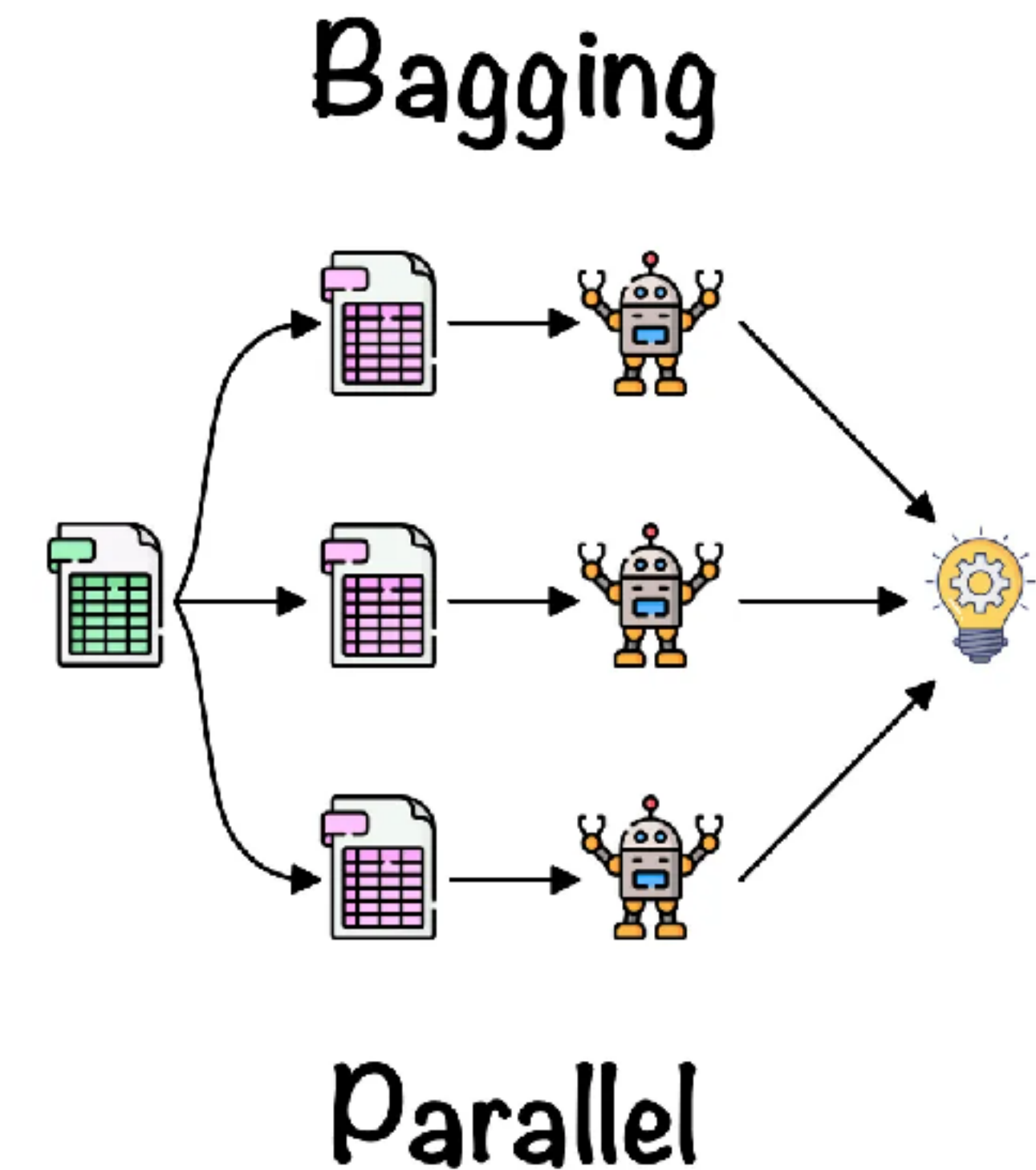
Ensemble learning

Types



Bagging (Bootstrap Aggregating)

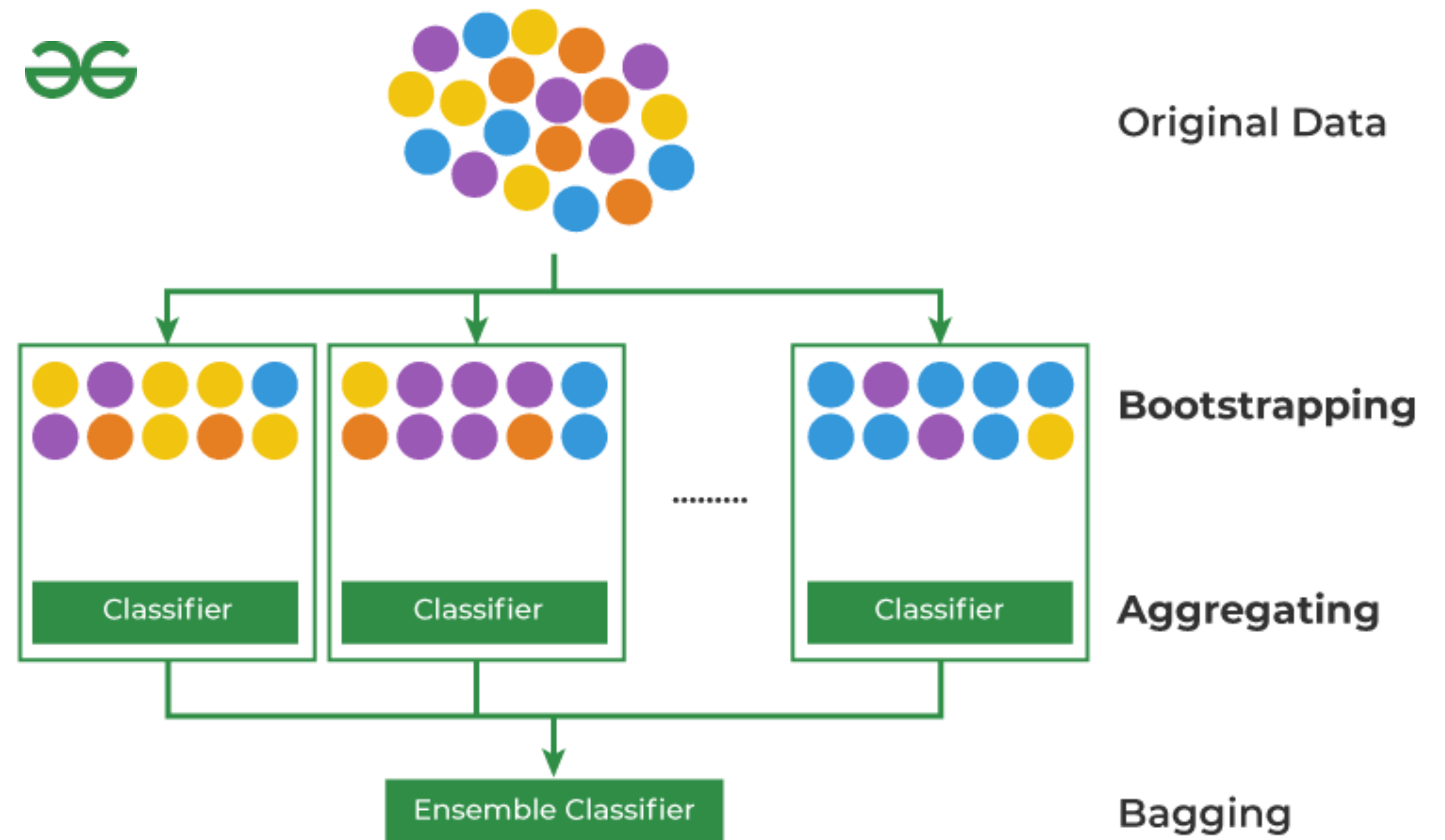
- **Multiple base models** are trained independently and in parallel on different **subsets of the training data**.
- Each subset is generated using **bootstrap sampling**, in which data points are picked at **random with replacement**.
- In the case of the **bagging classifier**, the final prediction is made by **aggregating** the predictions of the all-base model using **majority voting**.
- In the models of **regression**, the final prediction is made by **averaging** the **predictions** of the all-base model, and that is known as **bagging regression**



Bagging (Bootstrap Aggregating)

Bootstrap Sampling

- Original training dataset:
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
- Resampled training set 1:
[2, 3, 3, 5, 6, 1, 8, 10, 9, 1]
- Resampled training set 2:
[1, 1, 5, 6, 3, 8, 9, 10, 2, 7]
- Resampled training set 3:
[1, 5, 8, 9, 2, 10, 9, 7, 5, 4]



Bagging (Bootstrap Aggregating) Algorithm



Classifier generation:

Let N be the size of the training set.

for each of t iterations:

 sample N instances with replacement from the original training set.

 apply the learning algorithm to the sample.

 store the resulting classifier.

Classification:

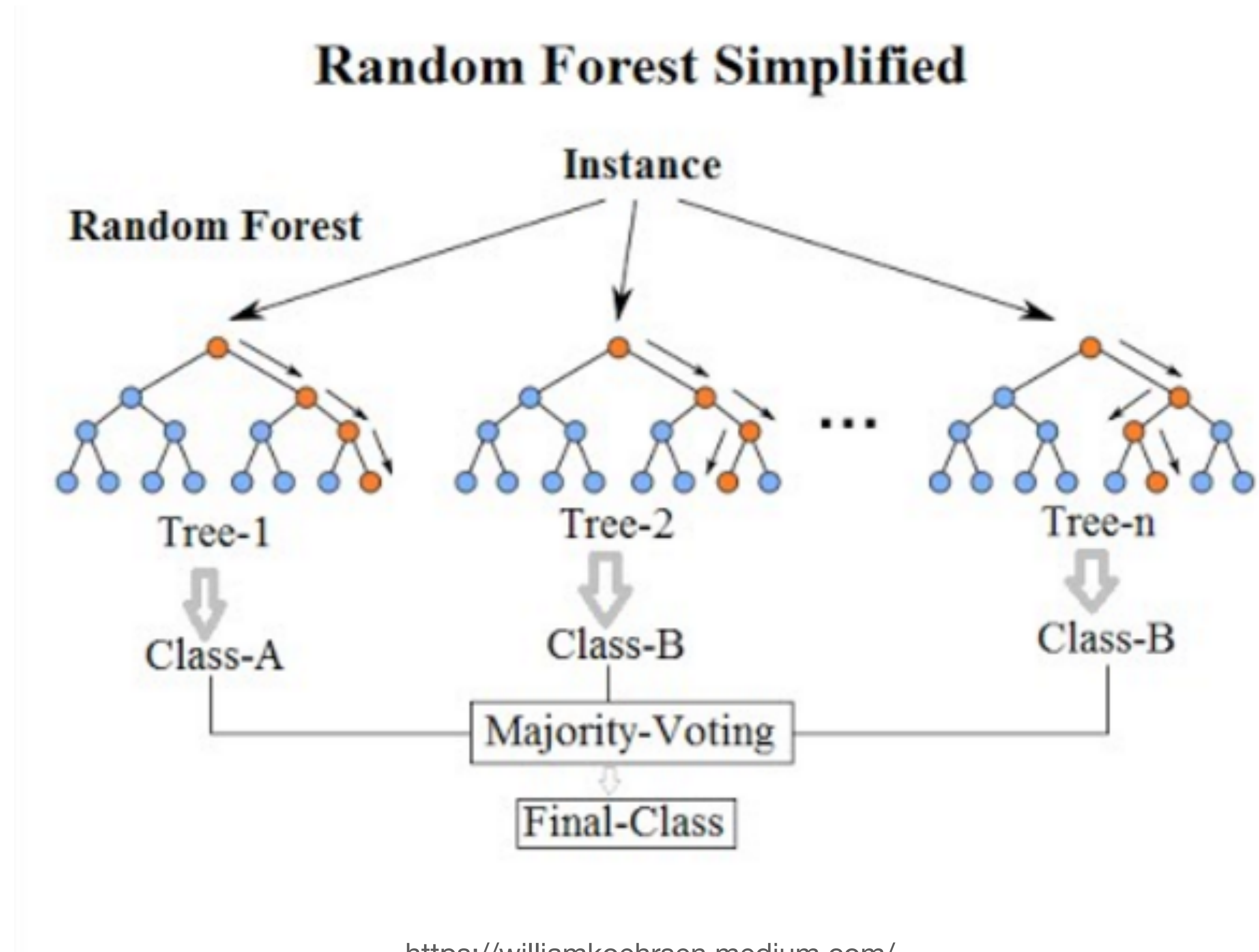
for each of the t classifiers:

 predict class of instance using classifier.

return class that was predicted most often

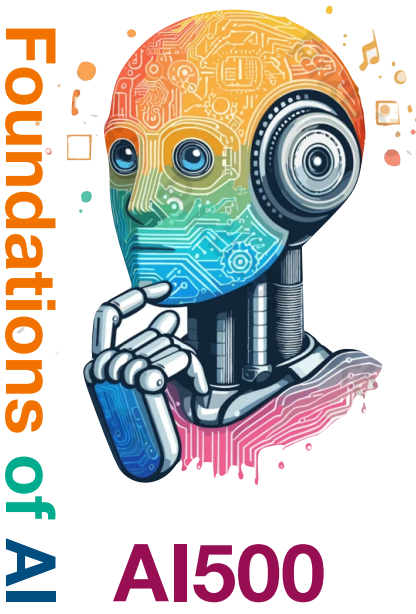
Random Forest

We sample the dataset into subsets that are randomly created



Random Forest

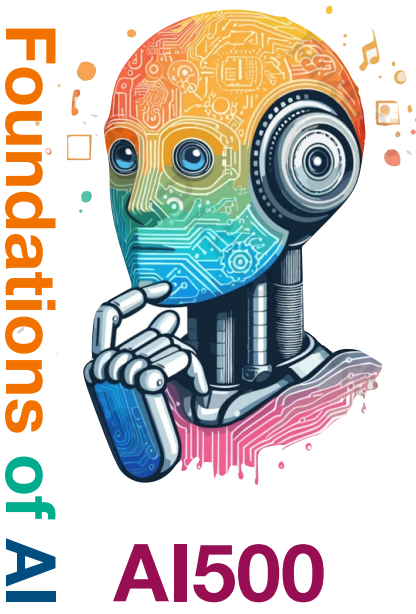
Training Steps



- Given the whole dataset:
 - Get a **subset** of the **dataset**
 - Create a **decision tree**
- **Repeat** for as many times as the number of trees

Random Forest

Testing Steps



- Given a data point:
 - Get the predictions from each tree
 - **Classification**: hold a majority vote
 - **Regression**: get the mean of the predictions

Random Forest

Python Implementation - Training

- Given the whole dataset:
 - Get a **subset** of the **dataset**
 - Create a **decision tree**
 - **Repeat** for as many times as the number of trees

```
function bootstrap(X,y)
```

```
    idxs = randomly choose n samples from X,y
```

```
    X_s = X[idxs], y_s = y[idxs]
```

```
    return X_s, y_s
```

```
class RandomForest
```

```
    trees
```

```
    n_trees
```

```
    max_depth
```

```
    min_samples_split
```

```
    n_features
```

```
function fit(X,y)
```

```
    for n_trees
```

```
        tree = DecisionTree(..)
```

```
        X_s, y_s = bootstrap(X,y)
```

```
        tree.fit(X_s,y_s)
```

```
    trees.append(tree)
```

Random Forest

Python Implementation - Testing

- Given a data point:
 - Get the predictions from each tree
 - **Classification**: hold a majority vote
 - **Regression**: get the mean of the predictions

```
function predict(X)
for each ith tree in trees
    pred[i] = tree.predict(X)

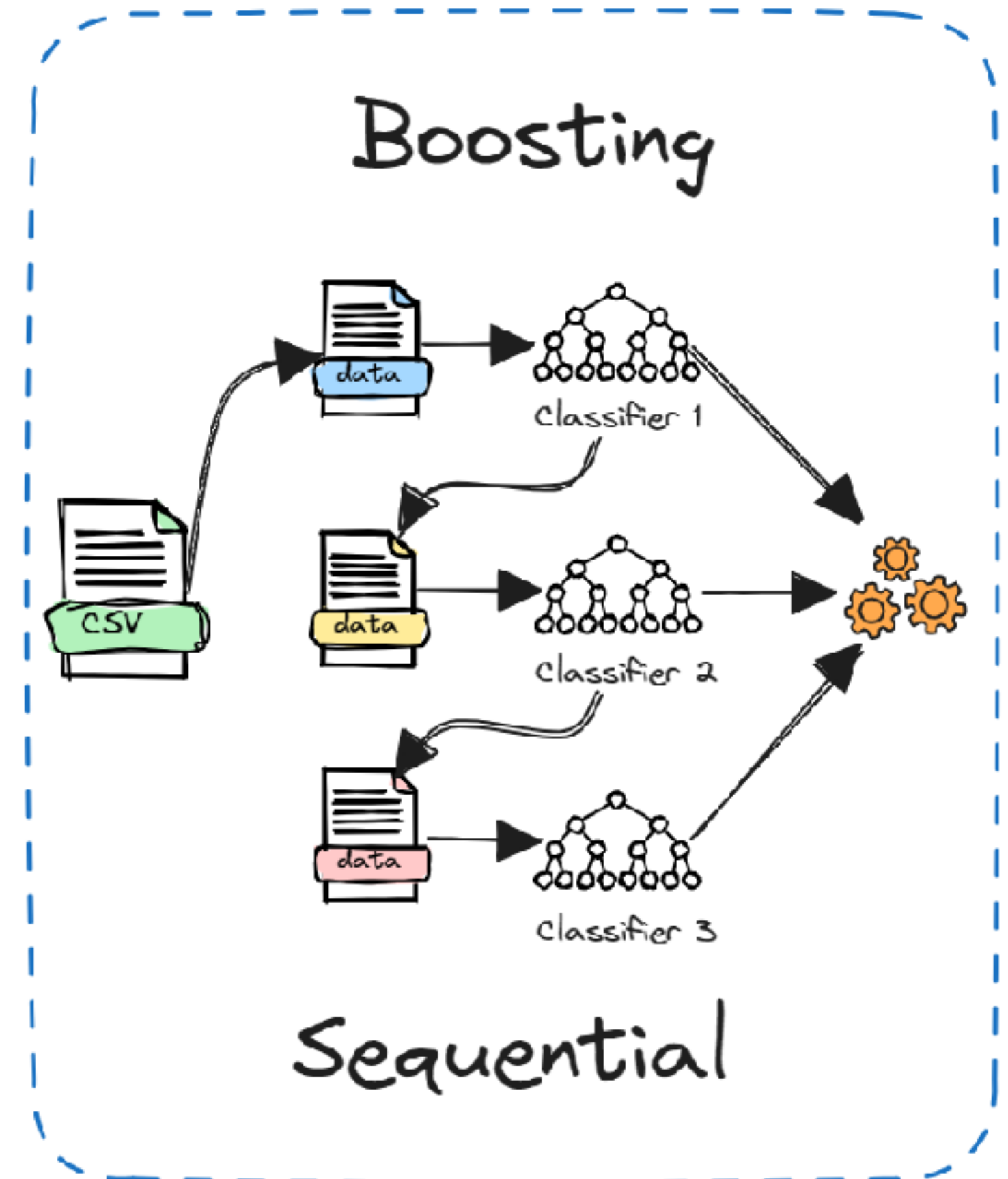
for each sample
    preds = get most common label

return preds
```

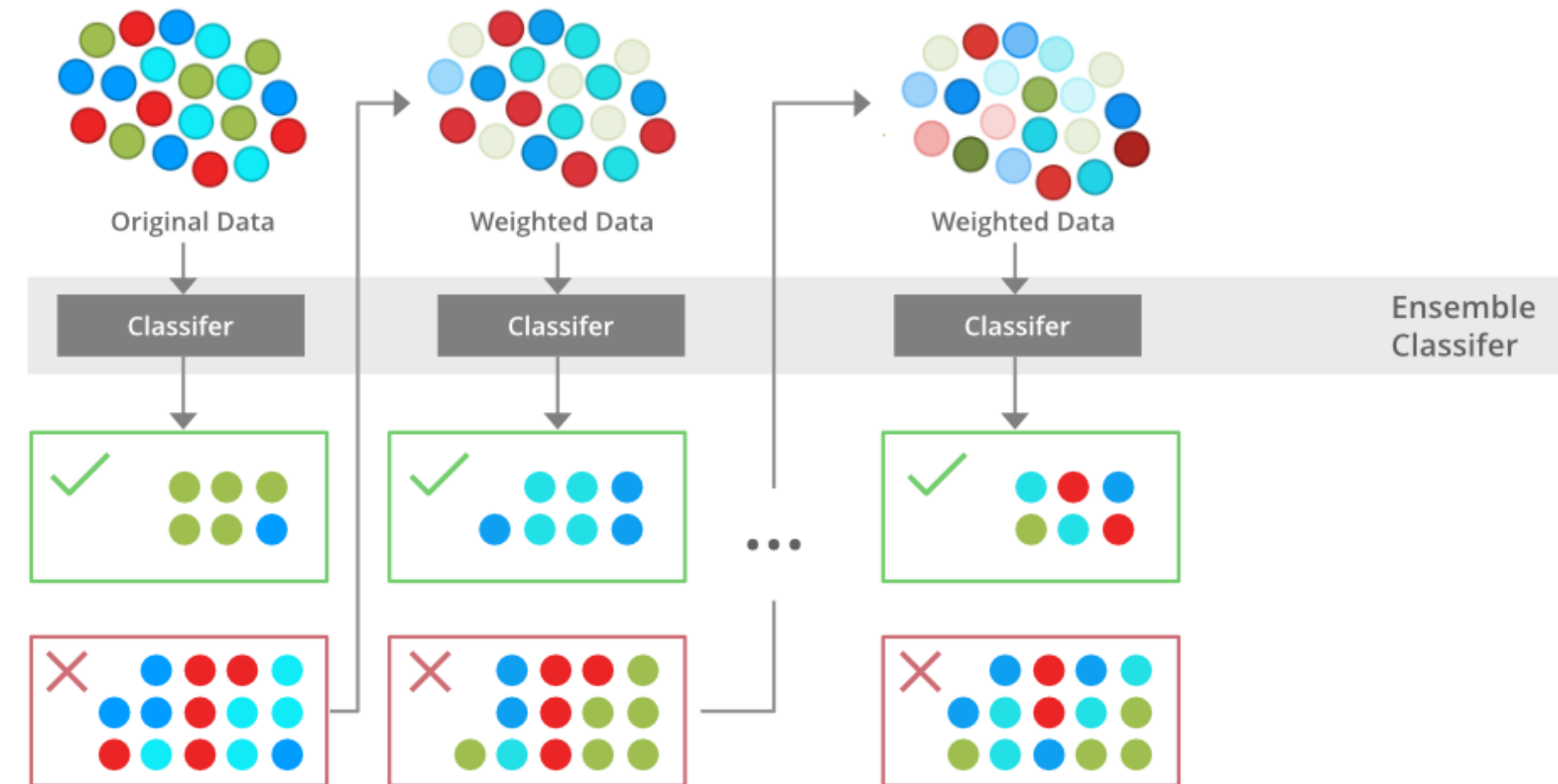
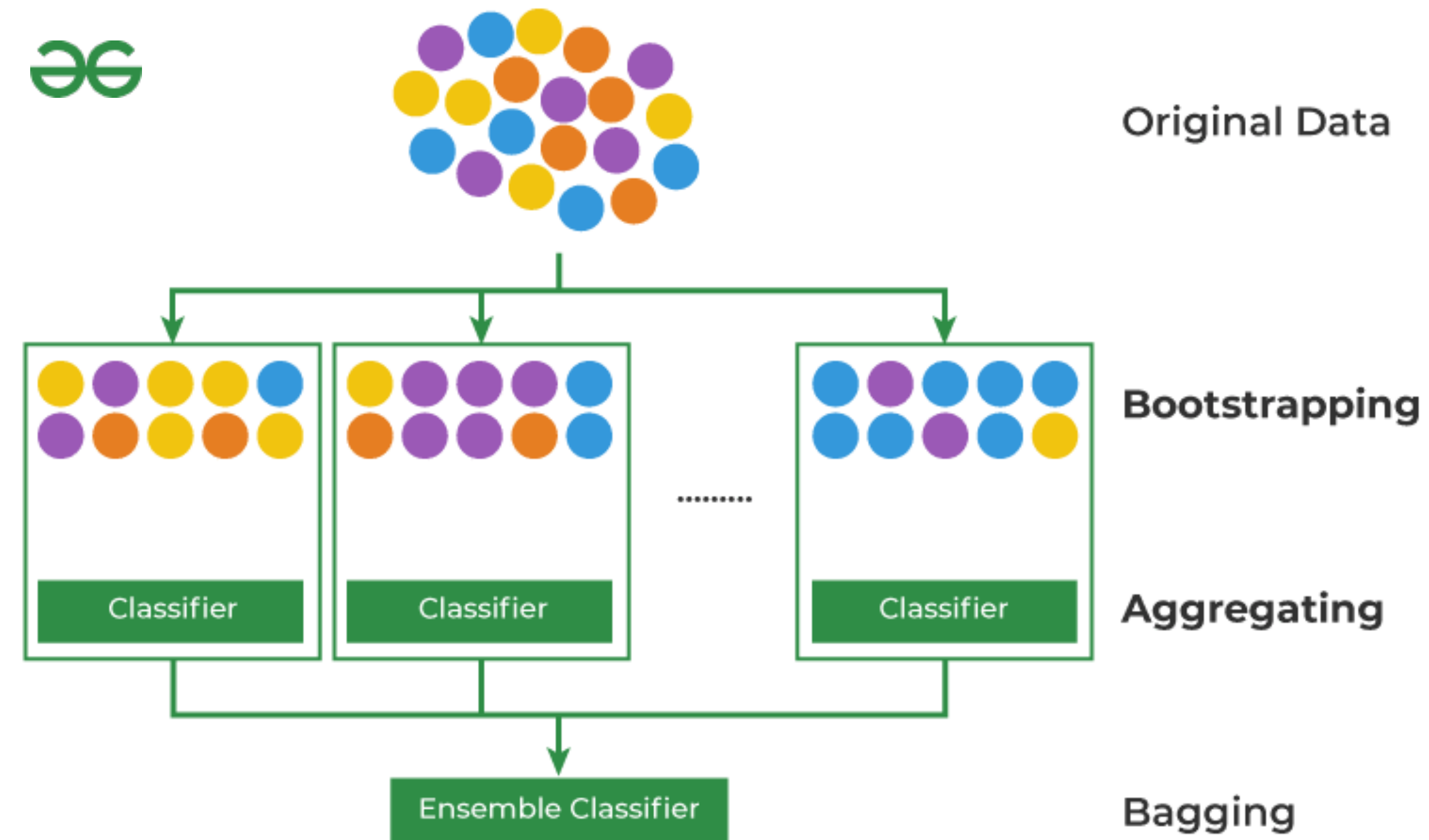
```
function most_common_label(preds)
    count each value
    return label with highest count
```

Boosting

- It is done by building a model by using weak models in **series**.
- Firstly, a model is built from the training data.
- Then the second model is built which tries to **correct the errors** present in the first model.
- This procedure is continued and models are added until either the complete training data set is predicted correctly or the maximum number of models are added.

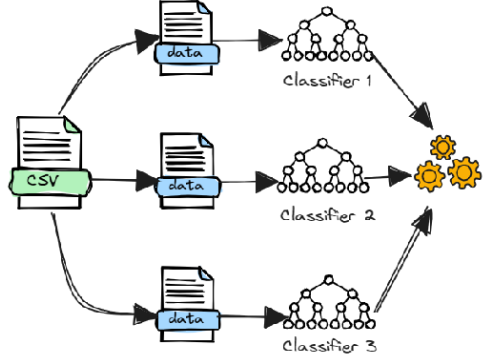
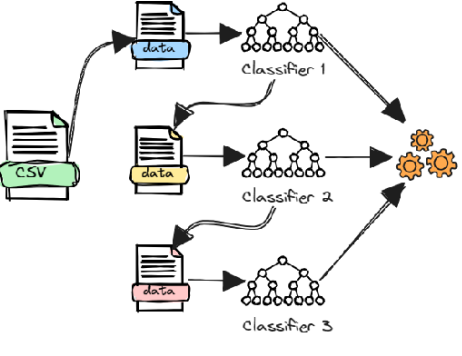


Bagging vs. Boosting



Bagging vs. Boosting



Bagging 	Boosting 
method of combining the same type of prediction	we combine predictions that belong to different types
decrease variance not bias	decrease bias, not variance
All the models have the same weightage	At every successive layer Models are weighted according to their performance.
All the models are independent of each other	New Models are influenced by the accuracy of previous Models

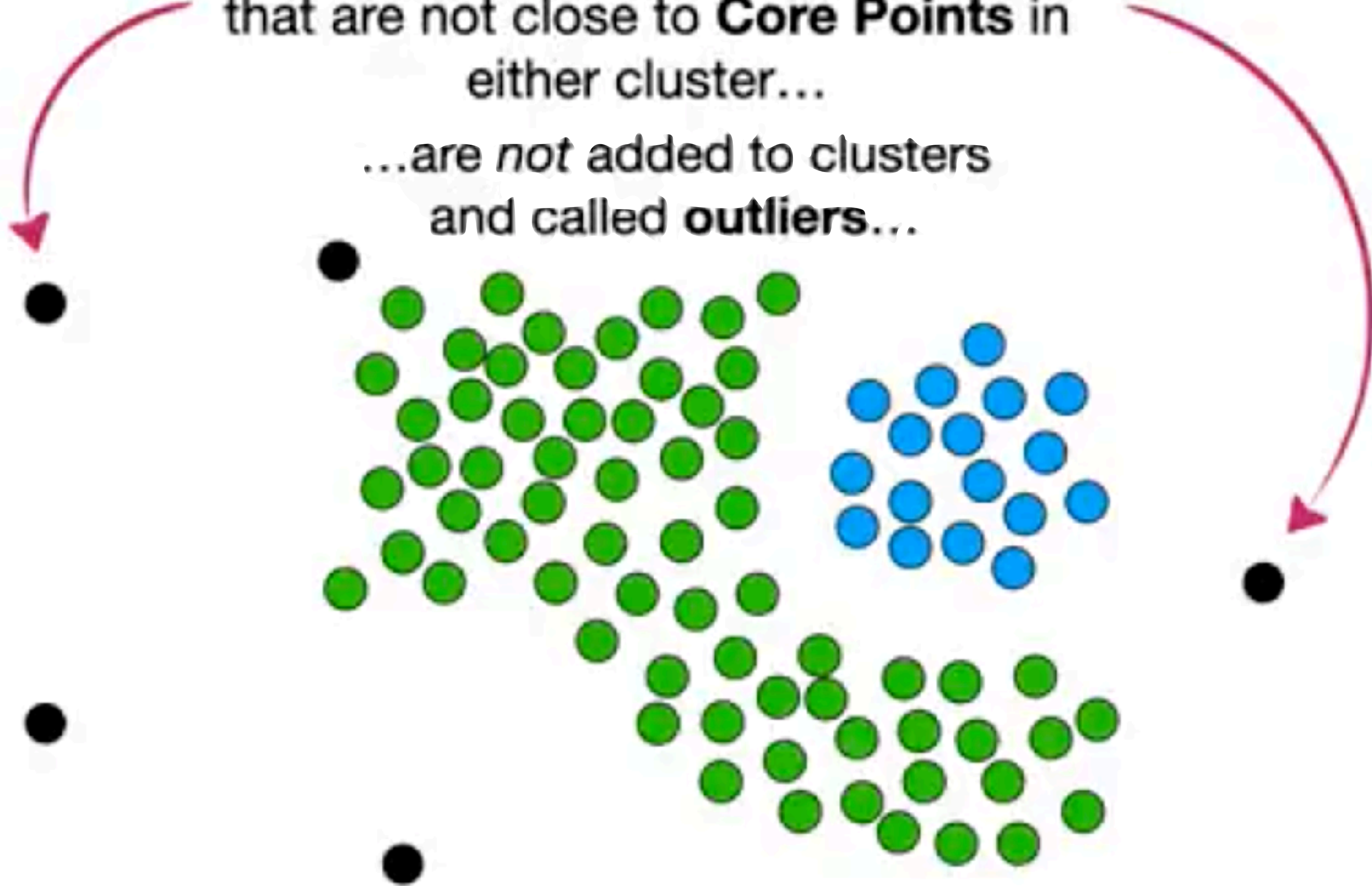
Isolated Forest Anomaly Detection

Photo by [Rupert Britton](#) on [Unsplash](#)

Recap: Outlier Detection

...and any remaining **Non-Core Points**
that are not close to **Core Points** in
either cluster...

...are *not* added to clusters
and called **outliers**...



Outliers/Anomalies Detection



- The detection of anomalies has significant relevance and often provides critical actionable information in various application domains.
 - Fraudulent transactions/Fraudulent use of credit cards
 - Wrong way traffic
 - Unusual computer network traffic
 - Suspicious activity after working hours
 - etc

Isolated Forest (iForest)

Introduction



- Anomalies are data patterns that have different data characteristics from normal instances
- Most existing model-based approaches to anomaly detection construct a **profile of normal instances**, then identify **instances that do not conform to the normal profile** as **anomalies**
- However there are two major **drawbacks** of this approach
 - The **anomaly detector** is optimized to profile normal instances, but **not optimized to detect anomalies** which causes too many **false alarms**
 - Many existing methods are constrained to **low dimensional data** and small data size because of their **high computational complexity**

Isolated Forest (iForest)

Proposed by Fei Tony Liu, Kai Ming Ting and Zhi-Hua Zhou in 2008

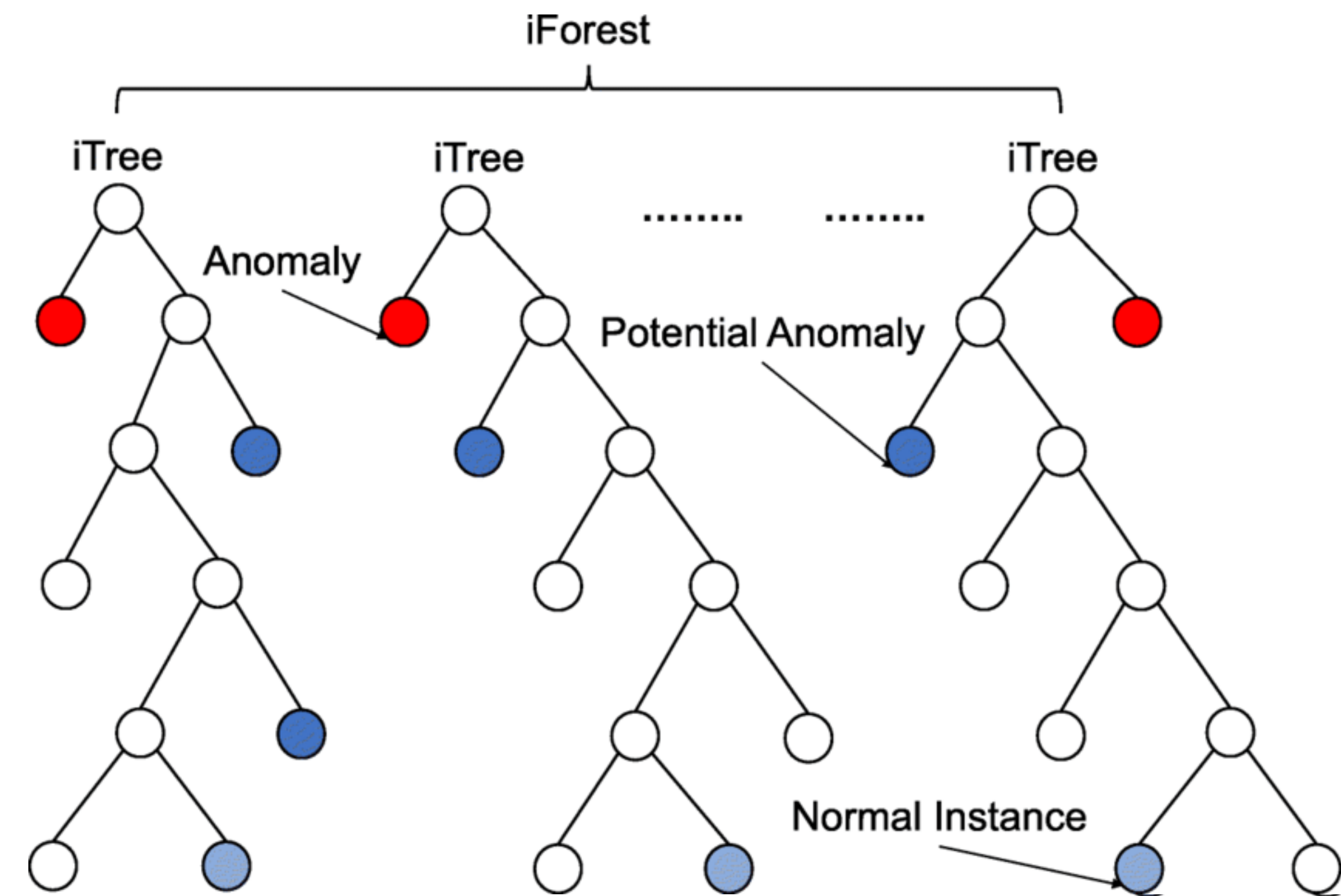


- iForest proposes a different type of model-based method that **explicitly isolates anomalies** rather than profiles normal instances
 - iForest utilizes **no distance or density measures** to detect anomalies. This **eliminates major computational cost** of distance calculation
 - iForest has a **linear time complexity** with a low constant and a **low memory requirement**
 - iForest has the **capacity to scale** up to handle extremely large data size and high-dimensional problems with a large number of irrelevant attributes
- Assume:
 - 1 Anomalies are the **minority** consisting of fewer instances
 - 2 Anomalies have **attribute-values** that are very **different** from those of normal instances

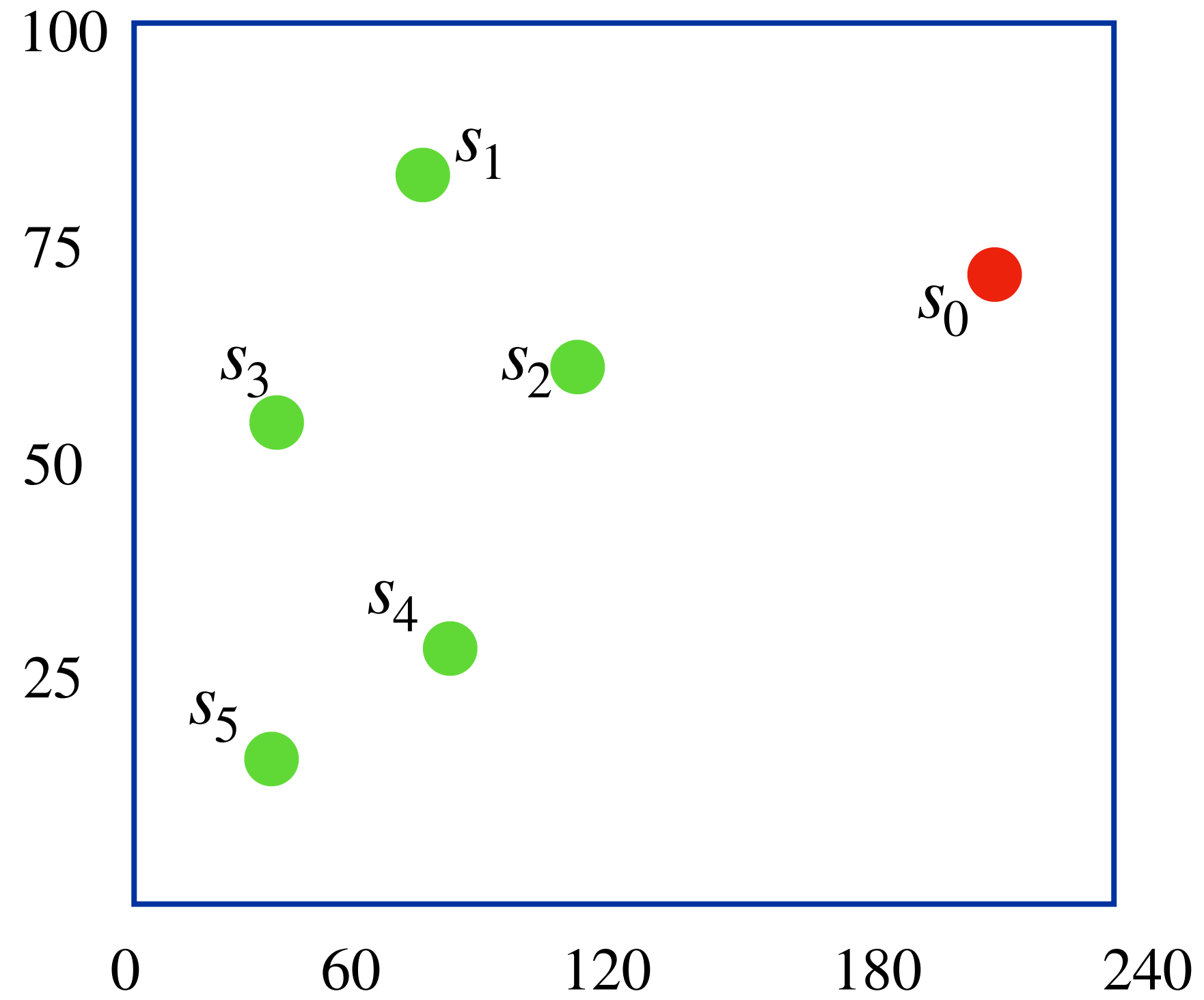
iForest

Basic Idea

- Construct an unsupervised decision tree which **randomly partition** the sample space with **binary split** until every single instance is isolated. Call this tree as **iTree**
- Because of their susceptibility to isolation, **anomalies will be isolated closer to the root of the tree**; whereas normal points will be isolated at the deeper end of the tree
- Build an **iForest** which is an **ensemble of iTrees** for sub-sampled, **bootstrapped without replacement**, data sets
- Calculate the **average path length** of each data point and get **anomaly score** by using some metric
- Identify data points as **anomalies** which have **high anomaly score**



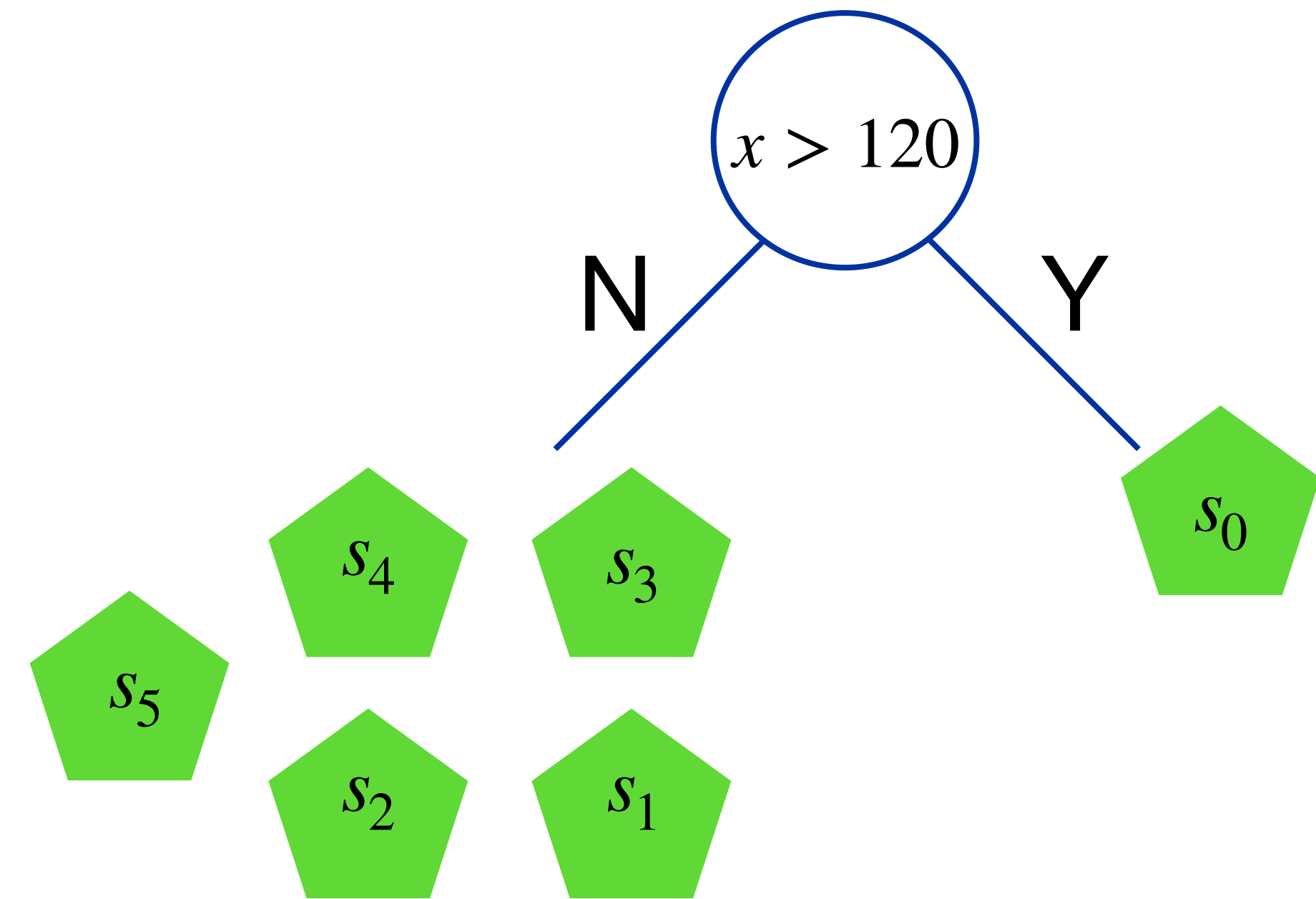
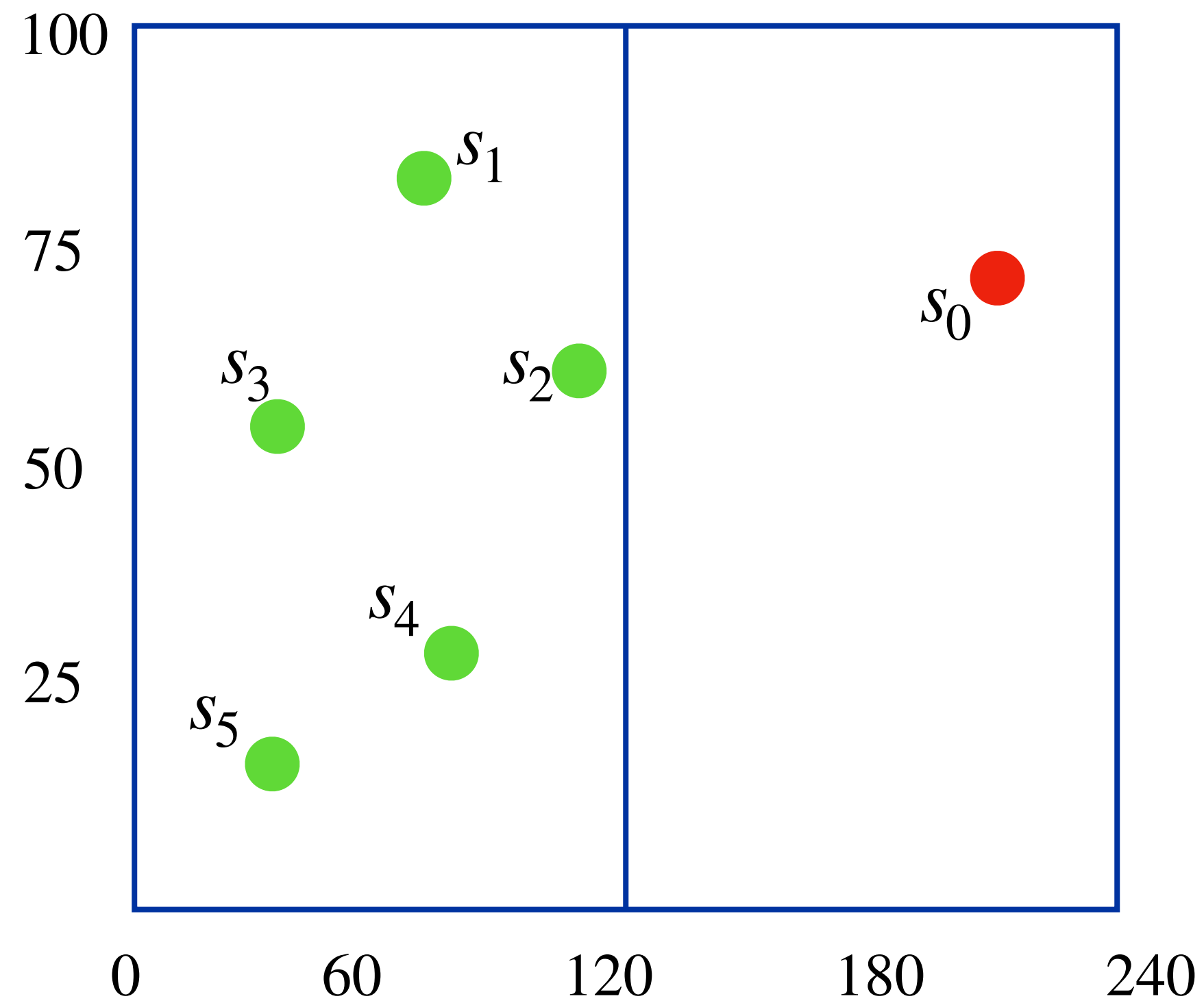
iForest



iForest

Attribute q : x axis

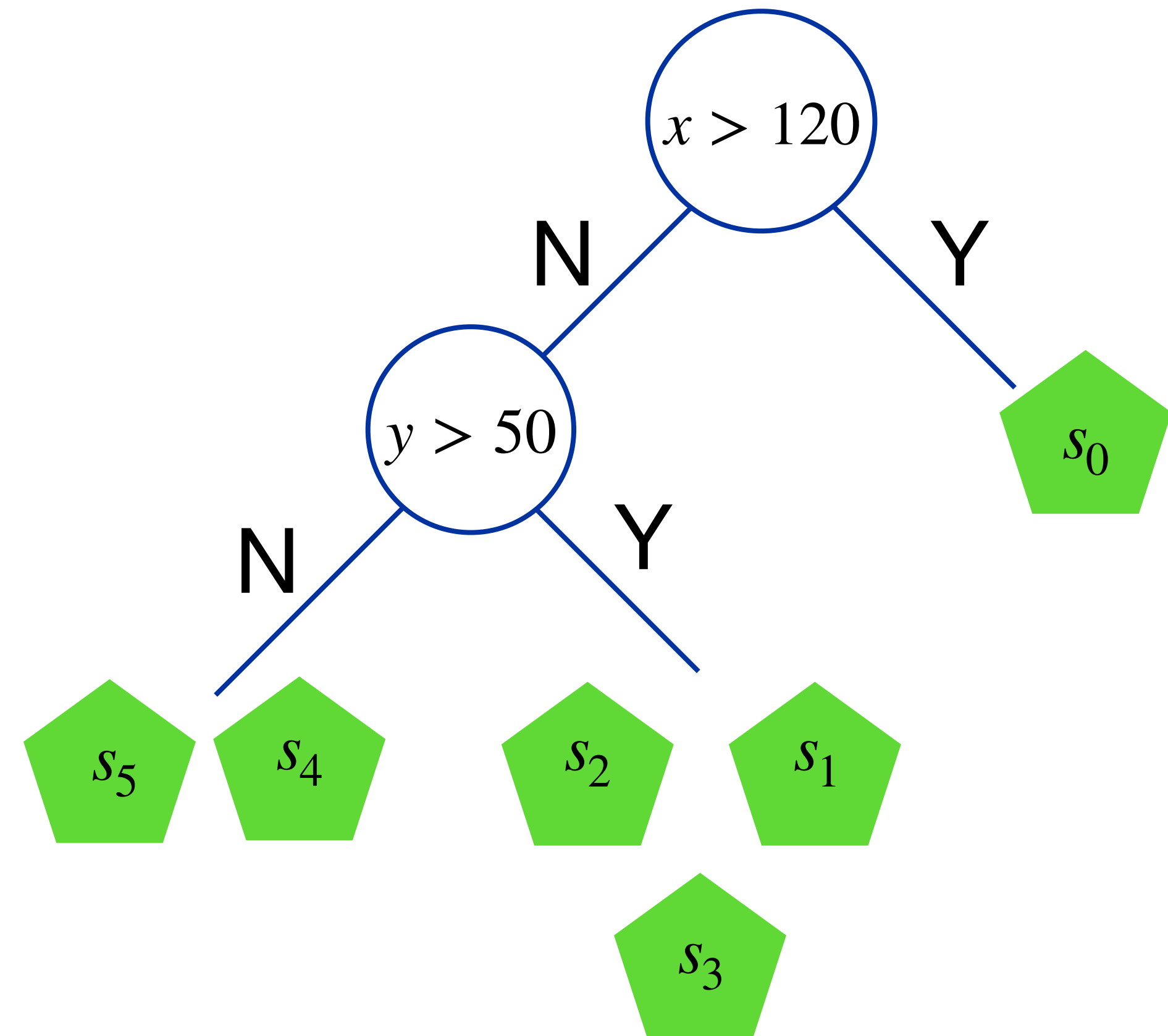
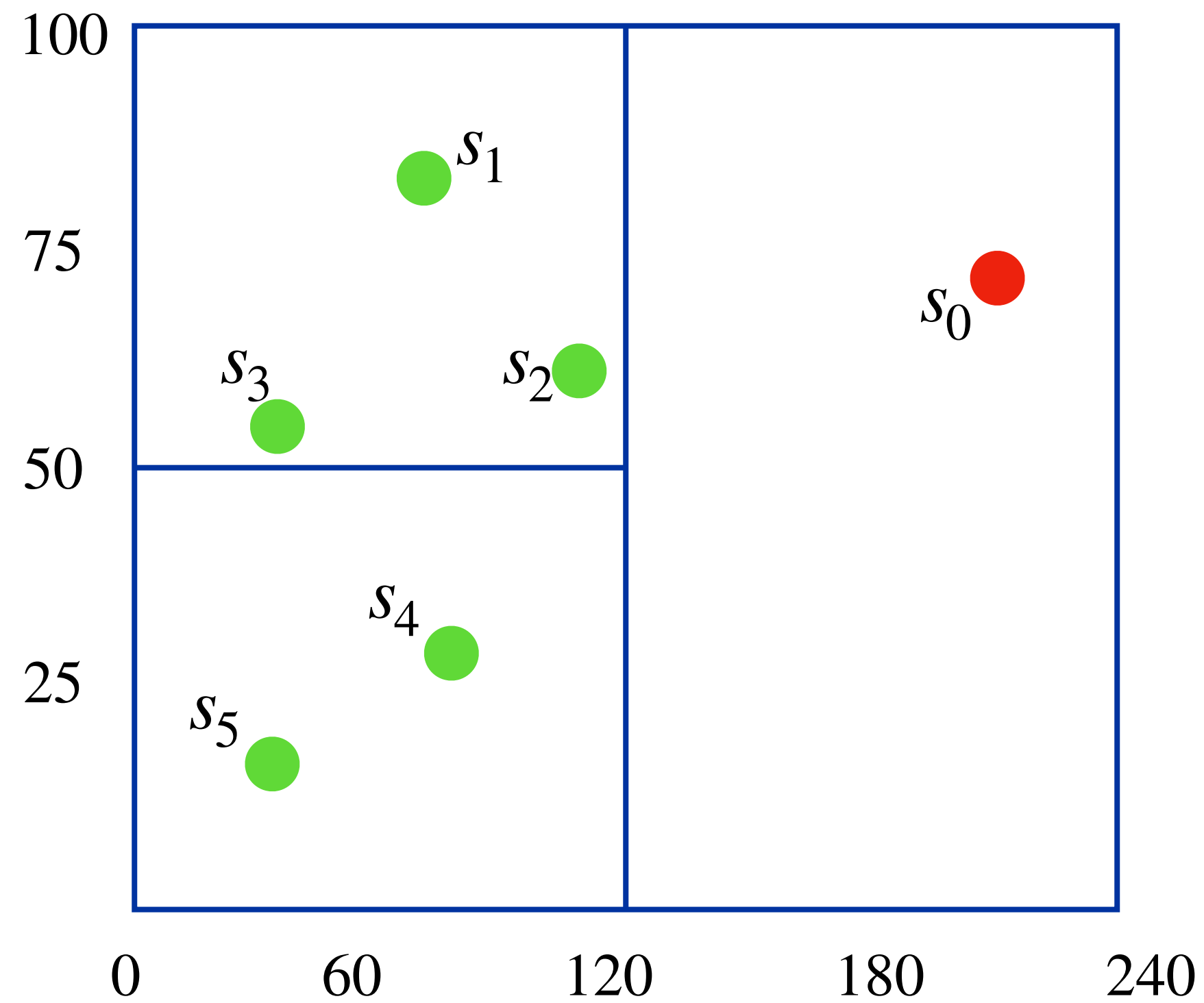
Split value: $p = 120$



iForest

Attribute q : y axis

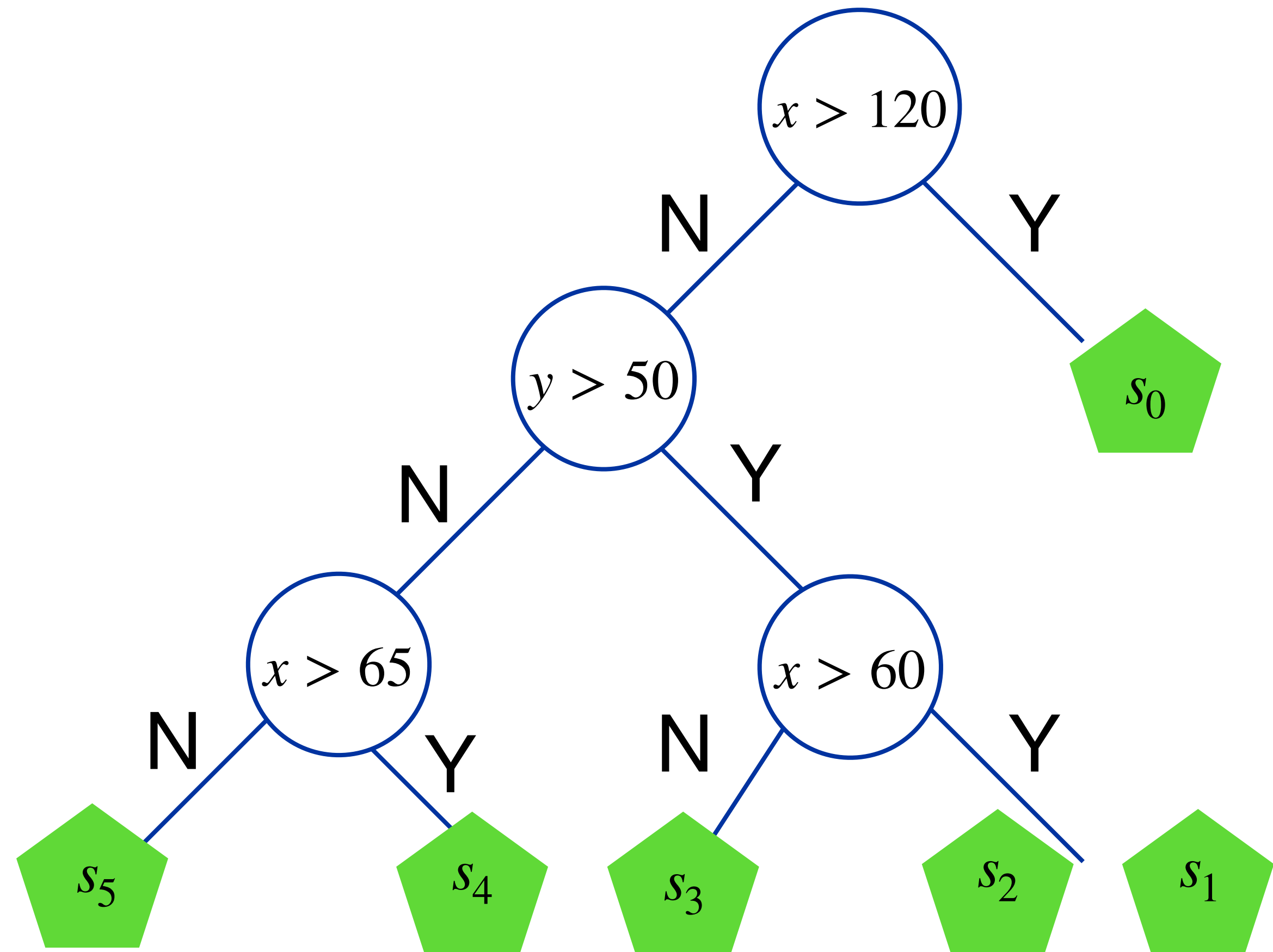
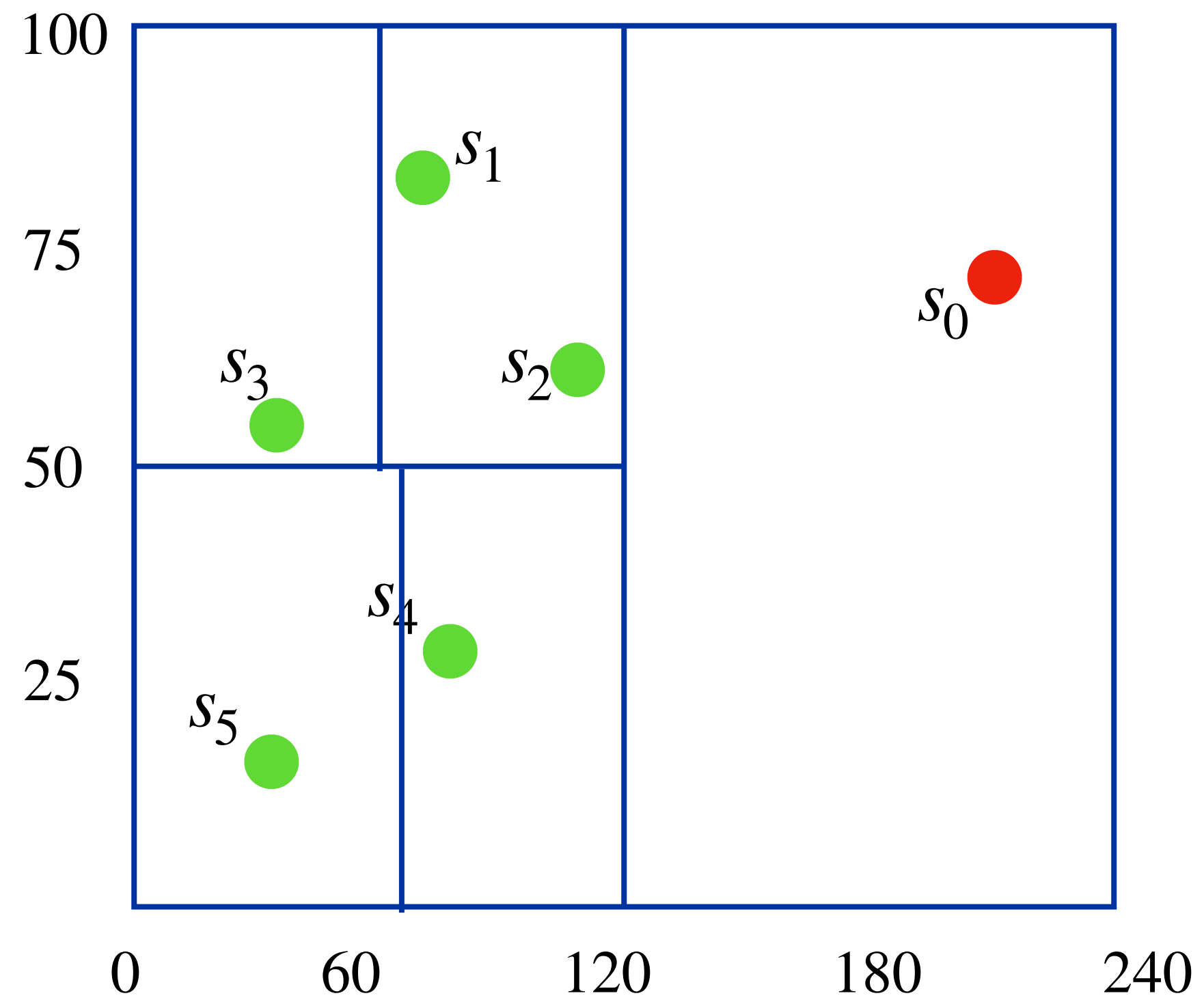
Split value: $p = 50$



iForest

Attribute q : x axis

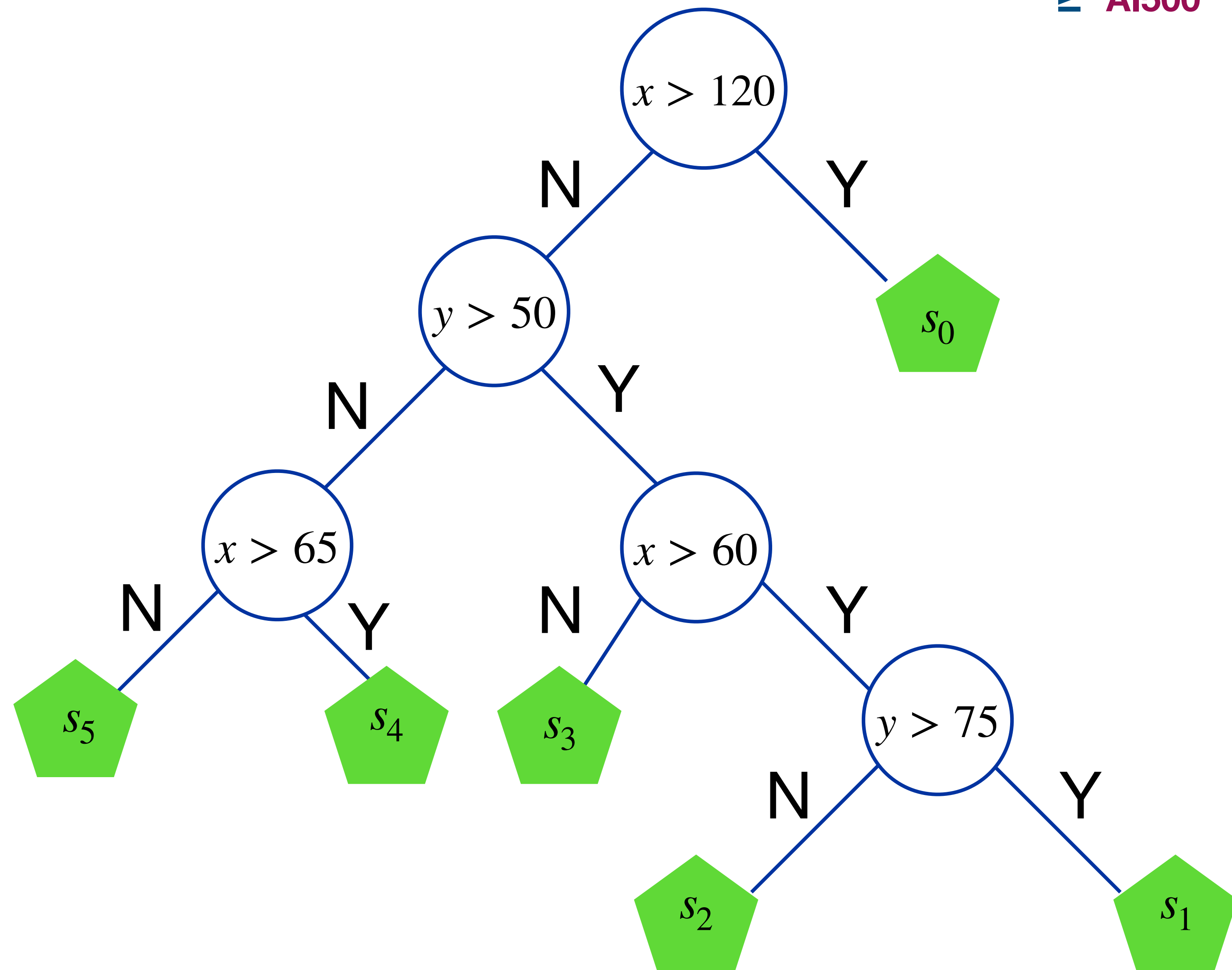
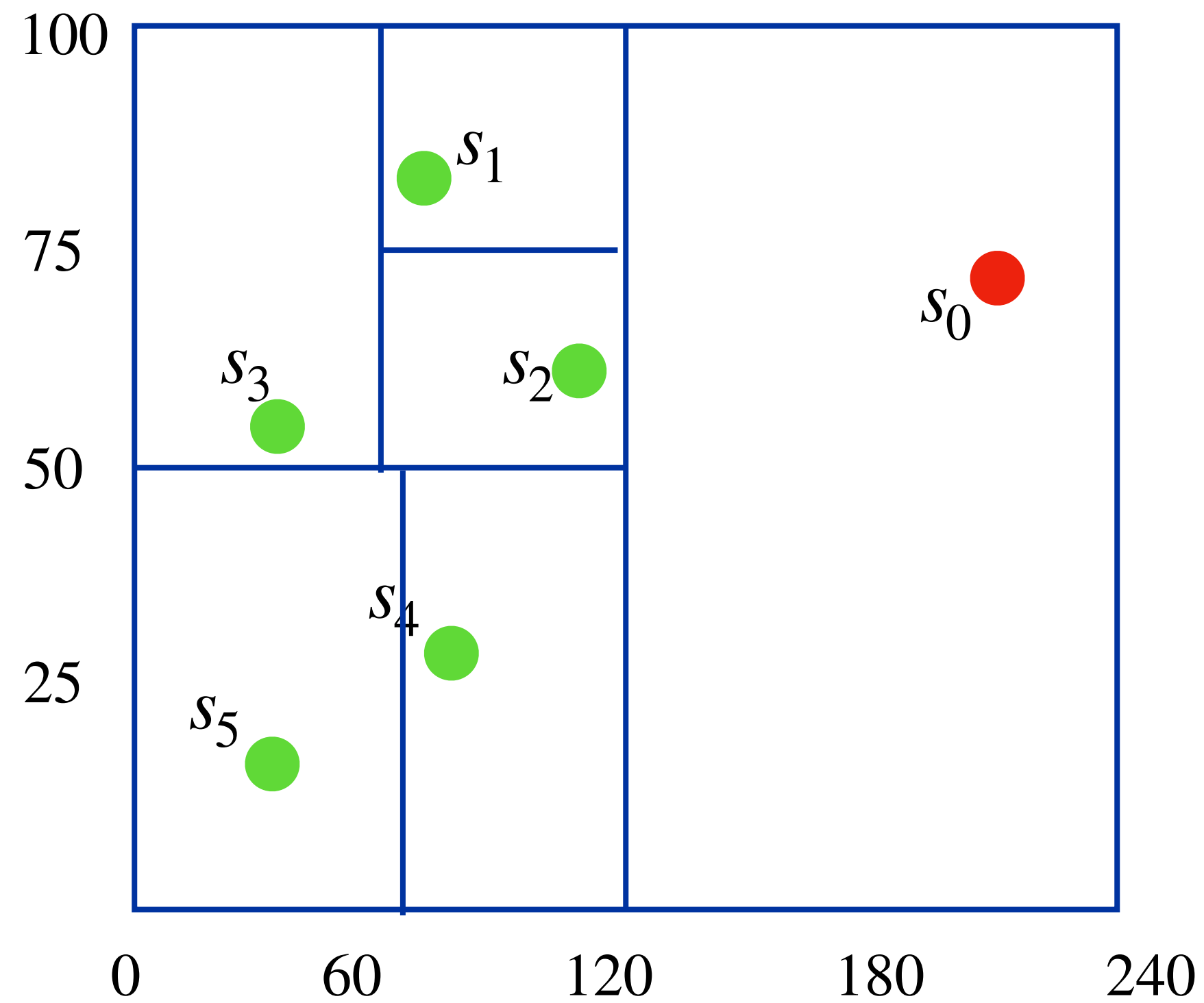
Split value: $p = 60$



iForest

Attribute q : y axis

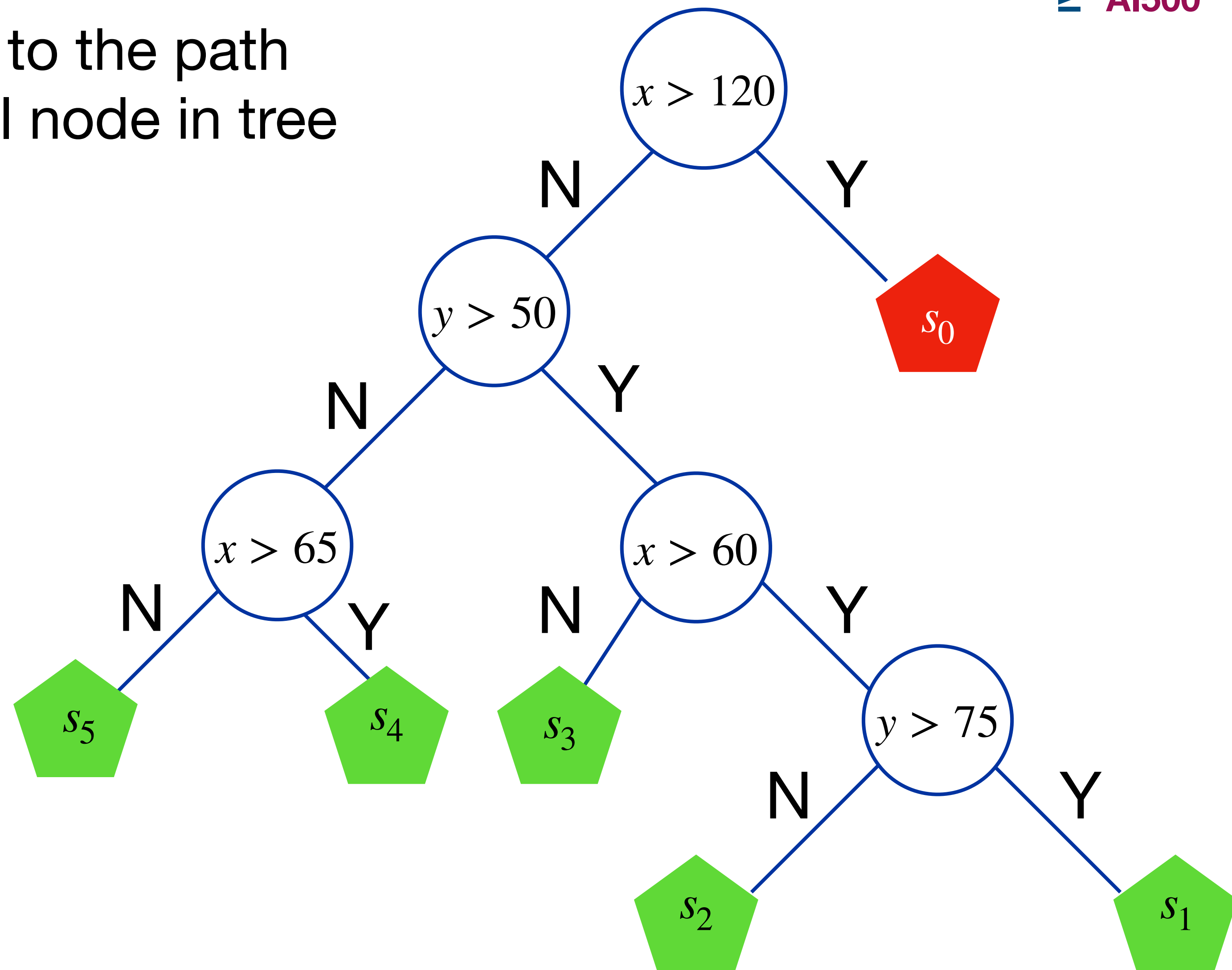
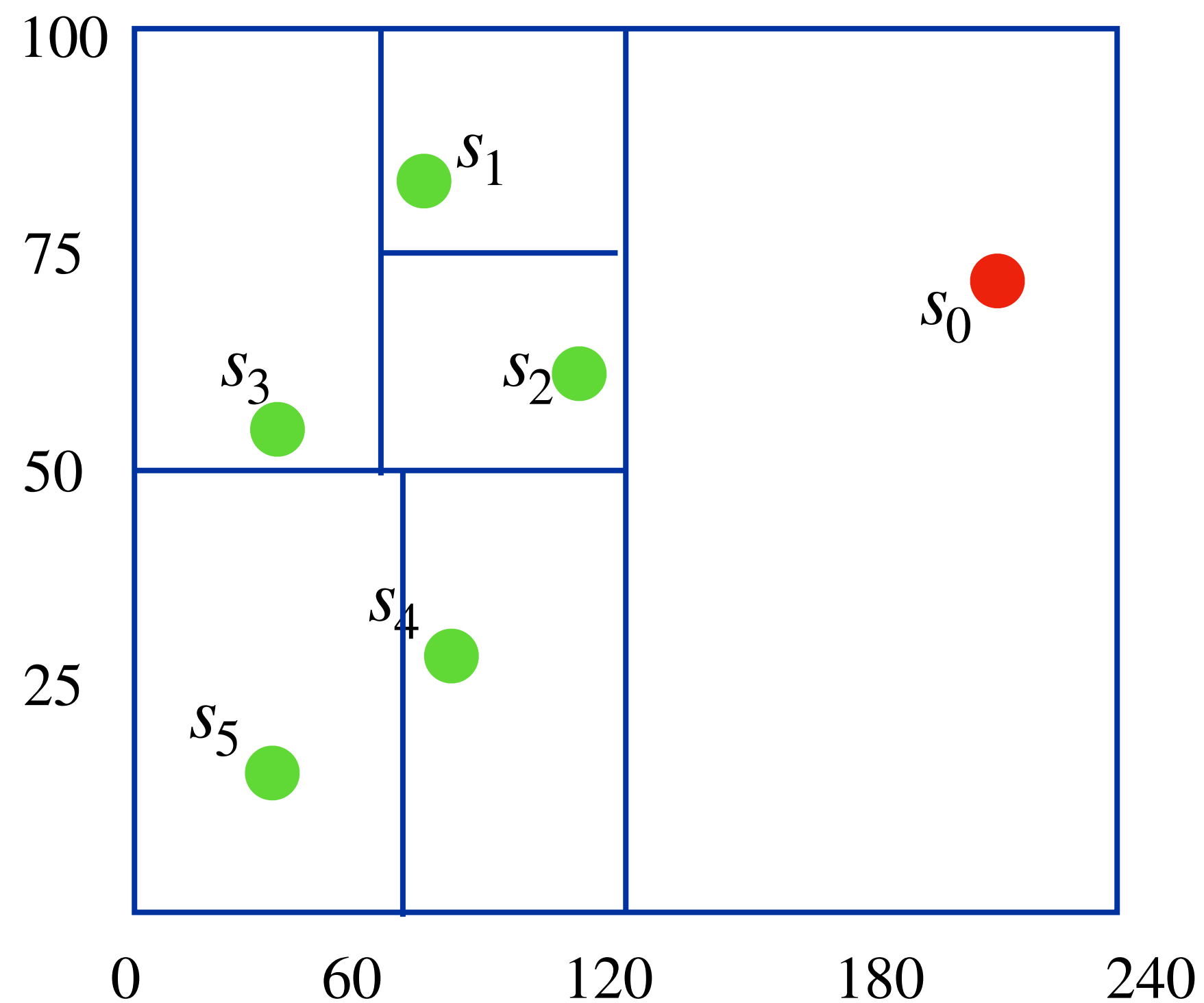
Split value: $p = 75$



iForest

Anomalies are more susceptible to **isolation** and hence have **short path lengths**

The number of partitions is equivalent to the path length from the root node to a terminal node in tree



iForest

Notation



- $X = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}, \quad \mathbf{x}_i \in \mathbb{R} \quad \forall i = 1, \dots, N$
 - (Only consider continuous values attributes)
- T is a node of an isolation tree
 - T is either an **terminal node** with no child, or an internal-node with one test and exactly two daughter nodes (T_l, T_r)
 - A test consists of an attribute q and a split value p such that the test $q < p$ **divides data points** into T_l and T_r
 - $h(\mathbf{x})$: **Path length of a point $\mathbf{x}$** which is measure by the **number of edges $\mathbf{x}$ traverses** an iTree from the root node until the traversal is terminated at tan external node
 - $c(n)$: **Average path length** of unsuccessful search in BST (Binary Search Tree). This is used for normalization of $h(\mathbf{x})$, $(H(i) \approx \ln(i) + 0.5772156649)$

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n}$$

iForest

Notation

- $s(\mathbf{x}, n)$: Anomaly score

$$s(\mathbf{x}, n) = 2^{-\frac{E(h(\mathbf{x}))}{c(n)}}$$

- where $E(h(\mathbf{x})) \rightarrow c(n)$, $s \rightarrow 0.5$
- where $E(h(\mathbf{x})) \rightarrow 0$, $s \rightarrow 1$
- where $E(h(\mathbf{x})) \rightarrow n - 1$, $s \rightarrow 0$
- $\Rightarrow s$ is monotonic to $h(\mathbf{x})$

- $0 < s \leq 1$ for $0 < h(\mathbf{x}) \leq n - 1$
- If instances return s very close to 1, they are definitely **anomalies**
- If instance have s much smaller than 0.5, then they are quite safe to be regarded as **normal instances**
- If all instances return $s \approx 0.5$, then the **entire sample does not have any distinct anomaly**

Isolated Forest (iForest)

Training/Testing

- Anomaly detection using iForest is a **two-stage process**
- The first (**training**) stage builds isolation trees using sub-samples, bootstrapped sample without replacement, of the training set
- The second (**testing/evaluating**) stage passes the test instances through isolation trees to obtain an anomaly score for each instance

Isolated Forest (iForest)

Training Stage

Algorithm 1: $iForest(X, t, \psi)$

Inputs : X - input data, t - number of trees, ψ - sub-sampling size

Outputs: a set of t $iTrees$

begin

 set height limit $l = \text{ceiling}(\log_2 \psi)$;

for $i = 1$ to t **do**

$X' \leftarrow \text{sample}(X, \psi)$;

$\text{Forest} \leftarrow \text{Forest} \cup iTree(X', 0, l)$;

end

return Forest

end

Isolated Forest (iForest)

Training Stage

Algorithm 2: $iTree(X, e, l)$

Inputs : X - input data, e - current tree height, l - height limit

Outputs: an iTree

begin

if $e \geq l$ or $|X| \leq 1$ **then**

 return $exNode\{Size \leftarrow |X|\}$;

else

 let Q be a list of attributes in X

 randomly select an attribute $q \in Q$

 randomly select a split point p from max and min values of attribute q in X

$X_l \leftarrow filter(X, q < p)$

$X_r \leftarrow filter(X, q \leq p)$

 return $inNode\{Left \leftarrow iTree(X_l, e + 1, l),$

$Right \leftarrow iTree(X_r, e + 1, l),$

$SplitAtt \leftarrow q,$

$SplitValue \leftarrow p\}$

end

end

Isolated Forest (iForest)

Evaluating Stage



- An **anomaly score** s is derived from the **expected path length** $E(h(\mathbf{x}))$ for each test instances
- $E(h(\mathbf{x}))$ are derived by passing instances through each iTree in an iForest
- A **single path length** $h(\mathbf{x})$ is derived by counting the number of edges e from the root node to a terminating node
- When $h(\mathbf{x})$ is obtained for each tree of the ensemble, an **anomaly score** is produced by computing $s(\mathbf{x}, n) = 2^{-\frac{E(h(\mathbf{x}))}{c(n)}}$
- To find the top m anomalies, simply sorts the data using s in descending order. The first m instances are the top m anomalies
- The **complexity** of the evaluation process is $O(\psi t \log n)$, where n is the testing data size

Isolated Forest (iForest)

Evaluating Stage

Algorithm 3: $PathLength(x, T, e)$

Inputs : x - an instance, T - an iTree, e - current path length;
to be initialized to zero when first called

Outputs: path length of x

begin

if T is an external node **then**

 | return $e + c(T.size)$ { $c(.)$ is defined in Equation 1}

end

$a \leftarrow T.splitAtt;$

if $x_a < T.splitValue$ **then**

 return $PathLength(x, T.left, e + 1);$

else if $\{x_a \geq T.splitValue\}$ **then**

 | return $PathLength(x, T.right, e + 1)$

end

end

Next...Support Vector Machines

